The Australian National University

School of Computing

JHQ-GPU

Representation-Aware GPU ANN Search and Workload-Level Refinement Calibration

Rui Luo

Supervisor: Dr Mengxuan Zhang

A research thesis submitted for

COMP8800 — Advanced Computing Research Project

October 2026

Abstract

Reducing approximate nearest-neighbour (ANN) query time requires efficient screening and selective refinement. This thesis presents JHQ-GPU, which implements JHQ through representation-aware execution, GPU index construction and workload-level budget calibration. Cartesian codebook factorisation replaces each 256-entry distance table with two 16-entry tables, preserving the real-arithmetic represented distance while reducing table storage eightfold. Candidate-parallel scanning, packed access and lane-cooperative refinement complement this design. Refinement budgets are calibrated through sampled top-k agreement with a larger reference budget, without ground-truth labels or a learned predictor.

Experiments on an RTX 5090 cover six datasets. Two-group tables lead in 22/24 configurations; the largest index searches 17.78 million vectors at 20.44 GiB resident memory. Full-pool calibration meets a 10^{-3} held-out recall-loss target relative to the best tested budget in 12/12 cells, with maximum loss 0.0006; smaller samples expose workload-dependent quality risk. At recall 0.97 and batch 1,000, JHQ leads in three of four datasets and is near parity with IVF-RaBitQ on arXiv. OpenAI runs use TF32; FP32 checks retain favourable comparisons under a TF32-trained cache. At batch 8 and the same recall, CAGRA fp32 leads on all four datasets. These results locate the execution, quality and capacity trade-offs of GPU JHQ.

Acknowledgements

I thank Dr Mengxuan Zhang for supervising this project and acknowledge the JHQ authors and open-source ANN/GPU software cited throughout this thesis.

Research and AI assistance. This thesis extends joint JHQ-GPU work with Mengxuan Zhang [21] and builds on JHQ [16]. OpenAI Codex assisted with substantive drafting, revision, LaTeX, analysis and code/log checks [30]; ChatGPT supplied frameworks and reviews [29]; Anthropic Claude assisted with implementation, experiments and review [4]. Selected prompts and usage details are available in the AI-use record. The author is responsible for the submitted claims.

Contents

Abstract i

Acknowledgements i

Notation and Abbreviations vi

1 Introduction 1
- 1.1 Context and Practical Objectives . . . 1
- 1.2 Problem Definition and Scope . . . 1
- 1.3 Three Linked Technical Challenges . . . 1
- 1.4 Research Questions and Falsifiable Claims . . . 2
- 1.5 Research Contributions . . . 2

2 Background and Related Work 4
- 2.1 ANN Objective and Metrics . . . 4
- 2.2 IVF, PQ, ADC and Refinement . . . 4
- 2.3 JHQ Representation and Assumptions . . . 5
- 2.4 GPU Execution Concepts . . . 6
- 2.5 Graph Search on GPUs . . . 6
- 2.6 Quantised Search and Fast Tables . . . 6
- 2.7 Research Gap and Design Choices . . . 7

3 Research Design and Measurement 8
- 3.1 Research Questions and Comparisons . . . 8
- 3.2 Datasets, Queries, Ground Truth and Splits . . . 8
- 3.3 Timing, Variance and Memory Accounting . . . 9
- 3.4 Baselines and Frontier Construction . . . 10
- 3.5 Experimental Records and Reproducibility . . . 11
- 3.6 Evidence Strength and Threat Controls . . . 11

4 GPU Index Design and Construction 12
- 4.1 End-to-End Build and Search Boundaries . . . 12
- 4.2 Training, Rotation and Codebook Generation . . . 13
- 4.3 IVF Assignment, IDs and Physical Memory Layout . . . 14
- 4.4 Resident Storage, Workspace and Index Lifecycle . . . 16
- 4.5 Construction Cost and Tested Scale . . . 17
- 4.6 Index Representation and Safety Contracts . . . 18

5 Representation-Aware GPU Search 20
- 5.1 Query Path and Four Distinct Ranking Objects . . . 20
- 5.2 Exact Cartesian LUT Factorisation . . . 21
- 5.3 Packed Codes and Candidate-Parallel Memory Access . . . 23
- 5.4 Exact Global Primary Top-c via Multi-Block Scanning . . . 23
- 5.5 Shared Complete-Key Thresholds and Execution Scheduling . . . 26
- 5.6 Lane-Cooperative Residual Scoring and Final Top-k . . . 29
- 5.7 IVF Probe Selection as an Independent Primitive . . . 30

6 Workload-Level Budget Calibration 31
- 6.1 Why Fixed Refinement Budgets Can Be Wasteful . . . 31
- 6.2 GT-Free Sampled Output Agreement . . . 32
- 6.3 Budget Search, Nestedness and Tie Safety . . . 33
- 6.4 Budget Baselines and Their Questions . . . 35
- 6.5 Hold-out Risk, Calibration Cost and Payback . . . 35

6.6 Policy Failure Modes and Applicability 36

7 Experimental Evaluation **37**
7.1 Experimental Protocol 37
7.2 Modern GPU ANN Recall–QPS Frontiers 37
7.3 Cartesian LUT, Packed Loads and Lane Refinement 40
7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation 42
7.5 Held-out Budget Quality, Throughput and Payback 43
7.6 Batch Regimes, G Scheduling and Workload Skew 47
7.7 Cold Construction, Resident Memory and Tested Scale 48
7.8 Numerical Robustness, Anomalies and Recall Attribution 50

8 Discussion and Future Work **52**
8.1 Three Cross-Layer Insights 52
8.2 Two Deep Negative-Result Case Studies 52
8.3 Threats to Internal and External Validity 53
8.4 Testable Research Directions 54
8.5 Conditional Deployment Recommendation 54

9 Conclusion **55**
9.1 Answers to the Four Research Questions 55
9.2 Closing Principle and Outlook 55

Bibliography **56**

A Reproducibility and Experiment Manifest **59**
A.1 Source, Environment and Trained States 59
A.2 Measurement Contracts 59
A.3 Budget Harness Contract 59
A.4 Supplementary Evidence 59
A.5 Strict Selection, Small Batches and Artifact Access 60

B Supplementary Results and Correctness Checks **61**
B.1 Parameter Coverage 61
B.2 Correctness and Numerical Evidence 62
B.3 Secondary Results and Evidence Status 62
B.4 Construction Capacities and Precision Anchors 62
B.5 Artifact Access 63

List of Figures

1.1 Problem, mechanism and evidence . . . 2
2.1 Candidate and precision cascade . . . 5
3.1 Host and device timing boundaries . . . 10
4.1 Build and query architecture . . . 12
4.2 List-ordered physical index layout . . . 15
5.1 Query stages and selection boundaries . . . 20
5.2 Packed candidate-parallel access . . . 23
5.3 Local coverage and truncated pairwise merging . . . 25
5.4 Shared complete-key filtering . . . 27
5.5 Sixteen-lane residual refinement . . . 29
6.1 Work affected by the refinement budget . . . 31
6.2 Workload-level calibration and evaluation . . . 32
6.3 Nested and non-nested candidate pools . . . 34
7.1 E1 recall–throughput frontiers . . . 38
7.2 TF32 and FP32 precision frontiers . . . 39
7.3 Small-batch competitive regions . . . 40
7.4 Cartesian lookup-table grouping . . . 41
7.5 Packed access and refinement controls . . . 41
7.6 Held-out risk across sample sizes . . . 45
7.7 Finite-pool risk prediction and observation . . . 46
7.8 Direct cumulative calibrated-reuse time . . . 47
7.9 Batch-dependent execution regimes . . . 48
7.10 Recorded construction costs . . . 49
7.11 Resident-memory decomposition . . . 50
7.12 Large-dataset search frontiers . . . 51
8.1 Mailman and rank-oriented coding . . . 53

List of Tables

1.1 Technical contributions and evidence boundaries . . . 3
2.1 Design alternatives and research gap . . . 7
3.1 Research questions and experimental controls . . . 8
3.2 Dataset and training configuration . . . 9
3.3 Principal E1 tuning dimensions . . . 11
4.1 Per-vector storage model . . . 16
4.2 Analytical row-array occupancy by construction phase . . . 17
4.3 Original-input FP64 recoding audit . . . 19
4.4 Representation contracts and audit coverage . . . 19
5.1 Query-stage selection contracts . . . 21
5.2 Cartesian lookup-table trade-offs . . . 22
6.1 Budget controls and their roles . . . 35
7.1 Cross-system comparison at recall 0.97 . . . 37
7.2 FP32 sensitivity at matched configurations . . . 38
7.3 Small-batch matched-recall ratios . . . 40
7.4 Refinement gains at 128 probes . . . 42
7.5 Strict computed-score selection checks . . . 42
7.6 Probe-selection throughput ratios . . . 43
7.7 Full-pool budget calibration . . . 44
7.8 Sampled-budget held-out quality . . . 45
7.9 Sampled-calibration economics . . . 46
7.10 Direct costs for 500 reuse batches . . . 46
7.11 Integrated execution and scheduling gains . . . 47
7.12 Cold construction and resident memory . . . 49
7.13 E1 allocation ranges . . . 50
8.1 Threats to validity . . . 54
A.1 Component and run provenance . . . 59
B.1 Exact E1 configuration counts . . . 61
B.2 Validation coverage . . . 62
B.3 Independent construction-buffer capacities . . . 62
B.4 Same-session precision anchors . . . 63

Notation and Abbreviations

Symbol	Meaning
N, d	Number of database vectors and original dimension
M, D_s	Number of primary subspaces and dimensions per subspace
$a_{\text{add}}, a_{\text{in}}, a_{\text{assign}}$	Configured add, staged input, and independently allocated assignment rows
n_b, S_{stage}	Pipeline buffer count and byte budget per input/rotated array
Π	Square orthogonal transform; $q' = \Pi q$
B, B_r	Primary bits per subspace and residual bits per coordinate
$\rho(p), o_\ell$	Input ID at physical position p, and start of IVF list ℓ
k	Number of returned neighbours
n_c, c	Routed candidates and primary survivors receiving refinement
α	Refinement multiplier, with $c = \max\{k, \lceil \alpha k \rceil\}$
S, ϵ	Calibration sample size and allowed disagreeing result slots
$C_\alpha, \mathcal{R}_\alpha$	Valid primary survivor set and returned ID set at budget α
$\alpha_{\text{ref}}, B_{\text{reuse}}$	Reference refinement budget and number of subsequent reuse batches
g_{LUT}	Number of groups in a factorised lookup table
g_{scan}	Number of scan groups per query; zero labels the one-block path
$C_{\text{buf}}, T_{\text{blk}}$	Scan buffer capacity and threads per block
$n_{\text{LUT}}(g_{\text{LUT}})$	Number of LUT entries per primary subspace
$\kappa(D, p)$	Packed complete-key encoding for bound publication
ℓ_{ref}	Number of cooperating lanes per refined candidate
D_{raw}	Squared distance between original vectors
D_{P}	Distance under the primary reconstruction
D_{H}	Refined ranking score: reconstructed squared distance plus a query-only constant

Abbreviation	Meaning
ANN	Approximate nearest neighbour
ADC	Asymmetric distance computation (distinct from the ADC conference)
IVF	Inverted file
JHQ	Johnson–Lindenstrauss enhanced hierarchical quantisation
LUT	Lookup table
PQ	Product quantisation
QPS	Queries per second
GT	Ground truth
FP32	IEEE single-precision storage and arithmetic, with multiplication mode stated separately
TF32	Tensor-core multiplication format with reduced input precision and FP32 accumulation
CV	Coefficient of variation of candidate counts

1 Introduction

1.1 Context and Practical Objectives

ANN avoids scoring every full-precision vector. Graph search uses navigation; compressed IVF scans selected lists with cheap scores [18, 28, 33]. Here the data reach 17.78 million vectors and 3,072 dimensions. Stella's FP32 vectors alone need 67.81 GiB (section 7.7). Compact codes save space; their search cost still depends on tables, memory access and selection.

JHQ screens with primary codes, then refines a shortlist using residuals [16]. This leaves two questions: how to run the scan and how many candidates to refine. Reusing the trained index lets us vary each choice. We measure recall, host-visible speed, resident memory and construction cost, including cases where JHQ loses.

1.2 Problem Definition and Scope

The target is raw squared-L2 top-k for query q over $\mathcal{X} \subset \mathbb{R}^d$. IVF routes the query; primary score D_{P} selects up to $c = \max\{k, \lceil \alpha k \rceil\}$ survivors; residual score D_{H} selects the final results. These reconstructed scores differ from raw distance. The refined value additionally contains a query-only constant, as defined in section 2.3. Selection is exact for its computed score and tie rule; GT measures ANN quality.

We retain JHQ's square rotation, Cartesian primary codes and primary-relative residuals. The work covers GPU construction, layout, execution and one workload-level refinement multiplier. Unlabelled output agreement selects the multiplier for reuse on a stable workload. It supplies neither a per-query predictor nor an absolute-recall guarantee.

E1 uses one RTX 5090, squared-L2 top-10 and batch 1,000 on four datasets. Component and budget studies use six datasets; T5 adds batches 8/32 (section 7.2.5). The compressed index stays on the GPU, with host preparation and query transfers. Updates, multi-GPU search, host-backed residuals and billion-scale deployment are untested.

1.3 Three Linked Technical Challenges

Figure 1.1 connects three challenges to the mechanisms and evidence used to address them.

First, the primary codebook has useful mathematical structure that a direct GPU port can overlook. Each eight-bit subspace code indexes 256 distances. Its Cartesian coordinates permit an exact 16+16 factorisation, but more aggressive splitting adds lookups and arithmetic. The challenge is to exploit separability without assuming that the smallest table gives the fastest scan. Candidate-parallel word access and residual lane cooperation also need different data mappings.

Second, all global winners might lie in one part of a query's routed pool. Dividing the required survivor count between groups can therefore lose answers. Local retention and merging preserve coverage; shared bounds reduce extra work. These operations add overhead, so batch size and skew must guide the choice. IVF-list selection remains separate from database-candidate retention.

Third, a large survivor budget wastes refinement, while a small one can discard neighbours. Reference agreement needs no GT, but inherits reference errors. Calibration also costs time. We test quality and total cost over subsequent batches.

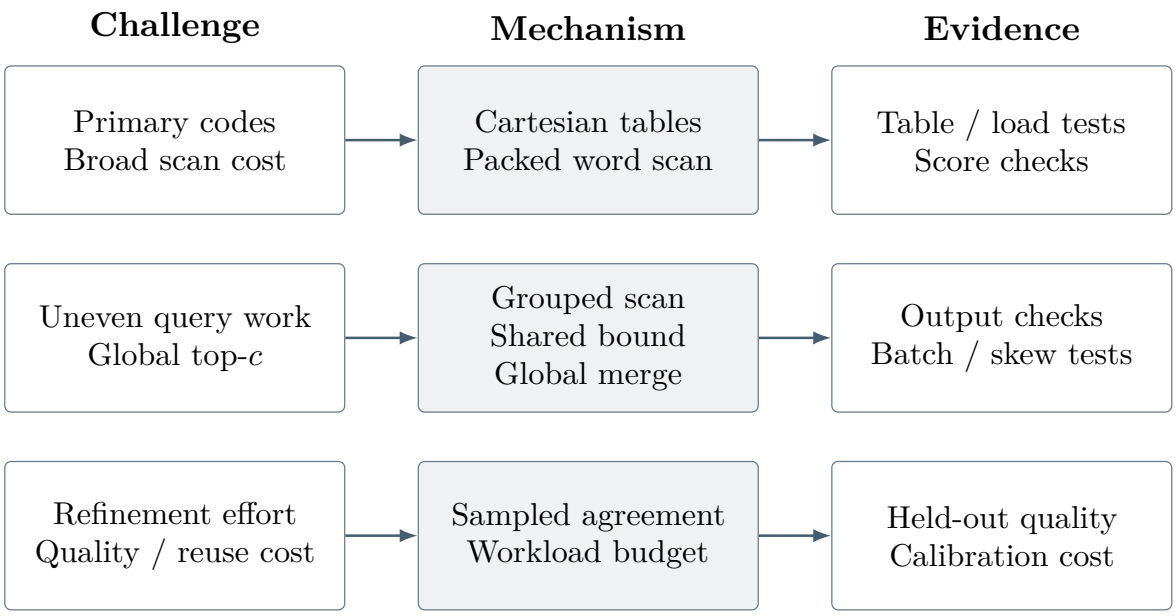


Figure 1.1. Problem, mechanism and evidence. The arrows organise research questions; they do not imply that independently measured speedups multiply or that an unlabelled agreement rule guarantees GT recall.

1.4 Research Questions and Falsifiable Claims

RQ1: Construction and capacity. How can JHQ become a GPU-resident searchable index, and what construction cost and capacity are demonstrated (section 7.7)? Resident size and construction peak are measured separately.

RQ2: Correct and efficient execution. Which scoring, selection and scheduling choices improve speed while preserving the specified scores and tie rules? Paired experiments and reference checks answer this in sections 7.3, 7.4 and 7.6.

RQ3: Budget quality and total cost. Can unlabelled agreement choose a useful survivor budget and repay calibration? Held-out tests compare fixed budgets, sampled/FULL rules and enumeration (section 7.5). The GT-labelled oracle is an evaluation control.

RQ4: Competitive operating regions. Where does JHQ compete with modern GPU ANN methods at matched recall and a common host-to-host boundary? The answer uses independently tuned E1 frontiers and explicit memory definitions (section 7.2). Int8 graph competitors, high-recall reversals and absent coverage are part of the answer.

Chapter 3 turns these questions into controlled comparisons. Together they separate what was built, whether its execution is correct, what effort a workload requires, and whether the resulting system has useful operating regions.

1.5 Research Contributions

Table 1.1 states three linked project contributions and their inherited foundations. The core contribution is the adaptation and evaluation of a particular representation–execution–budget combination. JHQ, Cartesian quantisation, IVF, bitonic/radix selection, merging and CUDA Graph execution retain their prior-art status.

Table 1.1. Technical contributions and their evidence boundaries.

	Inherited foundation	Project contribution	Evidence
C1	JHQ rotation, codebooks and hierarchy	GPU training/encoding, bounded batch construction and list-ordered packed representation	Search of 17.78M vectors at 20.44 GiB resident; repeated train+add 10.54–10.65 s; sampled construction peak 22.58 GiB
C2	Cartesian additivity and established GPU primitives	Integrated factorised scoring, complete-key grouped retention/bounds, lane refinement and execution scheduling	Two-group LUTs lead in 22/24 cells; scheduled execution 2.727× at batch 8; explicit correctness scope
C3	Candidate expansion and output-agreement ideas	Workload-level budget rule, conditional selection analysis, held-out controls and cost/risk evaluation	FULL meets the relative-loss target in 12/12 cells; smaller samples expose risk; four cells directly measure calibrated reuse

Performance depends on batch size and quality target; Chapters 7–8 report the unfavourable regions and numerical limits.

2 Background and Related Work

This chapter covers candidate access, quantisation and selection, then places JHQ's execution and budget choices against existing GPU ANN methods.

2.1 ANN Objective and Metrics

For database $\mathcal{X} = \{x_i\}_{i=0}^{N-1} \subset \mathbb{R}^d$ with stable IDs, exact top-k selects the smallest distances

$$D_{\text{raw}}(q, x) = \|q - x\|_2^2. \tag{2.1}$$

We use the stored GT top-10 set, including its tie choices. Recall@10 measures how many of those IDs ANN recovers (equation (3.1)); it is not a worst-case distance guarantee.

We measure query throughput, batch-call latency, construction cost and storage. These are separate costs: faster batch processing does not establish concurrent-request tail latency, and resident index bytes exclude some workspace and temporary allocations. Chapter 3 gives the boundaries.

Tuning gives a finite recall–throughput frontier. A measured threshold chooses a qualifying configuration; interpolation estimates between points. Missing coverage proves neither a win nor an impossibility. Section 3.4 fixes both estimators and endpoint tolerance.

2.2 IVF, PQ, ADC and Refinement

IVF partitions the database around coarse centroids and stores a list of vectors assigned to each centroid. A query chooses n_{probe} lists, producing a routed universe $U(q)$ of n_c candidates. Visiting fewer lists reduces access work but can exclude a true neighbour before any fine scoring. The centroids are a routing device; their distance ordering is a different selection problem from choosing database vectors.

Product quantisation divides a vector into M coordinate subspaces and represents each with a label into a small learned codebook [17]. If $\widehat{x}_m = \mathcal{C}_m[z_m]$, asymmetric distance computation (ADC) keeps the query unquantised and scores

$$\|q - \widehat{x}\|^2 = \sum_{m=0}^{M-1} \|q_m - \mathcal{C}_m[z_m]\|^2. \tag{2.2}$$

A query-specific table precomputes each subspace distance. Scanning then uses code loads, table lookups and accumulation. The product is between subspaces; the centroids *within* a learned subcodebook need not have a further coordinate product. This distinction determines whether Chapter 5's exact internal table factorisation is valid.

IVF-PQ often encodes $x - \mu_{\ell(x)}$. JHQ instead uses a residual relative to the primary reconstruction after rotation. Adding the IVF centroid during JHQ reconstruction would therefore add a term the encoder never removed. Candidate expansion and more accurate refinement are established ideas [10, 18].

With at least k valid routed candidates, the survivor cap is $c(\alpha) = \max\{k, \lceil \alpha k \rceil\}$. The output and survivor sets satisfy

$$\mathcal{R}_\alpha(q) \subseteq C_\alpha(q) \subseteq U(q) \subseteq \mathcal{X}. \tag{2.3}$$

Refinement can reorder retained vectors, but cannot recover excluded IDs. Stage-wise GT coverage bounds final recall; measuring each stage's loss needs counterfactual scoring over a fixed universe.

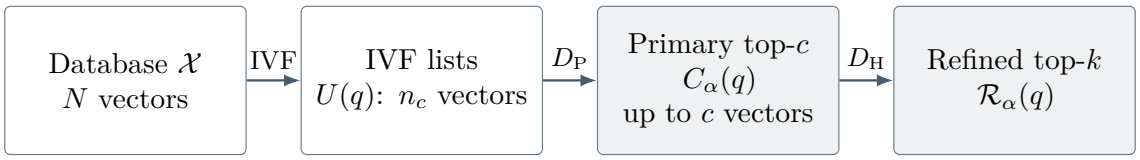


Figure 2.1. Candidate and precision cascade. IVF selects lists; primary scoring selects vectors; only survivors read residuals. The hierarchy is inherited. This diagram defines logical sets and score fidelity, not measured recall losses or GPU kernel boundaries.

Broad scoring takes $O(n_c M)$ lookups; refinement takes $O(cd)$ coordinate work. Actual latency also includes access and selection. Probes change the routed universe U; α changes retention within it. They control different losses.

2.3 JHQ Representation and Assumptions

2.3.1 What is inherited

JHQ combines an orthogonal transformation, an analytical Cartesian primary code and a higher-precision residual layer [16]. Here the transform is square: $\Pi \in \mathbb{R}^{d\times d}$, $\Pi^\top \Pi = I$, $y = \Pi x$ and $q' = \Pi q$. Therefore

$$\|q - x\|^2 = \|\Pi(q - x)\|^2. \tag{2.4}$$

Although the prior work calls this an orthogonal JL transformation, this configuration does not reduce dimension. It preserves real-arithmetic distance exactly; quantisation subsequently introduces approximation. Actual FP32 orthogonality and GEMM error are separate implementation checks.

Partition y into M subspaces of dimension $D_s = d/M$. With B bits per primary subspace, $K = 2^B$ labels are formed from $L = 2^{B/D_s}$ scalar levels, requiring the implemented Cartesian parameters to make L an integer. Writing Φ for the standard normal CDF, the constructor uses

$$a_i = \sigma\Phi^{-1}\left(\frac{i - 1/2}{L}\right), \quad i = 1, \ldots, L, \qquad \mathcal{C}_m = \{a_1, \ldots, a_L\}^{D_s}. \tag{2.5}$$

At the evaluated $B = D_s = 8$, $L = 2$ and $\mathcal{C}_m = \{-a, +a\}^8$, with $a \simeq 0.67449\sigma$. Each byte records eight coordinate signs. The implementation estimates σ from vector energy; unit-normalised inputs motivate the scale $1/\sqrt{d}$. This representation is prior art, not an invention of the GPU execution layer.

Residuals are $r = y - \widehat{p}$, giving reconstruction $\widehat{y} = \widehat{p} + \widehat{r}$. The evaluated codes use $d/8$ primary bytes and d residual bytes per vector, before metadata. Refining fewer rows reduces reads, not stored residual size.

2.3.2 Model assumptions and score semantics

Random rotation can spread coordinate energy, motivating a shared Gaussian-inspired scalar model. It does not force an arbitrary realised database to have independent, zero-mean Gaussian coordinates. For a fixed rotation and empirical database mean μ, the transformed mean is $\Pi\mu$ and $\|\Pi\mu\| = \|\mu\|$. Averaging over random rotations is different from centring one fixed corpus. Even a uniformly random point on a sphere has dependent coordinates through its fixed norm. Consequently, isometry is an algebraic guarantee; the distributional approximation is a modelling assumption. Chapter 8's centred-coding pilot examines one practical consequence of that distinction.

The inherited primary levels are Gaussian mid-quantiles. Under an exact Gaussian model, zero-threshold one-bit MSE-optimal levels would instead be $\pm\sigma\sqrt{2/\pi} \simeq \pm 0.79788\sigma$. For fixed signs and positive scale, however,

$$D_{\mathrm{P}} = \|q'\|^2 + da^2 - 2a\langle q', s\rangle, \qquad s \in \{-1, +1\}^d. \tag{2.6}$$

Thus scale cancels from primary ordering but still affects residual reconstruction. Our design uses separability and measured quality, without claiming optimal quantisation.

D_{raw} uses original vectors; $D_{\mathrm{P}} = \|q' - \widehat{p}\|^2$ uses primary reconstructions. The refined score is

$$D_{\mathrm{H}} = \|q' - \widehat{p} - \widehat{r}\|^2 + \|q'\|^2. \tag{2.7}$$

Its query-only term changes distance values, not ranking. Routing and reconstruction errors remain after exact computed-score selection.

2.4 GPU Execution Concepts

CUDA uses 32-thread warps and blocks with limited registers and shared memory [25]. JHQ spreads scanning across candidates and refinement across coordinates of one survivor. More blocks can hide latency, but also duplicate tables and buffers.

Faiss uses warp selection [18]; bitonic, radix and merge primitives trade work for state. AIR top-k and GridSelect study global selection [38]. JHQ adapts these existing tools.

CUDA Graph replay saves launch setup. Capture, workspace changes and shared-key access still follow the lifecycle and correctness contracts in Chapter 5.

2.5 Graph Search on GPUs

SONG exposes distance parallelism in GPU graph search [39]. CAGRA designs GPU construction and navigation, tuned by connectivity and search width [28]. We test both fp32 and quantised-vector CAGRA because their speed and storage differ.

BANG pairs GPU compressed vectors with a host-resident graph [37]. This trades transfer costs for capacity. JHQ's index remains device-resident.

E1 compares graph navigation with regular scans at batch 1,000; T5 tests their small-batch behaviour (section 7.2.5).

2.6 Quantised Search and Fast Tables

2.6.1 Learning a representation versus exploiting its structure

OPQ learns rotation and subspace codebooks [13]; this can remove the Cartesian structure needed for JHQ's exact split. ScaNN weights error for inner-product ranking [15]. Both show why reconstruction error alone is insufficient; Chapter 8 tests this within JHQ.

2.6.2 Small lookup tables have substantial prior art

PQ Fast Scan and Quicker ADC already shape tables for CPU register lookup [1, 2]. JHQ exploits Cartesian structure on a GPU, with separate checks for FP32 rounding and cutoff ties.

RaBitQ analyses a randomised compact-code estimator [11]; multi-bit variants trade storage for accuracy [12]. Its guarantees do not transfer to JHQ's primary/residual hierarchy. GPU IVF-RaBitQ supplies fused GPU search [33] and is a direct baseline.

2.6.3 Refinement effort and adaptive stopping

Faiss and cuVS expose refinement controls [7, 10]. Alternatives include GT-based autotuning [9], learned termination [19] and recall-targeted DARTH [6]. Chapter 6 instead selects one reusable budget from unlabelled reference agreement.

2.7 Research Gap and Design Choices

Table 2.1 compares the alternatives by the constraint they address. The literature already provides GPU graph search, compressed distance computation, register-oriented lookup, fast selection and adaptive effort. A useful contribution must therefore specify the representation contract, the integration decision and the evidence for it. "A GPU version" alone does not identify that contribution.

Table 2.1. Design alternatives and the thesis's narrower research gap. This is a mechanism comparison; measured rankings follow in Chapter 7.

Alternative	Useful structure	Principal trade-off	JHQ-GPU question
IVF-PQ / OPQ	Learned subspace codes; ADC	Distortion and training; arbitrary subcodebooks couple coordinates	What does an internally Cartesian primary code permit in GPU scoring?
Fast Scan / Quicker ADC	Small tables and SIMD lookup	Table precision, shape and ISA constraints	Does exact splitting repay extra GPU lookups?
SONG / CAGRA	Selective graph navigation	Dependent access, graph storage and construction	Where does a resident hierarchical scan compete?
BANG	Compressed GPU scoring with host graph	Capacity versus communication	What can complete compressed residency achieve?
IVF-RaBitQ	GPU code construction and fused search	Bit budget, estimator and probe effort	Which recall regions justify the residual hierarchy?
Adaptive effort	Refine ratios, autotuning, learned or declarative stopping	Labels, reference fidelity, prediction or calibration cost	Can unlabelled workload agreement select useful effort at an acceptable risk and total cost?

The open questions are practical: can this hierarchy support fast, correct GPU execution; when does extra scan parallelism pay; and does unlabelled budget calibration preserve enough quality to repay its cost? We keep the inherited representation fixed to study those choices.

3 Research Design and Measurement

We use paired execution tests, held-out calibration and tuned system frontiers. Each needs different controls. This chapter defines them; Chapters 4–6 describe the methods and Chapter 7 reports results.

3.1 Research Questions and Comparisons

Table 3.1 defines the comparison unit and what would challenge each research claim. A *cell* fixes a dataset, index configuration and workload setting; arms are alternative implementations or budgets within that cell. For paired execution measurements, the index state, queries, probe count, refinement budget and scoring representation are held fixed unless the named treatment changes them. The recorded paired ratio is $Q_{\text{treatment}}/Q_{\text{control}}$. Geometric means summarise multiplicative ratios over the stated cells, giving each cell equal weight.

Table 3.1. Research questions as testable comparisons. A capacity observation has a different control from a paired execution intervention.

Question	Intervention / comparison	Held fixed or accounted for	Measure and counter-evidence
RQ1: build and capacity	GPU construction; cached versus cold lifecycle; compressed versus raw storage	N, d, representation, routing and timed construction boundary	Train/add time and resident bytes; failed builds or model–allocation gaps delimit feasibility
RQ2: execution	LUT grouping, word access, refinement lanes, grouped scan, shared bound and probe selection	Paired state, workload and score; reference output semantics	Wall QPS, stage time and output checks; slowdowns or non-tie mismatches challenge the mechanism
RQ3: budget	Sampled / full-pool rule versus fixed 64, 100, 200, enumeration and GT oracle	One index and route; disjoint calibration and held-out query positions	Held-out recall loss, selected-arm QPS and calibration cost; threshold crossings challenge reliability
RQ4: operating region	Independently tuned JHQ, CAGRA, IVF-PQ and IVF-RaBitQ	Exported vectors, GT, k, device and host-to-host boundary; memory reported separately	Matched-recall QPS and coverage; stronger competitors at other recall targets challenge broad superiority

Paired ablations measure an implementation change. Explaining it through cache misses or occupancy would need hardware counters. Cross-system frontiers compare whole systems; the integrated scheduling test in section 7.6 avoids multiplying gains from different baselines.

The E1 grid was extended where recall coverage was incomplete. Low-recall losses, high-recall reversals and unsuccessful alternatives remain part of the evaluation. The rule uses reference agreement; held-out evaluation measures raw-vector GT recall.

3.2 Datasets, Queries, Ground Truth and Splits

Table 3.2 records the six exported datasets used by the newer budget, LUT and scale campaigns. The poster's E1 comparison uses the first four. Vogue contains fashion-image embeddings; arXiv contains abstract embeddings; the two OpenAI datasets use Qdrant's DBpedia exports [5, 31, 32, 34]. BGE uses the Italian Wikipedia configuration, and Stella uses the TREC24 BioGen corpus export [35, 36]. Sizes describe the measured exports after preparation, rather than the current upstream cards.

Table 3.2. Data and JHQ training configuration. n_q is the actual query count; $n_t = 39n_{\text{list}}$ is the requested coarse-training sample size. E1 uses the first four rows. All rows use squared L2 and $k = 10$.

Dataset	N	d	n_q	n_{list}	n_t
Vogue	932,328	768	1,000	4,096	159,744
arXiv	2,253,000	768	1,000	8,192	319,488
OpenAI-1536	999,000	1,536	1,000	8,192	319,488
OpenAI-3072	999,000	3,072	1,000	8,192	319,488
BGE-M3	10,091,524	1,024	1,000	32,768	1,277,952
Stella	17,776,615	1,024	999	32,768	1,277,952

OpenAI/BGE/Stella preparation removes non-finite rows, normalises norms above 10^{-12} and samples queries without replacement using seed 42. Stella supplies 999 corpus-sampled queries, not its separate 65-query benchmark. These exports test vector neighbours, not relevance labels. The OpenAI exports share the sampling rule; checks support row alignment on all queries and 10,299 base rows, without original-ID joins (sections 7.2 and 7.2.3).

All methods in a cell read the same query and GT files. Ground truth is computed by exhaustive squared-L2 search over exported FP32 vectors; the streamed converter maintains top-k across database tiles. Evaluation uses stored GT order, including its floating-point tie choices. With $\mathcal{T}_k(q)$ the first k GT IDs and $\mathcal{R}_k(q)$ the returned ID set,

$$\text{Recall}\,@k = \frac{1}{|\mathcal{Q}|k} \sum_{q \in \mathcal{Q}} |\mathcal{R}_k(q) \cap \mathcal{T}_k(q)|. \tag{3.1}$$

GT rows are 100 wide for Vogue/arXiv and 20 wide for the other four; only ten entries are scored. Duplicate returned IDs do not earn repeated credit. Range, duplicate and repeat-call checks are recorded separately.

JHQ uses $M = d/8$, $B = B_r = 8$, eight coarse iterations and 100,000 residual-training rows. Sweeps reuse trained states but re-encode the database; cold builds start afresh. Training uses database rows. Even/odd query positions give calibration/held-out pools of 500/500, or 500/499 for Stella. Seeded subsamples of sizes 32, 64 and 128 are drawn 64 times per cell; FULL uses all 500 calibration queries. GT labels enter the offline oracle and held-out assessment, while the deployable agreement rule uses search outputs (sections A.3 and 6.2).

Preparation code and shape logs establish this protocol; complete historical vector-file and trained-state hashes are unavailable.

3.3 Timing, Variance and Memory Accounting

The principal comparison starts with FP32 queries in host memory and ends when top-10 IDs and scores are available in host memory after stream completion (figure 3.1). It includes pinned staging, H2D, query preprocessing, search and D2H. CAGRA int8 query quantisation is inside this boundary; quantiser training and index construction are outside it. Resource handles and output buffers are reused. JHQ returns to pageable host arrays and the baselines to pinned arrays; CAGRA/PQ use Python APIs, whereas JHQ/RaBitQ use C++. These interface differences remain visible when interpreting small margins.

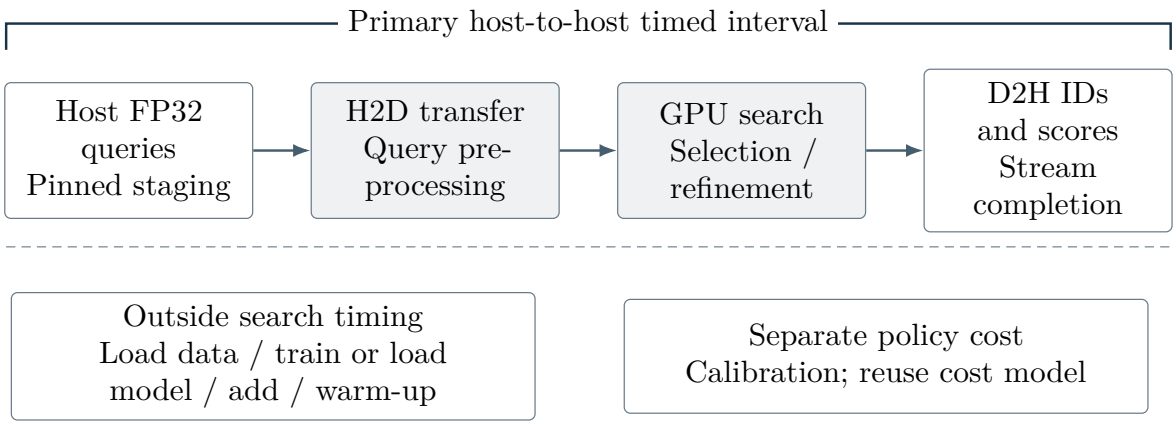


Figure 3.1. Timing boundaries. The host interval includes transfers and query-specific computation. Device-stage profiles explain parts of that interval; separate profiling and synchronisation can change execution, so their sum is not an interchangeable end-to-end latency.

With t in seconds, round throughput and its summary are

$$Q_r = \frac{m_r b}{\sum_{i=1}^{m_r} t_{ri}}, \qquad Q = \operatorname{median}_r Q_r, \tag{3.2}$$

E1 uses batch 1,000 and five rounds. JHQ warms up once per variant/round; baselines warm up three times initially. Normally each round has 24 timed calls, fewer for slow baseline points. Warm-up excludes JIT and capture. Latency percentiles describe repeated batch calls, not individual or concurrent requests.

We retain slow valid calls and both round-QPS and call-latency summaries. Their medians need not be reciprocal. Shared indexes and query pools do not provide independent training replications, so sweep cells receive no independence-based significance claim.

Runs use an RTX 5090 (32,607 MiB), driver 595.71.05, CUDA 13.0 and cuVS 26.08.01. OpenMP uses 16 threads within a 25-CPU quota; E1 records 575 W. A cooperative lock and snapshots do not fix clocks or host load. JHQ JIT-compiles Release compute_80 PTX [27]; baselines use installed builds. Device profiles are reported separately.

Resident memory includes encoded arrays, models, workspace and context. E1's context-subtracted JHQ totals and CAGRA's index deltas therefore differ in scope (sections 4.4 and 7.7). Later-created search pools and build temporaries prevent an equal-peak-memory comparison. Separate 20 ms samples estimate build peaks. Cold train/add excludes input loading and preceding host setup.

Supplementary evidence boundary. Supplementary checks (section A.4) use separate sessions. Precision changes and cached-model reuse follow sections 4.2 and 7.2.3; historical baseline ratios remain cross-session.

3.4 Baselines and Frontier Construction

The competing families cover graph traversal (CAGRA fp32/int8), product-quantised IVF and GPU IVF-RaBitQ [17, 28, 33]. FP32 GPU brute force provides an exhaustive-search reference. Each family sweeps its own implementation parameters (table 3.3); the recorded index configurations can differ in storage and reconstruction fidelity. Consequently the main frontier measures achievable speed at a quality target within these grids. A separate memory account accompanies it.

Table 3.3. Principal E1 tuning dimensions. Exact per-point parameters and coverage extensions remain in the raw records. The grid is finite and differs between methods.

Family	Recorded search dimensions
JHQ	Probes and α; initial probes 64/128/256 and 0.5/1/2 times the workload budget; coverage extensions reach probe 1,024 and larger/smaller budgets
CAGRA fp32/int8	Graph degree 32/64; search width 1/2/4/8; intermediate top-k capacity 32–512
IVF-PQ	Eight-bit subcodes; 192/384/768-byte codes at $d = 768$, 384/768/1,536 at higher dimensions; fp16/fp32 lookup settings; probes 16–512
IVF-RaBitQ	4/8/9 bits per dimension, supported search modes, probes 16–1,024; additional 4,096-list OpenAI indexes
GPU brute force	Exhaustive FP32 squared-L2 search, common host boundary

A point dominates another if recall and QPS are both no worse and one is better. The frontier can span index configurations. At target r, the measured estimate is

$$Q_{\text{meas}}(r) = \max\{Q_i : R_i \geq r\}. \tag{3.3}$$

It is undefined without a qualifying point. For adjacent frontier points, interpolation uses

$$\log Q_{\text{int}}(r) = (1-\lambda)\log Q_a + \lambda \log Q_b, \qquad \lambda = \frac{r - R_a}{R_b - R_a}. \tag{3.4}$$

The parser first accepts a point within 5×10^{-4} of the target; otherwise it interpolates inside the measured range only. T5 also keeps qualifying measured endpoints below a method's minimum frontier recall. We report both estimates near ties; neither matches storage budgets.

Failures and invalid outputs stay outside the valid frontier. Repeat-call set and order changes are logged separately. Isolated reruns replace contaminated builds; skipped Stella batch-1,000 runs supply no points (section A.2). Current frontiers include int8 CAGRA (figure 7.1); historical ADC/poster plots retain their original scope.

3.5 Experimental Records and Reproducibility

Each family records version, index/query settings, timing and outputs. Paired tests share trained states; state or precision changes define a new family. Missing historical executable/model fingerprints limit exact reproduction (appendix A).

3.6 Evidence Strength and Threat Controls

Correctness refers to stated computed scores and tie rules. Raw recall also depends on routing and reconstruction. Sampling/order conditions are in sections 6.2 and 6.3; section 8.3 covers finite pools, shared indexes, tuning grids, interfaces and hardware limits.

4 GPU Index Design and Construction

GPU construction trains and encodes JHQ's representation into list-ordered arrays with bounded temporary storage [16]. Query kernels and measured construction costs are in Chapter 5 and section 7.7.

4.1 End-to-End Build and Search Boundaries

Training creates reusable models; add encodes rows and builds lists; search uses the completed index. A cache hit skips fitting, not database encoding. Cold train-plus-add and cached-state search therefore measure different work.

The host prepares samples, the square transform and primary codebook. GPU products apply rotation; device training fits IVF centroids and residual codebooks. Add then transfers bounded batches, encodes both levels and writes list-ordered arrays.

Model state holds the transform, codebooks and centroids. Database state adds offsets, IDs, codes and corrections. Queries overwrite workspace but reuse its allocations; a captured graph references those existing arrays.

One successful `add` creates a fixed snapshot; another add is rejected. Updates require a replacement snapshot. Physical positions stay fixed during search; mutable insert/delete and transactional publication are unimplemented.

Search maps host queries to host IDs and computed scores. Equation (5.20) explains the query-only score term; section 3.3 defines timing.

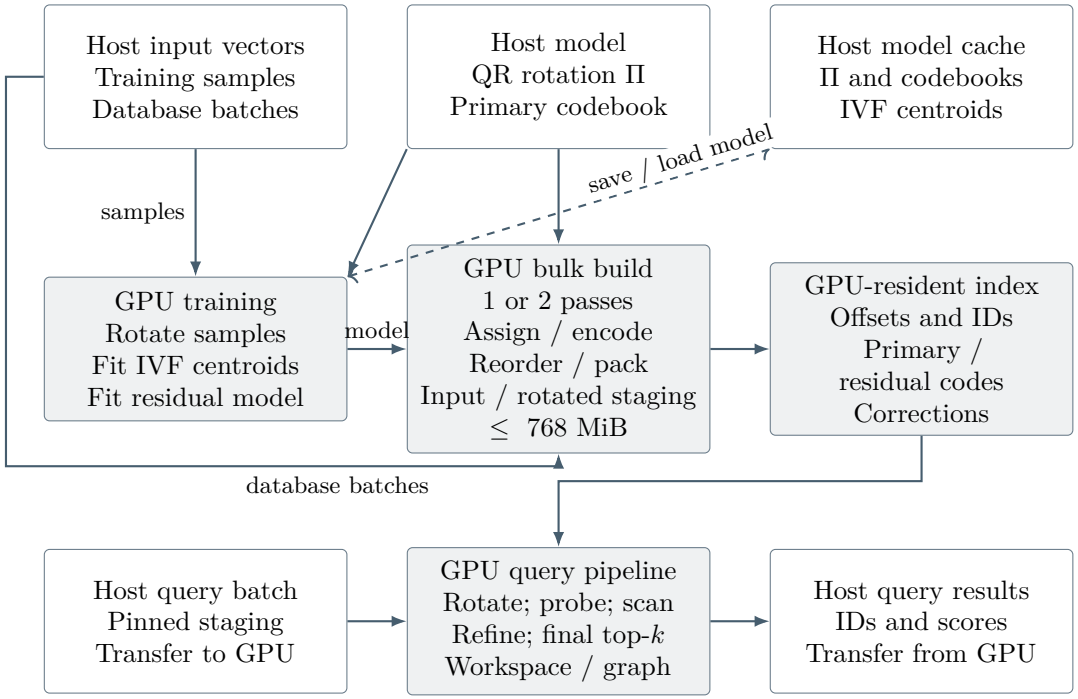


Figure 4.1. Implemented build and query boundaries. Host work includes sampling, orthogonal-matrix construction and analytical primary-codebook generation; the evaluated configuration fits IVF and residual models on the GPU. The cache stores trained state, whereas the completed database representation resides on the device. Dashed arrows denote trained-state reuse. The staging label bounds only raw/rotated float buffers at source defaults; other pipeline scratch is additional.

4.2 Training, Rotation and Codebook Generation

4.2.1 Transform and sampling

The implementation draws a seeded Gaussian matrix on the host and uses LAPACK QR routines to construct a square orthogonal matrix Π. For row-major training data X_t, device matrix multiplication computes

$$Y_t = X_t \Pi^{\mathsf{T}}, \qquad y = \Pi x, \qquad \|\Pi(q-x)\|_2^2 = \|q-x\|_2^2 \quad \text{when } \Pi^{\mathsf{T}}\Pi = I. \tag{4.1}$$

The output retains dimension d. This is an orthogonal change of coordinates; QR construction and device products have finite-precision error, whose measurement is part of C0 in section 4.6.

The harness selects database rows at a fixed stride. The primary/IVF sample has 159,744 rows for Vogue, 319,488 for arXiv and both OpenAI datasets, and 1,277,952 for BGE and Stella: exactly $39n_{\text{list}}$ in each case. The $39n_{\text{list}}$ rule matches FAISS's warning threshold [8], without guaranteeing fit quality. A separate 100,000-row strided sample bounds residual-training cost.

An earlier sweep fixed primary/IVF models and varied residual samples on Vogue/OpenAI-3072 ($B_r \in \{4, 8\}$, 32/128/256 probes). Across twelve configurations, 100K rather than all rows lowered recall in eleven cases and tied once; the largest gap was 0.0018. Intermediate sizes were non-monotone. This 25-Lloyd-iteration study motivates a bounded sample, not the later six-dataset result.

4.2.2 Why retain the analytical primary model?

The constructor estimates scale as

$$\sigma^2 = \frac{1}{n_t d} \sum_{i=0}^{n_t-1} \|x_i\|_2^2, \qquad D_s = d/M, \qquad K = 2^B. \tag{4.2}$$

For $B = D_s = 8$, levels are $\{-a, +a\}$ with $a = 0.674\sigma$: one byte stores eight signs. JHQ's analytical structure [16] allows $O(d)$ encoding and exact small-LUT factorisation (section 5.2). Arbitrary Lloyd-refined codewords need not remain separable.

For fixed signs $\widehat{p} = as$, $s_j \in \{-1, +1\}$,

$$\|q' - as\|^2 = \|q'\|^2 + da^2 - 2a\langle q', s\rangle. \tag{4.3}$$

Positive a does not change real-arithmetic primary order, without a Gaussian assumption. It does change residuals and refined ranking. Zero scale is degenerate; FP32 ties still matter.

On historical Vogue ($M = 96$, $B_r = 4$, 128 probes), earlier/analytical-primary builds gave primary recall 0.6618/0.6342 and full recall 0.9452/0.9409. Refinement narrowed the gap. Residual training also changed, so this is not a primary-codebook-only experiment. Chapter 7 tests the complete frontier.

4.2.3 Coarse assignment as filter and refine

Eight IVF iterations update centroids seeded from spaced rotated rows. Assignment ranks $s_\ell = \|\mu_\ell\|^2 - 2\langle y, \mu_\ell\rangle$. Under supported dimensions, the default filters with int8 products and resolves ambiguity in FP32; other precision paths also exist.

Lemma 4.1 (Assignment with certified score intervals)**.**
Let $\widetilde{y}, \widetilde{\mu}_\ell$ be dequantised approximations, $e = y - \widetilde{y}$ and $f_\ell = \mu_\ell - \widetilde{\mu}_\ell$. Keeping centroid norms exact, the approximate score $\widetilde{s}_\ell = \|\mu_\ell\|^2 - 2\langle \widetilde{y}, \widetilde{\mu}_\ell \rangle$ has real-arithmetic error bounded by

$$E_\ell = 2(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|). \tag{4.4}$$

Given valid intervals $[L_\ell, U_\ell]$ containing every target score, discarding only $L_\ell > \min_j U_j$ and exactly resolving the remaining scores preserves the minimum and its specified tie rule.

Proof.
Expand the dot-product error as $\langle y, f_\ell \rangle + \langle e, \widetilde{\mu}_\ell \rangle$ and apply Cauchy–Schwarz. A discarded score exceeds an upper bound on some retained score, so it cannot attain the minimum. Keeping interval equality preserves possible ties. □

The filter uses conservative norm bounds and FP32 headroom. A bitwise FP32 guarantee would also need accumulation-error bounds. The archived FP16 comparison differs on 0/14,838 Vogue, 6/31,748 BGE and 3/16,359 Stella tail assignments. This is finite evidence; cold logs do not identify the executed assignment precision.

4.2.4 Residual fitting and the iteration budget

Residual fitting follows IVF but uses $r = \Pi x - \widehat{p}(x)$, not an IVF-centroid residual. Each subspace shares a sorted $K_r = 2^{B_r}$ scalar codebook across its coordinates. GPU sorting and one-dimensional Lloyd updates fit it, capped at 2,000 iterations.

At the historical 100K sample, 2,000 rather than 25 updates gained 0.0063–0.0076 recall. Later changes exceeded reported two-run variation in BGE/Stella, and sparse tail bins kept changing. Cold runs hit the cap. This is budgeted FP32 fitting, not a claimed convergence result or a counterexample to exact Lloyd descent. At 20,000 updates, historical Stella all-row/100K recall was 0.9921/0.9920.

4.2.5 Construction work and observed imbalance

Dense QR costs $O(d^3)$ once; dense rotation costs $O(n_t d^2)$. Direct coarse assignment costs $O(n_t n_{\text{list}} d)$ per iteration. Analytical primary construction writes $O(MKD_s)$ values; residual extraction touches $O(n_r d)$ coordinates, followed by sorting and scalar updates. These describe work, rather than predict latency. Synchronised phase diagnostics place about 83% of BGE/Stella training in IVF k-means (about 2.97 s). They use one add buffer and forced synchronisation, separately from the normal overlapped timings in section 7.7.

List-size imbalance enters the scan rule in section 5.5. The individual contributions of eight IVF iterations, spaced seeds and finite sampling to that imbalance were not isolated.

Arithmetic used in the measured implementation. The evaluated implementation (v69) stores FP32 matrices but enables cuBLAS TF32 tensor-core math for $d \geq 1{,}536$. In its compiled faithful branch, a null override falls through to the dimension-based default. This affects training/add/query rotation and centroid GEMMs on both OpenAI datasets; an environment override cannot disable that branch. Thus FP32 storage does not imply full FP32 multiplication. The supplementary FP32 variant (v72) corrects this switch. Its sensitivity experiment reuses the same TF32-trained cache and changes add/query arithmetic, rather than retraining the model in FP32 (section 7.2.3).

4.3 IVF Assignment, IDs and Physical Memory Layout

4.3.1 From database IDs to list-ordered positions

An input row has identifier $i \in \{0, \ldots, N-1\}$. Coarse assignment places its transformed vector in a list according to the computed centroid scores. The mathematical assignment

target is

$$a(i) = \arg\min_{0 \le \ell < n_{\text{list}}} \|\Pi x_i - \mu_\ell\|_2^2. \tag{4.5}$$

Its implementation uses the selected numerical assignment path. Encoding the primary and residual representations does not subtract $\mu_{a(i)}$; list membership restricts the future search universe independently of those reconstructions.

The builder sorts assignments together with a sequence of input IDs. Let $\rho(p)$ be the input ID at sorted physical position p and let o_ℓ be the start of list ℓ. The completed arrays satisfy

$$0 = o_0 \le o_1 \le \cdots \le o_{n_{\text{list}}} = N, \qquad p \in [o_\ell, o_{\ell+1}) \Longrightarrow a(\rho(p)) = \ell. \tag{4.6}$$

Offsets are obtained by lower-bound searches over the sorted assignments. Equal consecutive offsets represent an empty list. Sorting by assignment does not promise stable ordering of equal-list rows across independent builds. A position is stable within the completed snapshot; its meaning is the row $\rho(p)$ shared by every associated array.

Resident IDs map positions p back to input rows $\rho(p)$. Two-pass add builds the inverse temporarily for scatter; search needs no permanent inverse. Primary ties use positions, final ties use database IDs.

4.3.2 Exact addresses and representation widths

Let $Z[m, p]$ denote the primary byte for subspace m at physical position p. The intermediate subspace-major copy has byte offset $mN + p$. Final packing produces

$$\mathsf{w}[j, p] = \sum_{t=0}^{3} 2^{8t} Z[4j + t, p], \qquad \text{addr}_{\text{byte}}(\mathsf{w}[j, p]) = 4(jN + p). \tag{4.7}$$

Here $j = 0, \ldots, M/4 - 1$. Word layout requires $M \bmod 4 = 0$; the active word-layout build rejects other values. A separate byte-layout compilation retains the byte representation. Each packed array occupies NM bytes, but repacking temporarily holds both old and new copies, costing $2NM$. The one-pass builder temporarily gathers a vector-major $[N, M]$ primary array, transposes it, and then repacks; each obsolete copy is freed. The two-pass builder scatters

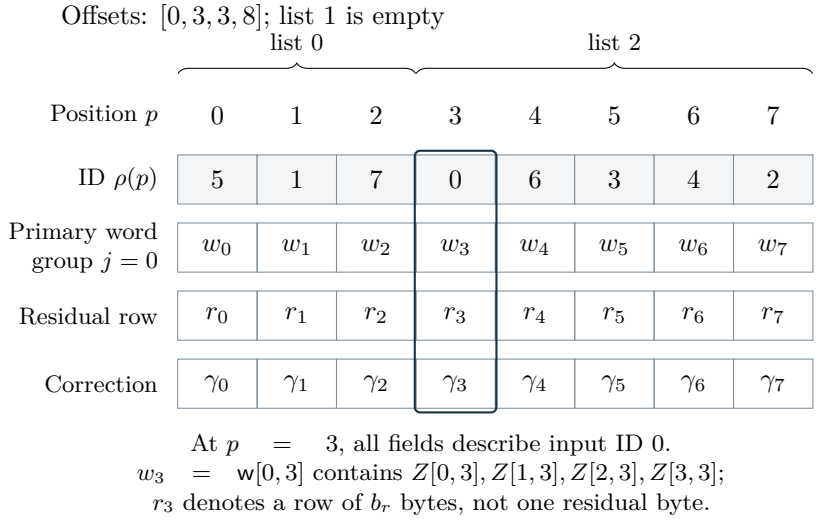


Figure 4.2. One common physical position joins IDs, packed primary words, residual rows and correction factors. This illustrative eight-row index has offsets $[0, 3, 3, 8]$, including an empty list. A primary word packs four subspaces of one vector. Values and list sizes are schematic.

directly into the subspace-major array before the same final repack. The search-time lane mapping is discussed in Section 5.3.

Residual codes remain row-major, with $b_r = \lceil dB_r/8 \rceil$ bytes per vector. In the eight-bit configuration, coordinate t of row p is at byte offset $pb_r + t$. In the four-bit path it is in byte $pb_r + \lfloor t/2 \rfloor$, with even coordinates in the low nibble and odd coordinates in the high nibble. A four-byte correction per physical row stores

$$\gamma[p] = 2\langle \widehat{p}(x_{\rho(p)}), \widehat{r}(x_{\rho(p)}) \rangle. \tag{4.8}$$

Its value must move with that row's codes during gathering or scattering. Residual access then reconstructs the same object that primary screening identified, even though the two code arrays use different layouts.

This separation favours the workload's two access patterns: primary codes are scanned broadly in list order, while complete residual rows are read for selected positions. Retaining raw FP32 vectors on the device would provide exact reranking but consume $4Nd$ additional bytes and change the reported search design. This index instead refines compressed reconstructions, with the quality consequences evaluated in Chapter 7.

4.4 Resident Storage, Workspace and Index Lifecycle

4.4.1 Stored bytes and allocated bytes

With one-byte primary subspace codes, resident row and offset bytes are

$$S_{\text{rows}} = N\left(M + \left\lceil \frac{dB_r}{8} \right\rceil + 8\right), \qquad S_{\text{offsets}} = 4(n_{\text{list}} + 1). \tag{4.9}$$

IDs and FP32 corrections add eight bytes per row. At $M = d/8$, $B_r = 8$, codes take $9d/8$ bytes, 12.5% above SQ8. Graph-free storage is not a unique compression advantage over int8.

Degree-d_g CAGRA uses $d + 4d_g$ bytes per row. For Stella at $d_g = 32$, JHQ's 19.20 GiB payload nearly equals CAGRA int8's recorded 19,656 MiB, despite different accounting scopes. JHQ instead offers graph-free construction and adjustable residual access; α does not shrink the stored array.

Basic FP32 arrays take $4d^2$ transform bytes, $4MKD_s$ primary-codebook bytes, $4MK_r$ residual-codebook bytes and $4n_{\text{list}}(d+1)$ centroid/norm bytes. The selected path adds derived arrays. Workspace holds two query matrices $[b_{\text{cap}}, d]$, centroid scores, LUTs, probes and survivors. Optional residual LUTs add $4b_{\text{cap}}dK_r$ bytes; defaults score directly. Group scratch (equation (5.13)) reserves about 93.8 MiB at 128 probes, even on the query-centric path.

Input remains caller-owned. Build staging uses $4n_b a_{\text{in}} d$ pinned-host bytes. Order/inverse arrays each take $4N$ device bytes. Assignment scratch is freed before gather (equations (4.10) and (4.13)).

Table 4.1. Per-vector bytes. JHQ includes codes, ID and correction; SQ8 is vector codes alone; CAGRA int8 models vector codes plus 32 four-byte neighbours. Offsets, model parameters, workspace and other runtime allocations are excluded. These components are not complete process totals.

d	Primary	Residual	ID + corr.	JHQ	SQ8	CAGRA int8	FP32
768	96	768	8	872	768	896	3,072
1,024	128	1,024	8	1,160	1,024	1,152	4,096
1,536	192	1,536	8	1,736	1,536	1,664	6,144
3,072	384	3,072	8	3,464	3,072	3,200	12,288

4.4.2 Reuse, recovery and invalidation

The cache stores rotation, codebooks and centroids. Its key omits code version and full database hashing, so changed code/data require explicit invalidation and model provenance. Sweeps reuse states; cold builds create new ones. Similar recall does not imply identical models.

A cache hit still performs constructor QR, warm-up, model upload and add. Stella's 6.67 s add record includes initial workspace allocation, but excludes schedule calibration and search warm-up. It is not a restart-time measurement.

Changing α needs no retraining. Before capture, search grows workspace as needed; changes to batch, survivors, probes, lanes, groups or bounds trigger recapture. Calibration can bypass capture. The index owns and reuses buffers across sequential calls, then releases them with its graph.

4.5 Construction Cost and Tested Scale

4.5.1 Bounded input batches and two-pass construction

Rotating all N raw vectors at once requires two FP32 device arrays, together $8Nd$ bytes. The builder instead stages bounded batches through pinned host memory and device raw/rotated buffers. For n_b pipeline buffers, configured add rows a_{add} and per-array byte budget S_{stage}, the source stages

$$a_{\text{in}} = \max\left\{1, \min\left(a_{\text{add}}, \left\lfloor \frac{S_{\text{stage}}}{4d} \right\rfloor\right)\right\}, \qquad S_{\text{float}} = 8n_b a_{\text{in}} d. \tag{4.10}$$

Three buffers and the default 128 MiB budget give at most 768 MiB of input/rotated device floats. Assignment products and encoder scratch are separate allocations.

One-pass add encodes input-order arrays, then gathers them into list order, temporarily keeping both copies. Two-pass add assigns/sorts IDs first, then revisits input to encode and scatter directly. It trades another read/rotation for lower simultaneous storage. Raw-input assignment against back-transformed centroids uses the real-arithmetic orthogonality identity.

4.5.2 Phase occupancy and the process peak

The one-pass capacity estimate is

$$H_{\text{single}} = N(2b_r + 3M + 20) + 2^{30} \quad \text{bytes}. \tag{4.11}$$

Ten live gather/transpose arrays contribute $3NM + 2Nb_r + 20N$ bytes: three primary copies, two residual/correction copies, assignments, sort order and IDs. The extra 1 GiB is heuristic headroom, not a full peak model.

At final two-pass repack, IDs, residuals, corrections and both primary layouts occupy

$$S_{\text{repack},2} = N(b_r + 2M + 8). \tag{4.12}$$

This counts that phase only. Whole-build peak also considers earlier pipeline/inverse scratch, model, sort, context and later workspace allocations.

Table 4.2 quantifies the compressed-copy saving. Stella's cold log selects two passes and prints 41.6 GiB for equation (4.11), against 30.6 GiB available at the tail. The separately

Table 4.2. Analytical row-array occupancy (GiB), $d = 1{,}024$, $M = 128$, $B_r = 8$. BGE uses one pass; Stella uses two. The one-pass Stella column is counterfactual capacity analysis. Pipeline, model, offsets and runtime costs are excluded from these phase values.

Dataset	Final row payload	One-pass gather	Two-pass repack
BGE	10.90	23.05	12.11
Stella	19.20	40.59	21.32

sampled resident process totals, including context and query workspace, are 11.93 GiB for BGE and 20.44 GiB for Stella (section 4.4). The supplementary 20 ms device samples observe a 26.08 GiB BGE peak near the start of add, and a 22.58 GiB Stella peak during repack. These are sampled device totals including context; shorter-lived peaks can be missed.

Assignment-product capacity uses an independent configured row count a_{assign}, selected by `JHQ_ASSIGN_BATCH` or the source default: 32,768 for at least 8,192 lists, otherwise 8,192.

$$S_{\text{dots}} = 4 n_b n_{\text{list}} a_{\text{assign}} \quad \text{bytes.} \tag{4.13}$$

Allocation capacity need not shrink with a_{in} or the rows in a particular call. Table B.3 checks four defaults: the 1,536/3,072-dimensional cases allocate 3 GiB, despite different staging rows. BGE/Stella still allocate three 4 GiB arrays, or 12 GiB, near add entry. The 23.05 GiB gather payload describes a later lifetime: adding 12 GiB to it would not give the actual peak. Peak modelling must take the maximum over stage-specific live arrays, including pipeline scratch. Stella's smaller two-pass repack payload, plus model/context, explains its lower observed peak despite its larger database.

4.5.3 Construction measurements

4.6 Index Representation and Safety Contracts

A historical 32-bit $n_t d$ overflow at 2^{31} zeroed the primary scale; a later process reloaded it from a version-blind cache. Chunked norms with double accumulation fixed it. Routing alone therefore cannot validate representation.

4.6.1 Completed representation evidence

The 13 October pilot exported cached models and decoded layouts for full Vogue (932,328), arXiv (2,253,000) and OpenAI-3072 (999,000) databases.

Codebooks passed separability, scalar-level and sorting checks. An FP64 loop checked $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$ on every exported row; maximum absolute error was 1.28×10^{-7}–2.77×10^{-7}. Sampled GPU-returned top-10 scores have maximum relative errors from 2.38×10^{-7} to 4.85×10^{-5}, using $|s_{\text{GPU}} - s_{64}|/s_{64}$ with the query-norm constant included. These are checks of the exported representation, not re-encoding from the original input vectors.

Across seven workloads and 896 query instances, the pilot additionally checked the proposed residual bounds against FP64 reconstructed distances for 72,800,590 routed candidate instances, with zero reported violations. The same population supports the FP64 composite-score identity check. GPU-returned-score checks cover only top-10 slots and skip IDs absent from the routed map, leaving output membership incompletely checked.

Encoder, primary/residual training and construction sources match the evaluated implementation after namespace normalisation and removal of the export method. This source correspondence covers the shared path, leaving later model identity and large-dataset two-pass construction to separate checks.

4.6.2 Independent input and placement audit

The supplementary exporter audit uses the shared construction path on Vogue, arXiv, OpenAI-3072 and Stella. Offsets are monotone with endpoints 0 and N; IDs form a permutation of every database row in all four. FP64 products of the stored rotation give maximum entrywise $\|\Pi\Pi^{\mathsf{T}} - I\|$ errors of 3.96–4.34 $\times 10^{-7}$. Sampled norm checks use FP64 rotation, rather than the GPU GEMM, and therefore measure the stored matrix's geometry. Table 4.3 summarises sampled recoding.

Table 4.3. Original-input FP64 recoding audit. Flips count coordinate decisions; the final column is the largest residual-boundary distance among mismatches. Structural checks cover all rows; recoding is sampled.

Dataset	Rows	Orth. error	Primary flips	Residual flips	Max margin
Vogue	200,000	4.11e-07	14	5,409	1.12e-07
arXiv	100,000	4.11e-07	1	2,515	7.31e-08
OpenAI-3072	20,000	3.96e-07	5,745	1,646,806	2.67e-05
Stella	100,000	4.34e-07	16	3,064	1.03e-07

At 768/1,024 dimensions, recoding mismatches fall within 1.12×10^{-7} of boundaries. OpenAI-3072 instead differs on 2.68% of residual coordinates, up to 2.67×10^{-5} from boundaries, consistent with TF32 rotation. Residual checks condition on stored primary digits to avoid double-counting flips.

ID-joined one-/two-pass builds have identical codes on the three smaller databases. Correction differences reach 8.94×10^{-8}; assignments differ on 1 Vogue, 1 arXiv and 40 OpenAI rows. Stella covers only its natural two-pass build. Exported centroids are absent, leaving membership decisions unchecked.

4.6.3 Contract coverage and remaining checks

Construction requires $M \mid d$, $D_s \mid B$ and $M \bmod 4 = 0$. It also needs finite inputs, representable counts and reserved sentinels. IDs/positions are signed 32-bit; address products use wider arithmetic.

T4 checks selection separately (section 7.4.1). Export audits do not certify centroid membership, empty lists or adversarial tails.

Table 4.4. Representation contracts after the pilot and supplementary input audit. Coverage separates stored geometry, sampled arithmetic and row placement.

Boundary	Required property	Evidence / remaining check
C0: geometry	Common transform with bounded floating-point isometry error	Stored-matrix orthogonality checked on four datasets; TF32 GPU arithmetic distinguished
C1: codebooks / correction	Cartesian levels, sorted residual codebooks, correction matches decoded codes	Pilot checks completed on three databases; all-row correction errors in section 4.6
C1: input encoding	Stored codes encode the intended original input row	FP64 original-input recoding on four samples; boundary and TF32 discrepancies quantified
C1: placement	Offsets cover rows; IDs form a permutation; all fields follow it	All-row offsets/ID checks; three ID-joined path comparisons; centroid membership unverified
Execution	Valid indexing, synchronisation and score-order contracts	Archived query and sanitizer checks; finite coverage, detailed in section 7.4

5 Representation-Aware GPU Search

The transform and primary/residual hierarchy follow JHQ [16]. This chapter develops their GPU scoring and computed-score selection; Chapters 6 and 7 cover measurement and budget choice.

5.1 Query Path and Four Distinct Ranking Objects

A query follows rotation $q' = \Pi q$, IVF routing, primary screening and residual refinement. Final selection returns k IDs and computed scores. Figure 5.1 distinguishes the three runtime selection decisions; the ground-truth ordering is an external evaluation object.

Four quantities must be kept separate:

$$\begin{aligned} s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,} \\ D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,} \\ D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,} \\ D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.} \end{aligned} \tag{5.1}$$

Here μ_i is an IVF centroid in transformed coordinates, $\widehat{p}(x)$ is the primary reconstruction and $\widehat{r}(x)$ is the decoded residual. The runtime refined score includes a query-only constant, as derived from the implemented correction in Section 5.6. It preserves the ordering of reconstructed distances for a fixed query, but neither it nor the primary score is the original-vector distance.

Routing defines the visited universe U of n_c candidate occurrences; primary screening retains up to $c = \max\{k, \lceil \alpha k \rceil\}$ for residual scoring. Table 5.1 records each stage's inputs, reads and costs; sections 5.2 to 5.7 develop their implementations.

The ordering and exactness columns of table 5.1 attach each claim to its score and tie rule. Chapter 7 tests survivor sets, ordered outputs and raw-space recall separately.

Table 5.1 gives the stage contracts. The E1/budget harness explicitly selects 16 refinement lanes and scan policy -1 (scheduling among 0/2/4/8), versus library defaults of scalar refinement and policy 0. Shared bounds are enabled but used only at $g_{\mathrm{scan}} > 1$; the harness calibrates scheduling with 512 database pseudoqueries, a step the library does not perform automatically. Component ablations retain their stated overrides; table A.1 maps mechanisms to recorded implementations.

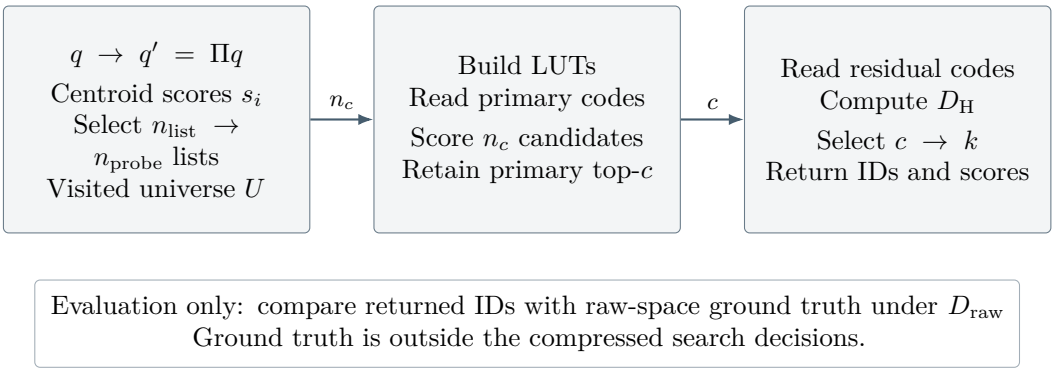


Figure 5.1. Query execution and its separate selection boundaries. Routing selects lists, primary screening selects vector candidates, and final selection ranks refined survivors. Raw-space ground truth is used for evaluation; it is not an input to these search decisions.

Table 5.1. Query-stage contracts. $c_v = \min(c, n_c)$ counts valid primary survivors. Exactness refers to the stated computed scores; routing and quantised reconstruction determine raw-space approximation.

Stage	Input → output	Data read	Main work	Order / exactness
Transform	$q \to q'$	Query; transform	Apply Π	Transform (precision in section 7.2.3)
Probe	$n_{\text{list}} \to n_{\text{probe}}$ lists	FP32 centroids; list lengths	Score; radix; rank count	(s_i, i); exact probe selection
LUT	$q' \to$ tables	Primary codebook	$MD_s 2^{8/g_{\text{LUT}}}$ terms	Real-arithmetic factorisation; FP32 rounding
Primary, $g_{\text{scan}} = 0$	$n_c \to c_v$	Packed primary codes; LUT	$O(n_c M)$ scoring; compaction	Exact distance-order selection; arbitrary cutoff ties
Primary, $g_{\text{scan}} \geq 1$	$n_c \to c_v$	Same primary representation	Score; compact; merge	(D_{P}, p); exact under stated premises
Refine	$c_v \to c_v$ scores	Residual codes; correction	$O(c_v d)$	Computed D_{H}; lane-dependent FP32 association
Final	$c_v \to k$	Refined scores; IDs	$O(kc)$ slot scans	$(D_{\text{H}}, \text{ID})$; exact with k finite valid entries

5.2 Exact Cartesian LUT Factorisation

Primary ADC keeps q' in FP32 and reconstructs stored codes [16, 17]:

$$T_m[z] = \|q'_m - \mathcal{C}_m[z]\|_2^2, \qquad D_{\text{P}}(q, x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

An eight-bit label normally needs 256 entries. The useful extra structure is inside each Cartesian subcodebook, beyond the PQ subspace partition.

At $K = 256$, $D_s = 8$, the analytical codebook is

$$\begin{aligned} a &= \sigma\Phi^{-1}(3/4) \simeq 0.67449\sigma, \qquad \mathcal{C}_m = \{-a, +a\}^8, \\ D_{\text{P}}(q, x) &= \|q'\|_2^2 + da^2 - 2a \sum_{j=0}^{d-1} s_j(x) q'_j. \end{aligned} \tag{5.3}$$

Φ is the standard normal CDF. Each byte stores eight signs, higher bits first. For $a > 0$, primary order equals sign-vector inner-product order. Cartesian structure also permits exact four-/eight-way splits; a learned replacement needs compatibility checks.

5.2.1 Factorisation without changing the quantiser

The high nibble selects the first four coordinates and the low nibble selects the last four: $\mathcal{C}_m[16h + l] = (A_m[h], B_m[l])$. Their tables are

$$T_m^{\text{hi}}[h] = \|q'_{m,\text{hi}} - A_m[h]\|_2^2, \qquad T_m^{\text{lo}}[l] = \|q'_{m,\text{lo}} - B_m[l]\|_2^2. \tag{5.4}$$

Disjoint squared-L2 sums give

$$T_m[z] = T_m^{\text{hi}}[z \gg 4] + T_m^{\text{lo}}[z \,\&\, 15]. \tag{5.5}$$

This stores 32 entries instead of 256, without changing real-arithmetic scores, codes or candidates.

The builder takes coordinate slices from codewords $16h$ and l. This requires all codewords sharing a nibble to share that slice. `verify_split_lut` rejects incompatible tables. An

arbitrary learned 256-centroid PQ subcodebook need not satisfy this condition: dividing its code labels into nibbles does not factor its centroid coordinates.

The same-candidate FP32 audit in section 7.3 measures numerical and ranking differences between table forms, including changes after identical primary survivor sets.

5.2.2 Grouping, construction work and resource cost

The table-grouping ablation generalises the split to $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ contiguous coordinate groups whose bit fields match the Cartesian digits. For eight bits and $D_s = 8$, each group has $2^{8/g_{\mathrm{LUT}}}$ entries; the total entries and FP32 storage per query are

$$n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = g_{\mathrm{LUT}} 2^{8/g_{\mathrm{LUT}}}, \qquad S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = 4M n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \text{ bytes.} \tag{5.6}$$

Independent entry construction evaluates $D_s 2^{8/g_{\mathrm{LUT}}}$ coordinate contributions per subspace. Two groups therefore reduce table storage eightfold and coordinate work sixteenfold relative to independently constructing every full-table entry. These are table-level cost ratios, not predicted application speedups.

The table is built in global workspace and staged cooperatively when its complete shared-memory footprint fits the queried per-block opt-in limit $S_{\max}$. For candidate-buffer capacity C_{buf}, the ordinary exact scan and the grouped scan respectively require

$$S_0 = 8C_{\mathrm{buf}} + 8 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}), \qquad S_{\mathrm{grp}} = 8C_{\mathrm{buf}} + 24 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \quad \text{bytes.} \tag{5.7}$$

These formulas concern the evaluated scan without optional fused refinement or early-exit scratch. The code selects shared lookup exactly when the corresponding footprint is at most $S_{\max}$.

In the table sweep, $c = 1{,}000$, block sizes are 512 or 1,024, and $C_{\mathrm{buf}} = 2{,}048$. Full tables span 96–384 KiB, versus 12–48 KiB for two groups. NVIDIA specifies a 99 KiB per-block limit for compute capability 12.0 [24]. With this limit, the full-table footprint exceeds capacity even at $M = 96$ (114,696 bytes in the query-centric path), whereas the largest two-group footprint is 65,560 bytes in the grouped path. Thus the source-level placement rule sends full tables to global memory and two-group tables to shared memory in these configurations. This placement is inferred from the launch rule and published limit. The measurement records do not include a device-attribute query.

Four and eight groups both store 16 entries per subspace but perform more lookups and combinations. Extraction, lookup dependencies, staging, synchronisation and residency jointly determine the measured trade-off. Multiple scan groups may stage the same table independently.

The $16 + 16$ representation is a nibble LUT for an eight-bit sign code. Small LUTs have precedents in Fast Scan, Quicker ADC and IVF-RaBitQ [1, 2, 33]; JHQ retains its own two-level score. Our contribution is their GPU integration with retention and refinement. Two groups lead in 22/24 measured cells; other margins are below 1% (section 7.3).

Table 5.2. Cartesian table trade-offs for an eight-bit, eight-coordinate primary subspace. Combining additions are the algebraic minimum for joining the looked-up values, excluding accumulation across subspaces; they are not measured instruction counts.

g_{LUT}	Entries	FP32 bytes	Lookups	Combining adds	Build terms
1	256	1,024	1	0	2,048
2	32	128	2	1	128
4	16	64	4	3	32
8	16	64	8	7	16

5.3 Packed Codes and Candidate-Parallel Memory Access

Neighbouring scan lanes process neighbouring candidates. Subspace-major storage makes their code accesses adjacent; LUT factorisation separately reduces the query-specific table.

5.3.1 Address mapping and four-byte packing

For list ℓ, row r has position $p = o_\ell + r$. Code $Z[m, p]$ lies at byte offset $mN + p$, rather than candidate-major $pM + m$. Adjacent lanes then read adjacent candidates at fixed subspace m. A warp crossing lists can still jump in memory.

Four subspace bytes from one candidate form one word:

$$\mathsf{w}[j, p] = \sum_{t=0}^{3} 2^{8t} Z[4j + t, p], \qquad \text{offset}_{\text{bytes}}(\mathsf{w}[j, p]) = 4(jN + p). \tag{5.8}$$

Shifts and masks recover each byte exactly: $(\mathsf{w}[j, p] \gg (8t)) \,\&\, 255$. Its nibbles index equation (5.5). This packs subspaces of one vector, not four candidates.

The build path explicitly repacks the $[M, N]$ byte array into $[M/4, N]$ words. Merely reinterpreting the original byte array as `uint32_t` would combine consecutive candidate bytes rather than the required subspaces. The implementation requires M divisible by four and rejects a partial final word. The repack reads and writes NM code bytes once at construction, with old and new storage briefly coexisting; it is outside the search timing boundary.

5.3.2 What packed access saves, and what it preserves

One word load replaces four byte loads, followed by register unpacking. For 32 positions the logical window is 128 bytes; alignment and active lanes determine transactions. Payload remains M bytes per candidate. Since both alternatives are subspace-major, the experiment isolates neither transpose nor cache effects.

Section 7.3 evaluates the packed path's throughput, recall and variability against the byte path.

5.4 Exact Global Primary Top-c via Multi-Block Scanning

Grouped scanning adds blocks within each query, then merges survivors [18]. It preserves computed-primary selection, not exact raw-vector neighbours.

5.4.1 Candidate partition and deterministic ordering

For a fixed query, concatenate the selected inverted lists in probe order to obtain n_c candidate occurrences. Assume that each visited database row occurs once and that the index

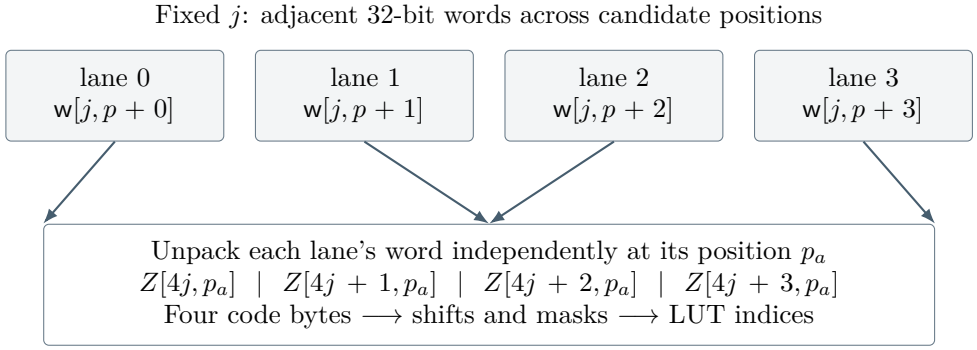


Figure 5.2. Packed candidate-parallel access, showing a subset of warp lanes. At fixed word group j, consecutive positions access adjacent four-byte words. Each word contains four subspace codes of one candidate. Unpacking preserves each code byte; LUT splitting happens afterwards. The diagram illustrates addresses, not a measured number of memory transactions.

stays fixed during search. For $g_{\text{scan}} \in \{1, 2, 4, 8\}$, block (q, j) owns the interval

$$U_j = \left[\left\lfloor \frac{jn_c}{g_{\text{scan}}} \right\rfloor, \left\lfloor \frac{(j+1)n_c}{g_{\text{scan}}} \right\rfloor \right), \qquad j = 0, \ldots, g_{\text{scan}} - 1. \tag{5.9}$$

Here an interval denotes the corresponding candidates, rather than their database IDs. The device computes its endpoints from the routed candidate count. Groups differ in size by at most one candidate; a long list can cross a group boundary. This balances candidate counts, whereas assigning equal numbers of lists need not do so. Each thread finds its first list through the probe-prefix offsets and advances its list cursor while scanning its interval. Residual codes are not read at this stage.

The selection key is

$$K_q(x) = \big(D_{\text{P}}(q, x), p(x)\big), \qquad (a, p) \prec (b, r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.10}$$

where $p(x)$ is the physical row position in the fixed inverted-list storage. It is unique among the query's candidates and independent of the order of atomic appends. It is not the database ID used to break final-result ties. With non-NaN scores, this defines a total order on distinct candidates. All local comparisons and merge decisions use that same order. Partition-independent selection additionally requires that each candidate receive the same computed score in every compared path; the ordering rule alone cannot remove floating-point scoring differences.

5.4.2 Exact local retention and global coverage

$\text{Top}_c(V)$ is the first $\min(c, |V|)$ elements under K_q, with $c = \max\{k, \lceil \alpha k \rceil\}$. Each group needs its full local top-c: all global winners could lie in one group. A c/g_{scan} quota can lose them.

Lemma 5.1 (Local-buffer retention)**.**
Let V_t be the candidates scored by a block through chunk t, and B_t its valid buffer after any compaction or filtering at that point. Assume fixed complete keys, sufficient append capacity and exact top-c compaction. Let β_t be the minimum of the complete shared bounds observed so far, including the initial top sentinel. Then

$$\text{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\} \subseteq B_t. \tag{5.11}$$

With sharing disabled, the bound is the sentinel and the buffer contains the whole prefix top-c.

Proof.
The empty prefix satisfies the invariant. Local rejection has c earlier witnesses, so the rejected key cannot become a later prefix winner. Shared rejection excludes only keys above β_t. Every other eligible winner is already retained or appended. Exact compaction keeps it because fewer than c prefix keys precede it; filtering removes only keys above an observed bound. Induction over these operations proves the claim. The running minimum handles stale reads of previously published bounds. □

Proposition 5.1 (Coverage and truncated merging)**.**
For fixed keys under a common total order, let U be the visited universe and V any retained subset. If $\mathrm{Top}_c(U) \subseteq V \subseteq U$, then $\mathrm{Top}_c(V) = \mathrm{Top}_c(U)$. Pairwise sorted merging with truncation to c preserves this result. In particular, for disjoint groups,

$$\mathrm{Top}_c\left(\bigcup_j U_j\right) = \mathrm{Top}_c\left(\bigcup_j \mathrm{Top}_c(U_j)\right). \tag{5.12}$$

The statement includes universes smaller than c.

Proof.
Every global winner is in V, and every other member of V follows the global winners. Selection from V therefore recovers them. For the independent-group special case, let x belong to the global top-c and to group U_j. Fewer than c candidates precede x globally, so fewer than c precede it within U_j. Consequently $x \in \mathrm{Top}_c(U_j)$, and the local union contains every global winner. Selecting its first c therefore recovers precisely the global winners in the same order. If the whole universe has fewer than c members, every member is retained.

For merging, the first c of two sorted lists are their union's top-c. A discarded candidate already has at least c predecessors within that union and cannot enter the top-c of any larger union. Induction over the merge tree establishes the claim. □

The CUDA implementation maintains a shared-memory buffer of C_{buf} key pairs, where C_{buf} is a power of two and $C_{\mathrm{buf}} \geq c + T_{\mathrm{blk}}$ for a scan block of T_{blk} threads. A compaction sorts this buffer with a bitonic network, retains at most c valid entries and records the last retained key as the local threshold when c entries exist. Before that point the threshold is a sentinel above every valid key. Between compactions, only candidates preceding the threshold are appended. Its c witness candidates make the threshold a safe upper bound for the best c of the scanned prefix, even as further candidates arrive.

Each chunk appends at most T_{blk} entries. Compaction precedes any chunk that would exceed capacity, leaving at most c entries and space for the next chunk because $C_{\mathrm{buf}} \geq c + T_{\mathrm{blk}}$. This supplies the capacity premise of Lemma 5.1 without a per-thread quota. With sharing, Proposition 5.2 bounds every observed threshold above the global cth key. The buffer lemma and coverage proposition therefore preserve global winners through filtering and merging.

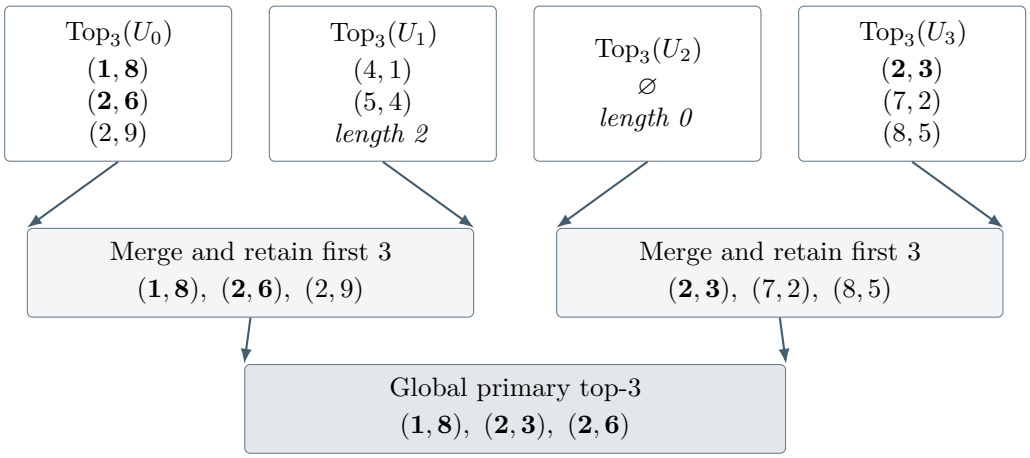


Figure 5.3. Local coverage and truncated pairwise merging, illustrated with $c = 3$ and $g_{\mathrm{scan}} = 4$. Entries are (D_{P}, p) keys, not measured data. Bold entries are the final global winners. A short group contributes its valid prefix; an empty group contributes no entries. Every merge uses the same key order and retains at most c entries.

Materialising all n_c score/position pairs needs $O(n_c)$ writes/storage. Fused scoring instead repeatedly sorts bounded local buffers and emits $O(g_{\text{scan}}c)$ entries. Register selection is another established option [18]; the experiments test the chosen scan variants.

5.4.3 Merge implementation, storage and boundary cases

For $g_{\text{scan}} > 1$, one 256-thread block merges each query over $\log_2 g_{\text{scan}}$ rounds. Each pair emits $\min(c, a + b)$ entries. Merge Path finds thread slices by binary search, then each thread merges its slice [14]. All comparisons use equation (5.10); block barriers separate rounds. Figure 5.3 illustrates the truncation invariant. The tree emits at most $c(g_{\text{scan}} - 1)$ key pairs across its internal nodes, in addition to the partition searches and synchronisation work.

The two global score/position workspaces have capacities $[b_{\text{cap}}, g_{\text{scan}}, c]$ and $[b_{\text{cap}}, g_{\text{scan}}/2, c]$ and alternate between rounds. With four-byte scores and positions, their combined payload is

$$S_{\text{pairs}} = 8b_{\text{cap}}c\left(g_{\text{scan}} + \frac{g_{\text{scan}}}{2}\right) \quad \text{bytes}, \qquad g_{\text{scan}} > 1. \tag{5.13}$$

Group counts require another $4b_{\text{cap}}g_{\text{scan}}$ bytes; shared bounds add $8b_{\text{cap}}$ when provisioned. These are additional to final survivor arrays and other search workspace. Allocation grows before graph capture and is retained for reuse, so choosing a smaller g_{scan} later does not necessarily release the reserved memory. The merge keeps only list lengths in shared memory rather than all $g_{\text{scan}}c$ keys.

Valid counts delimit each local prefix. Sort padding uses $(+\infty, \text{INT_MAX})$; valid positions must be smaller. Empty groups have count zero. Output padding instead uses $(+\infty, -1)$, which refinement skips and merging must not treat as valid. $g_{\text{scan}} = 1$ writes survivors directly, without merging.

$g_{\text{scan}} = 0$ labels one block per query. Its default selector is exact by distance but can choose different cutoff-tie IDs from the complete-key path. The lossy `K_LOCAL=4` alternative requires `JHQ_EXACT_TOPCK=0`; paper-faithful builds disable it.

For distance-only compaction, every rejected score has c retained witnesses no larger than it. The final buffer therefore contains all scores below the cutoff and enough equal scores, without fixing tied IDs. T4 checks this preorder and complete-key grouped selection (section 7.4.1). The reference sorts GPU-produced scores, rather than recomputing original-space distances on the CPU.

5.5 Shared Complete-Key Thresholds and Execution Scheduling

Full local top-c retention establishes correctness, but can produce $g_{\text{scan}}c$ intermediate survivors for only c global outputs. The shared-bound component reduces this excess selection work. The scheduler then chooses how much scan parallelism to expose. Both operate at a fixed refinement budget; choosing that budget is the separate problem in Chapter 6.

5.5.1 A complete-key upper bound with explicit witnesses

Let $K_{(c)}$ be the global cth key when $|U| \geq c$. If a group holds c distinct, fully scored candidates, the largest key among those witnesses, τ_j, satisfies $K_{(c)} \preceq \tau_j$. There are already c candidates no later than τ_j in the global universe. The shared bound is initialised to a top sentinel and updated by

$$\tau \leftarrow \min_{\prec}(\tau, \tau_j). \tag{5.14}$$

No bound is published from fewer than c valid candidates. If the entire universe has fewer than c, no group can publish such a bound and all valid candidates remain eligible.

Proposition 5.2 (Safe rejection by a witnessed bound)**.**
Assume fixed keys, publications backed by c distinct candidates, and reads that return either the initial sentinel or one complete published key. Rejecting a candidate only when $K_q(x) \succ \tau$ cannot remove a global top-c candidate, irrespective of publication timing.

Proof.
Every published bound is at least $K_{(c)}$, and the minimum of such bounds is also at least $K_{(c)}$. A global winner satisfies $K_q(x) \preceq K_{(c)} \preceq \tau$, so it is never rejected. An older bound remains an upper bound and merely permits extra work. Publications made after earlier safe rejections remain valid: any c retained distinct candidates still supply c witnesses, whether or not they are the best c of their group. □

Sharing may remove local winners certified as globally uncompetitive; the retained union still satisfies Proposition 5.1. Equality with the shared bound remains eligible. Admission requires

$$K_q(x) \prec \lambda_j \quad \text{and} \quad K_q(x) \preceq \tau_{\text{seen}}, \tag{5.15}$$

where λ_j is the local compaction threshold and τ_{seen} is the group's last observed shared bound.

5.5.2 Publication, filtering and implementation limits

Nonnegative FP32 squared-sum scores and positions are packed as

$$\kappa(D, p) = 2^{32}\,\text{uintbits}_{32}(D) + p. \tag{5.16}$$

Unsigned comparison matches lexicographic order for positive-zero, non-NaN scores and valid nonnegative positions. Infinity follows finite scores; the initial all-one word follows valid keys. Invalid positions cannot publish bounds.

One thread polls the bound per chunk. After compaction, c valid entries permit publishing the local cth key with 64-bit `atomicMin`. A final publication and binary search trim the sorted

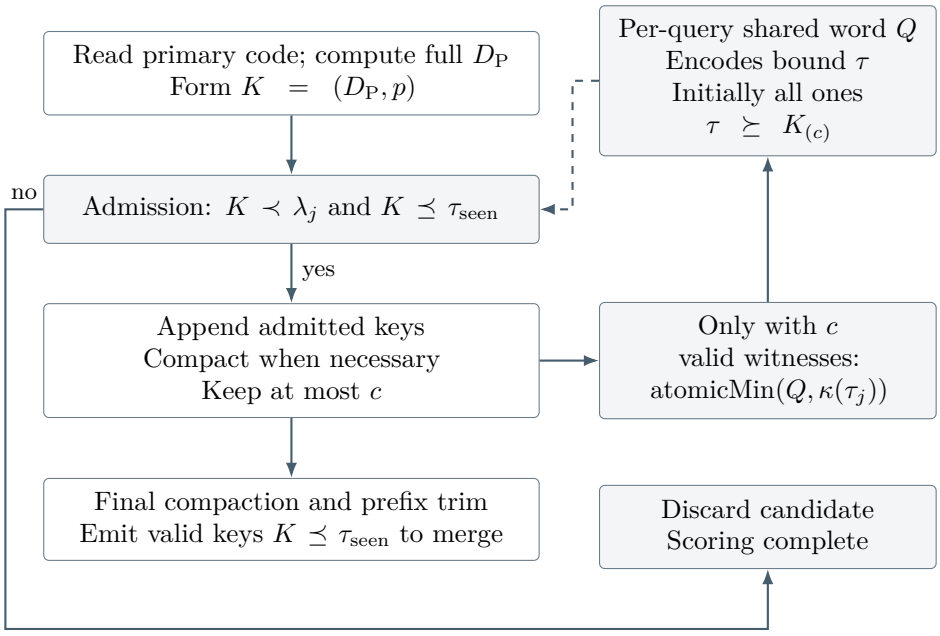


Figure 5.4. Shared complete-key filtering for one scan group. A candidate's full primary score is computed before admission. A valid c-candidate witness set can tighten the per-query bound; a stale but complete bound only weakens filtering. Final prefix trimming reduces the valid merge input, while the allocated workspace capacity remains unchanged. The whole-key-read premise is qualified in the text.

prefix. Groups never wait for one another: stale bounds leave extra entries for exact merging. Each search resets bounds in-stream; merging follows scan completion.

Extracted PTX uses aligned `ld.volatile.global.u64` and `atom.global.min.u64`. PTX's single-copy atomicity for compatible same-size strong accesses supplies the whole-key premise [26]. CUDA 13.0 `ptxas` for `sm_120` corroborates this (section A.4); actual driver-JIT SASS was not captured. The argument does not rely on C++ `volatile` alone [25].

Bounds are checked after full primary scoring (figure 5.4). They save buffer/merge work, not distance arithmetic, and add polling/atomics. Section 7.4 includes a slowdown.

5.5.3 Choosing parallelism without changing the budget

Grouping launches bg_{scan} blocks rather than b, adding parallelism but duplicating staging/selection and requiring merging. Smaller static quotas would reduce intermediate storage, but require a certificate or fallback to preserve coverage; the pilot's concentrated survivor distribution motivated using witnessed thresholds instead (Section 7.4).

The implemented scheduler uses a fill proxy and a workload-skew statistic. Let T_{blk} be scan threads per block, H the number of SMs times the maximum resident threads per SM, and $v = \text{sd}(n_c)/\,\text{mean}(n_c)$ the candidate-count coefficient of variation. Its rule is

$$g_{\text{fill}} = \max(\{1\} \cup \{h \in \{2,4,8\} : bhT_{\text{blk}} \le 0.6H\}), \tag{5.17}$$

$$g_{\text{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad g_{\text{chosen}} = \max(g_{\text{fill}}, g_{\text{skew}}). \tag{5.18}$$

The skew rule may override the fill limit. In the evaluation harness, v is calibrated using 512 database vectors as pseudoqueries and a fixed routing configuration; it uses no ground-truth labels. This is distinct from Chapter 6's sampled output-agreement calibration. Once calibrated, scheduling uses the batch size and stored statistic without an additional per-search host readback. The constants were fitted on four pilot configurations and then evaluated on further configurations. The one-time pseudoquery calibration is excluded from the reported steady-state search times. Supplementary timing separates host gathering (0.81–7.04 ms), the first calibration call (6.00–27.77 ms) and repeat medians (2.72–18.13 ms). This startup cost is separate from refinement-budget calibration (section A.4).

A chosen group count of one dispatches $g_{\text{scan}} = 0$, keeping index, probes and budget fixed but inheriting distance-only cutoff ties. Identical survivors thus require a common-order check. Integrated checks, gains and regret appear in section 7.6.

5.5.4 Observed execution paths

On the 170-SM RTX 5090, 48 warps/SM gives $0.6H = 156{,}672$ threads [24]. At batch 1,000 and $T_{\text{blk}} = 512$, two groups need a 1,024,000-thread proxy. Fill therefore chooses one; skew can force four.

Nine of 38 E1 run records cross $v = 0.3$, all Vogue (0.338–0.622). Their 132 run–budget cells contain 27 Vogue cells scheduled to four groups and 105 arXiv/OpenAI cells scheduled to the query-centric path. Non-Vogue full-batch gains therefore use LUTs, packed access, lane refinement and probe selection, without grouped retention or shared bounds.

Vogue checks find 3–15 tie-only primary differences per 1,000 queries, zero non-tie differences and identical final outputs. The 7–11 range applies only at 128 probes.

Held-out budget tests use one block in ten cells and four groups in two Vogue cells. At $S = 128$, eight use two groups, two use four and two use one block: 8/12 change paths. Grouping mainly serves smaller/skewed workloads, without a fixed batch-256 cutoff.

The heuristic omits register/shared-memory residency, compaction and merge costs, and its pseudoqueries can become stale. Its worst recorded regret is 1.217 at batch 256. Measurements hold the refinement budget fixed; section 5.6 processes the resulting survivors.

5.6 Lane-Cooperative Residual Scoring and Final Top-k

Residual access is restricted to primary survivors, reducing coordinate work from $O(n_c d)$ to $O(cd)$. Lane cooperation accelerates that work at a fixed budget.

5.6.1 Reconstructed scoring and the query-only constant

With primary reconstruction $\widehat{p}$, residual $\widehat{r}$ and stored correction $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$, refinement computes

$$D_{\mathrm{H}}(q, x) = D_{\mathrm{P}}(q, x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) \tag{5.19}$$

$$= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. \tag{5.20}$$

Norm expansion gives the second equality. The query-only constant preserves ranking, while reconstruction error remains [16].

The scalar path sums coordinate-wise residual differences from q', staging queries and, when possible, codebooks in shared memory. It then adds the primary score and correction.

5.6.2 Mapping lanes to words and reducing partial sums

At $\ell_{\mathrm{ref}} = L \in \{1, 8, 16, 32\}$, adjacent lanes share one candidate. A residual word holds $d_{\mathrm{word}} = 32/B_R$ codes: four for eight-bit, eight for four-bit. Lane a reads words $a, a + L, \ldots$; butterfly shuffles sum partials and lane zero writes the score.

Rows must be word-aligned and $d_{\mathrm{word}} \mid D_s$, keeping words within a codebook. All lanes join shuffles; invalid slots contribute zero, then receive infinite output scores. At $L = 16$, each warp handles two candidates.

L0 uses scalar byte loads; L1 packs word loads but keeps one lane and the accumulation order. L8/L16/L32 divide coordinate work, shortening each walk at the cost of shuffles and fewer concurrent candidates. Larger L need not win; survivor locality and codebook placement still matter.

5.6.3 Final selection and the numerical boundary

Given k finite valid survivors with unique IDs, one block selects exact top-k under $(D_{\mathrm{H}}, \mathrm{ID})$. Each round scans all c slots, emits the minimum and sets its score to infinity. There is no per-thread quota. If finite entries run out, a valid smallest ID can repeat with infinity; an entirely invalid pool returns invalid IDs. Evaluated pools satisfy the finite-entry contract.

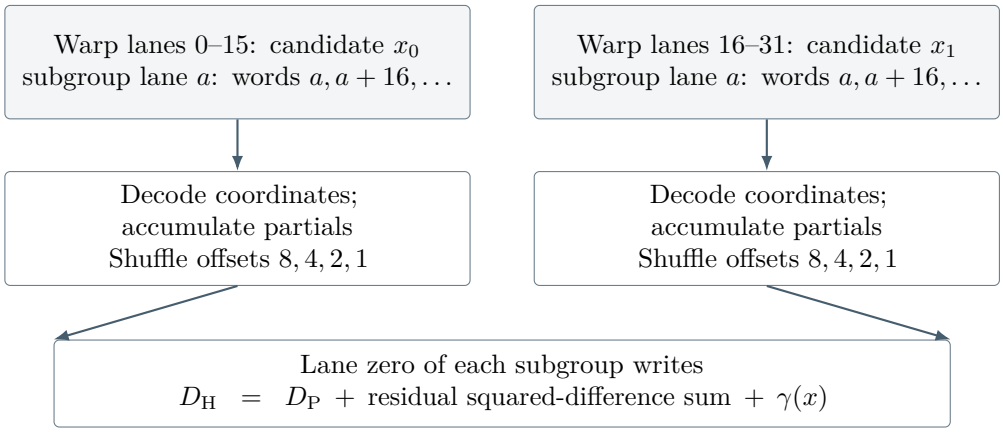


Figure 5.5. Sixteen-lane residual refinement. A warp handles two candidates; lane a within a candidate reads residual words $a, a + 16, \ldots$. With eight-bit residual codes, each 32-bit word decodes four coordinates. Subgroup reductions combine partial scores before adding the retained primary score and correction. Candidate positions can be scattered even when each subgroup's word access is contiguous.

Final ties use database IDs; primary ties use positions. Refinement cannot restore excluded candidates or remove quantisation error. Lane reductions change FP32 association and may move near ties.

Section 7.3 reports the load-only and cooperative controls, numerical differences, kernel times and end-to-end gains separately.

5.7 IVF Probe Selection as an Independent Primitive

Probe selection precedes primary scanning and selects inverted lists, not database-vector survivors. For IVF centroid μ_i, the selector orders the FP32-valued computed routing scores

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.21}$$

Omitting the common query norm preserves centroid ordering. In contrast to primary squared-distance tables, these scores can be negative. The required output is the first $n_{\text{sel}} = \min(n_{\text{probe}}, n_{\text{list}})$ centroid IDs in score order, using the smaller list ID at a tie, plus the prefix offsets of their list lengths.

OpenAI rotation and centroid GEMMs use TF32 inputs with FP32 accumulation in v69. Probe selection is exact for those computed scores; later FP32 `fma` cannot undo GEMM rounding (section 7.2.3).

5.7.1 Radix selection followed by deterministic ordering

The radix replacement uses one block per query and four most-significant-byte-first passes. Each histogram includes the chosen prefix only. Selecting its next bin updates the remaining rank; four passes determine the cutoff and required equal-key count.

Signed zeros first map to positive zero; then float bits u map to

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.22}$$

This orders non-NaN values, including negatives and infinities, unlike equation (5.16). Equal zeros share a key. Every pass uses the same rounded `fma`, keeping recomputed keys fixed.

The 256-bin histogram has 32 lane replicas: 32 KiB of counters. This separates same-warp updates but leaves inter-warp contention, trading shared storage and reduction for less concentrated atomics.

Collection takes all keys below cutoff. Ballots and prefix counts rescan cutoff ties in list-ID order, so atomic arrival cannot choose the tied subset. Each selected entry then counts its predecessors under (s_i, i) and writes to that rank, avoiding a full centroid sort.

A block prefix scan converts selected list lengths into offsets. Excess probes repeat the last ID with zero length, so they do not rescan its list. Centroid sets and requested probe count must be nonempty/positive.

5.7.2 Costs and separate correctness contracts

Histogram/collection passes reread scores; ranking costs $O(n_{\text{sel}}^2)$. Larger selections also need more shared storage, while small ones still pay histogram allocation. We evaluate this existing radix method's deterministic integration, not a uniformly linear replacement.

Tests include cutoff ties, signed zeros, infinities, extreme values and excess probes (section 7.4).

The optional FP16 routing changes score precision and is evaluated as a separate approximation in section 7.4, alongside the cross-process pipeline-identity checks.

6 Workload-Level Budget Calibration

The policy selects one workload-level refinement multiplier through unlabelled agreement with a larger-budget reference, then reuses it across batches. Quality and calibration cost constrain that reuse.

6.1 Why Fixed Refinement Budgets Can Be Wasteful

For routed universe $U(q)$, JHQ's refinement multiplier sets [16]

$$c(\alpha) = \max\{k, \lceil \alpha k \rceil\}, \qquad C_\alpha(q) = \mathrm{Top}^{\mathrm{primary}}_{\min\{c(\alpha), |U(q)|\}} U(q). \tag{6.1}$$

C_α contains valid survivors; underfull output slots receive sentinels. Analysis requires at least k valid candidates. α changes retention within U, while probes change U itself.

Too much refinement wastes residual reads; too little can lose neighbours. The grid spans 20–2,000 survivors at $k = 10$ without retraining. Smaller budgets do not release stored residuals or necessarily shrink previously grown workspace.

For fixed batch and routing, the cost split is

$$T(\alpha) = T_{\mathrm{base}} + T_{\mathrm{select}}(c(\alpha)) + T_{\mathrm{refine}}(c(\alpha)) + T_{\mathrm{final}}(c(\alpha)). \tag{6.2}$$

T_{base} covers transfers, rotation, routing and primary scores; survivor-dependent work follows figure 6.1. Capacity/resource thresholds prevent a monotone-latency claim. The rule minimises accepted budget, not noisy QPS (section 7.5.5).

Each calibration trial still runs rotation, probing and primary scoring before refinement. Reusing the index avoids rebuilding or retraining; it does not cache every intermediate query result across budget trials. The cost of calibration must therefore count full searches, not merely the difference between two residual-scoring workloads.

The policy selects one budget for an index, routing configuration, k and query workload. It uses neither a trained per-query predictor nor an online choice for each arriving query. Scan-group scheduling remains the separate mechanism of section 5.5; its execution path must still be considered when interpreting budget calibration.

Position among adaptive ANN methods. α serves the candidate-expansion role of Faiss's `k_factor` and cuVS refinement [7, 10]. Alternatives use GT tuning [9], learned per-query stopping [19] or predicted recall [6]. Here unlabelled agreement chooses one reusable workload budget. Scores still use reconstructed vectors, unlike the cuVS source-vector refinement example.

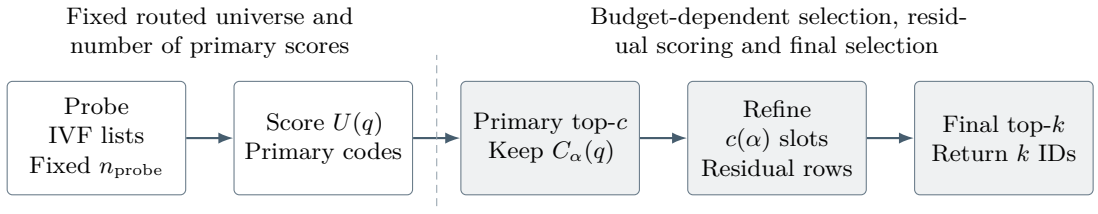


Figure 6.1. The refinement budget changes work after primary-score production. With a fixed query, index and probe count, it leaves the routed candidate universe unchanged. The highlighted stages depend on survivor capacity; this is a logical work decomposition, not a measured latency breakdown. The scan kernel also performs budget-dependent admission and compaction. Padding applies when fewer than c candidates exist.

6.2 GT-Free Sampled Output Agreement

Let $\mathcal{R}_\alpha(q)$ be the set of k valid, unique database IDs returned at budget α on a fixed index. Define $\mathcal{R}_{\text{ref}}(q) = \mathcal{R}_{\alpha_{\text{ref}}}(q)$. The descending grid is

$$A = (200, 100, 64, 32, 16, 8, 4, 2), \qquad \alpha_{\text{ref}} = A[0] = 200. \tag{6.3}$$

For S calibration queries $\mathcal{Q}_S$, define

$$\begin{aligned} D_S(\alpha) &= \sum_{q \in \mathcal{Q}_S} \left(k - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{\text{ref}}(q)|\right) \\ &= \frac{1}{2} \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_\alpha(q) \triangle \mathcal{R}_{\text{ref}}(q)|. \end{aligned} \tag{6.4}$$

A budget is accepted iff $D_S(\alpha) \le \epsilon$, with $\epsilon = 1$: one missing reference ID across the sample, independent of result order or reported distances. The finite-grid objective is

$$\widehat{\alpha} = \min\{\alpha \in A : D_S(\alpha) \le \epsilon\}. \tag{6.5}$$

The reference supplies a nonempty fallback. The objective minimises an accepted multiplier; section 6.3 states when bisection solves it exactly.

At $k = 10$, the normalised allowance $\epsilon/(Sk)$ is 0.003125, 0.0015625 and 0.00078125 for $S = 32, 64, 128$, and 0.0002 at $S = 500$. Larger S changes coverage and strictness. Disjoint held-out queries assess draws from one finite calibration pool (section 7.5).

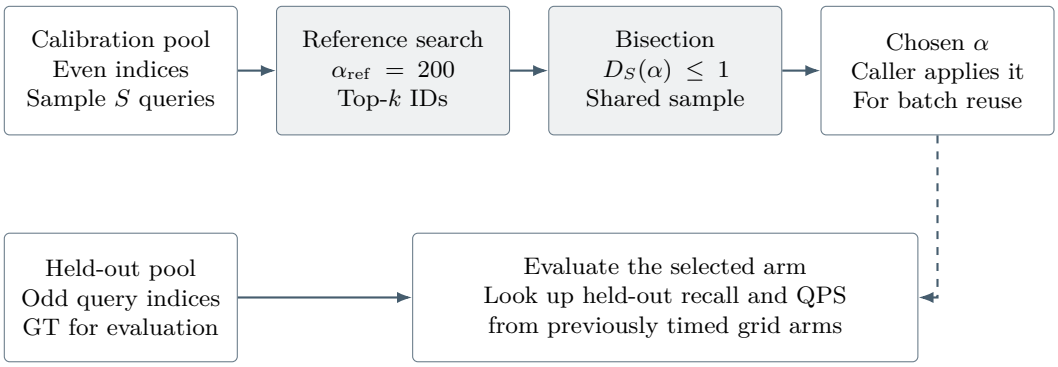


Figure 6.2. Workload-level calibration and its evaluation. Budget decisions use reference IDs; GT assesses held-out quality. Selected-arm results are looked up from timed fixed arms. Implementation boundaries are in section A.3.

6.2.1 What agreement controls

Under Proposition 6.1, disagreement counts reference answers excluded by primary screening:

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_{\text{ref}}(q) \setminus C_\alpha(q)|. \tag{6.6}$$

For valid k-element outputs, each missing reference ID costs at most one GT hit. On the same sample,

$$\begin{aligned} r_{\mathcal{Q}_S}(\alpha) &\ge r_{\mathcal{Q}_S}(\text{ref}) - \frac{D_S(\alpha)}{Sk}, \\ D_S(\alpha) \le \epsilon \implies r_{\mathcal{Q}_S}(\alpha) &\ge r_{\mathcal{Q}_S}(\text{ref}) - \frac{\epsilon}{Sk}. \end{aligned} \tag{6.7}$$

Scope. This is a sample-local bound relative to the reference. GT evaluation measures the reference's remaining error (section 7.8.2).

6.2.2 Sampling resolution and ordered testing

Let H_S count queries with any missing reference ID. Since $H_S \leq D_S$, acceptance implies $H_S \leq 1$. For a prespecified budget and i.i.d. queries, the one-sided 95% exact binomial upper limit after one event solves [22]

$$(1-u)^S + Su(1-u)^{S-1} = 0.05. \tag{6.8}$$

The query-event limits are 0.03652 at $S = 128$ and 0.009452 at 500. Treating all Sk result indicators as homogeneous independent Bernoulli trials is stronger: the corresponding limits are 0.003701 and 0.0009484, approximately $4.74/(Sk)$. Query i.i.d. alone does not justify dividing the query-event limit by k; neither model certifies repeated subsamples of this fixed pool.

Monotone disagreement makes accepted indices a prefix. Valid per-arm tests can then use fixed-sequence error control [3]. Our one-slot threshold tests stability; it does not certify GT risk.

A finite-pool extension. Assume each query has fixed $X_q(\alpha)$ at the intended execution regime, independent of subset membership/position. For pool size N_{cal} and counts $n_j = |\{q : X_q = j\}|$, uniform sampling without replacement accepts only all-zero samples or one single-count query [23]:

$$\Pr\{D_S(\alpha) \leq 1\} = \frac{\binom{n_0}{S} + n_1\binom{n_0}{S-1}}{\binom{N_{\text{cal}}}{S}}. \tag{6.9}$$

Out-of-range binomial coefficients are zero. Predicting GT risk also needs the grid acceptance structure and held-out fixed-arm quality. Recorded subset sums test count invariance (section 7.5.5).

6.3 Budget Search, Nestedness and Tie Safety

6.3.1 Bisection and complete enumeration

Write $J = |A|$. The implementation computes the sample reference once, then searches for the largest acceptable index in the descending grid:

1. Set $l = 0$, $h = J - 1$. Index zero is the reference fallback.
2. While $l < h$, test $j = \lfloor (l + h + 1)/2 \rfloor$ by running $A[j]$ and computing $D_S(A[j])$.
3. If $D_S(A[j]) \leq \epsilon$, set $l = j$; otherwise set $h = j - 1$.
4. Return $A[l]$. A caller applies it before processing reuse batches.

The eight-point grid needs at most three complete candidate searches in addition to the reference. Invocation, host-comparison complexity and timing boundaries are specified in section A.3.

Enumeration reruns the reference and all seven budgets on the same sample. Both timers include a fresh reference. Enumeration handles acceptance holes; bisection matches its minimum only when accepted indices form a prefix.

6.3.2 A sufficient condition for monotonicity

Proposition 6.1 (Nested survivors preserve reference membership)**.**
For one query, assume fixed routing and survivor sets $C_\alpha \subseteq C_\beta \subseteq C_{\alpha_{\text{ref}}}$ whenever $\alpha \le \beta \le \alpha_{\text{ref}}$. Assume at least k valid survivors, fixed refined scores and one deterministic total order for final ties. Each result is the exact top-k in that order. Then

$$|\mathcal{R}_\alpha \cap \mathcal{R}_{\text{ref}}| = |C_\alpha \cap \mathcal{R}_{\text{ref}}|, \qquad D_S(\alpha) \ge D_S(\beta). \tag{6.10}$$

The latter statement holds when these conditions hold for every sampled query.

Proof.
For $x \in \mathcal{R}_{\text{ref}} \cap C_\alpha$, the common total order gives

$$|\{y \in C_\alpha : y \prec x\}| \le |\{y \in C_{\alpha_{\text{ref}}} : y \prec x\}| < k.$$

Hence $x \in \mathcal{R}_\alpha$. Together with $\mathcal{R}_\alpha \subseteq C_\alpha$, this proves the intersection identity in equation (6.10). Nestedness gives $|C_\alpha \cap \mathcal{R}_{\text{ref}}| \le |C_\beta \cap \mathcal{R}_{\text{ref}}|$. Summing missing reference counts over queries proves $D_S(\alpha) \ge D_S(\beta)$. □

Fixed-key exact primary top-c gives nesting; fixed-key refinement then gives monotone agreement. This supports bisection, without making GT recall or runtime monotone as the pool grows.

6.3.3 A corollary for the evaluated grouped path

Within a draw, S, routing and scan threads are fixed; `scheduled_groups(S)` ignores c. Grouped budgets use the same LUT sums. Fixed B_r, 16 refinement lanes and final $(D_{\text{H}}, \text{ID})$ order preserve each candidate's scoring.
Corollary 6.2 (Grouped calibration)**.**
For the evaluated grouped configuration, under the representation and shared-bound contracts of Chapters 4 and 5, primary survivors are nested across budgets. Consequently Proposition 6.1 applies and bisection returns the same minimum accepted grid budget as enumeration.

Proof.
Lemma 5.1 and propositions 5.1 and 5.2 give exact primary top-c under the common key $(D_{\text{P}}, \text{position})$. Its prefixes nest as c grows. The fixed per-candidate refinement and exact final total order supply the remaining premises of Proposition 6.1. □

The corollary covers 2,176/2,304 draws. The other 128 are OpenAI-3072, $S = 128$, distance-only. All recorded curves are monotone and choices match enumeration (section 7.5.5); that check remains finite.

Full-pool coverage. Two FULL cells group Vogue; ten use $g_{\text{scan}} = 0$. All twelve choices match enumeration. The ten distance-only cells remain outside corollary 6.2. A common cutoff order, or no tie crossing any grid/reference cutoff, suffices for general nesting. Matching held-out execution does not prove that condition.

Case	C_1	C_2	C_3	Reference overlap
Nested	$\{b\}$	$\{b, c\}$	$\{a, b, c\}$	0, 0, 1
Non-nested	$\{a\}$	$\{b, c\}$	$\{a, b, c\}$	1, 0, 1

Figure 6.3. Constructed $k = 1$ example with refined order $a \prec b \prec c$ and reference $\{a\}$. The nested case preserves reference membership. With all primary scores tied, different tie choices can produce the non-nested middle pool and break monotonicity. These are explanatory sets, not measured GPU outputs.

A rank-based extension. With the same nesting/fixed-score conditions and one-based primary rank,

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \sum_{x \in \mathcal{R}_{\text{ref}}(q)} \mathbf{1}\{\text{rank}_{\text{P}}(x) > c(\alpha)\}. \tag{6.11}$$

Exported reference ranks could recover the whole curve from one search. The current harness instead reruns every budget; rank extraction remains future work.

6.4 Budget Baselines and Their Questions

Table 6.1 separates budget selection from its evaluation. Fixed arms share the index, route and held-out queries; invocation and timing details are in section A.3.

Let $\widehat{r}, \widehat{Q}$ denote held-out recall/QPS and $\widehat{r}^* = \max_{\alpha \in A} \widehat{r}(\alpha)$. The diagnostic oracle is

$$\alpha_{\text{oracle}} \in \arg\max_{\alpha \in A} \left\{ \widehat{Q}(\alpha) : \widehat{r}(\alpha) \geq \widehat{r}^* - \delta \right\}, \qquad \delta = 10^{-3}. \tag{6.12}$$

It uses labels and timings. Its recall slack δ differs from agreement's slot allowance ϵ.

FULL remains label-free bisection and tightens $\epsilon/(Sk)$. Same-sample enumeration checks the search procedure. Agreement minimises an accepted multiplier; the oracle maximises quality-constrained speed, so their choices can differ.

Fixed 64 tests a smaller default; fixed 200 tests a high budget (section 7.5 and table 7.8).

6.5 Hold-out Risk, Calibration Cost and Payback

6.5.1 Choosing a calibration scale

FULL's held-out quality appears in table 7.7; sampled risks appear in figure 7.6. Under the strong independent-slot model, $4.74/(Sk) \approx 10^{-3}$ suggests $S \approx 500$ at $k = 10$. This concerns reference disagreement, not a GT guarantee (section 6.2).

The path and common-order conditions for FULL are addressed in section 6.3.

6.5.2 Risk and selected-arm performance

Over m calibration draws, let draw r select $\widehat{\alpha}_r$. Define held-out loss and threshold-crossing frequency by

$$L_r = \widehat{r}^* - \widehat{r}(\widehat{\alpha}_r), \qquad \widehat{p}_\delta = \frac{1}{m} \sum_{r=1}^{m} \mathbf{1}\{L_r > \delta\}. \tag{6.13}$$

Mean loss, crossing frequency and tail size answer different questions. Table 7.8 and figure 7.6 report the archived summaries; the largest supplementary per-draw losses are reported in section 7.5.5.

Table 6.1. Budget controls and the questions they answer. Held-out GT is used to assess every result, but only the diagnostic oracle uses it to choose a budget. The reference is fixed at 200 for agreement-based arms.

Control	Selection information	Question
Fixed 100	No calibration	Gain relative to the inherited default?
Fixed 64 and smaller grid arms	No calibration	Does a simple fixed choice remove the need for calibration?
Sampled bisection	S unlabelled queries; slot agreement	How well does a low-cost sample choose a reusable budget?
Same-sample enumeration	Same S, reference and threshold; all grid arms	Did bisection miss the smallest accepted grid budget?
Full-pool calibration	All 500 calibration queries; same bisection rule	What changes with broader coverage and a stricter normalised allowance?
Held-out oracle	GT recall and timed QPS for all arms	Which measured arm is fastest under the diagnostic quality constraint?

Reported selected-arm rate is $\overline{Q} = m^{-1} \sum_r \widehat{Q}(\widehat{\alpha}_r)$, a lookup from timed fixed arms. It is distinct from the median budget's rate and from the throughput of a calibrated query stream. Quality and speed must be read together (table 7.9).

6.5.3 Cost of reuse

For B_{reuse} equal-sized subsequent batches, model

$$T_{\text{policy}}(B_{\text{reuse}}) = T_{\text{cal}} + B_{\text{reuse}} T_{\text{chosen}}, \qquad T_{\text{fixed}}(B_{\text{reuse}}) = B_{\text{reuse}} T_f. \tag{6.14}$$

When $T_{\text{chosen}} < T_f$, the break-even count is

$$B^*_{\text{reuse}} = \frac{T_{\text{cal}}}{T_f - T_{\text{chosen}}}. \tag{6.15}$$

Positive saving requires $T_{\text{chosen}} < T_f$ and $B_{\text{reuse}} > B^*_{\text{reuse}}$. Batch time is $T = 1000n/Q$ milliseconds. When choices vary, $1/\mathbb{E}[Q] \leq \mathbb{E}[1/Q]$, so inverse mean QPS understates mean time. Timing boundaries are in section A.3; section 7.5.6 tests direct calibration-plus-reuse cost on four cells with quality alongside time. Campaign estimates are in table 7.9.

Recalibration every H batches adds T_{cal}/H per batch. Drift detection and automatic recalibration remain future work.

6.6 Policy Failure Modes and Applicability

Reuse assumes a stable index, k, routing and queries. Ordering and batch-path conditions follow section 6.3; quality, tails and total cost follow section 6.5. Drift detection is unimplemented.

The caller must explicitly install α before reuse.

Budget tests and their trace rerun use v69, including OpenAI TF32. FP32 frontier sensitivity does not validate or recalibrate those budget decisions.

7 Experimental Evaluation

The experiments address construction and capacity (RQ1), execution (RQ2), budget quality and reuse cost (RQ3), and competitive search regions (RQ4).

7.1 Experimental Protocol

7.1.1 Comparison protocol and reproducibility

Protocol follows sections 3.2, 3.3 and 3.5. Construction/calibration are separate from host-to-host search; each ablation keeps its own baseline.

CPU JHQ records use unmatched configurations or legacy recall protocols. For example, October Vogue uses 1,024 lists and $[8, 4]$ bits, versus E1's 4,096 lists. They cannot supply a matched CPU/GPU speedup.

Adaptive call counts and auxiliary device-only timing are documented in section A.2.

Metrics and frontier estimators follow sections 3.2 and 3.4; g_{LUT} and g_{scan} retain the distinct meanings in section 5.1.

7.2 Modern GPU ANN Recall–QPS Frontiers

7.2.1 Comparison scope and frontier construction

Figure 7.1 compares JHQ-GPU with the graph, compressed-IVF and brute-force baselines of section 3.4 [17, 28, 33], using the first four datasets in table 3.2. Each uses one batch of 1,000 queries with common GT. T5 separately evaluates small batches in section 7.2.5; section 7.6 reports internal ablations.

The recorded schedule uses $g_{\mathrm{scan}} = 0$ for all 105 arXiv/OpenAI run–budget cells and $g_{\mathrm{scan}} = 4$ for all 27 Vogue cells, triggered by Vogue's candidate-count CV of 0.338–0.622. Thus the non-Vogue frontiers exercise query-centric scanning, while Vogue also exercises grouped retention, shared bounds and merging (section 5.5).

Applying the frontier rule in section 3.4, JHQ contributes 27–39 points per dataset and each baseline family 54–264. E1 contains 1,590 valid points, no failure records and no invalid-ID or duplicate-row flags. These are parameter-sweep points, rather than independent index builds.

The tuning dimensions are listed in table 3.3, with complete coverage in section B.1. A curve ending before a target records missing coverage of that finite grid.

Storage is unmatched: nine-bit RaBitQ has JHQ's nominal $9d/8$ payload; 4/8-bit variants use less. Table 4.1 and section 7.7 give representation costs and measured allocations alongside this performance envelope.

7.2.2 Matched recall and opposing operating regions

Table 7.1 reports recall 0.97 using the interpolated and qualifying-measured estimators defined in section 3.4.

At batch 1,000 and recall 0.97, Vogue's JHQ/RaBitQ ratio falls from 1.059 to 0.933 without grouping. The index, probes, budget and final outputs stay fixed. This identifies an integrated-path benefit, not a shared-bound-only effect.

Table 7.1. Recall 0.97 comparison. Each ratio is interpolated / measured; values above one favour JHQ. JHQ kQPS is the interpolated estimate. The small arXiv–RaBitQ difference changes direction with the estimator.

Dataset	JHQ kQPS	vs CAGRA fp32	vs IVF-RaBitQ
Vogue	230.8	1.416 / 1.537	1.059 / 1.046
arXiv	258.6	1.317 / 1.422	0.996 / 1.015
OpenAI-1536	126.7	1.747 / 2.294	1.130 / 1.111
OpenAI-3072	55.9	1.747 / 2.088	1.157 / 1.346

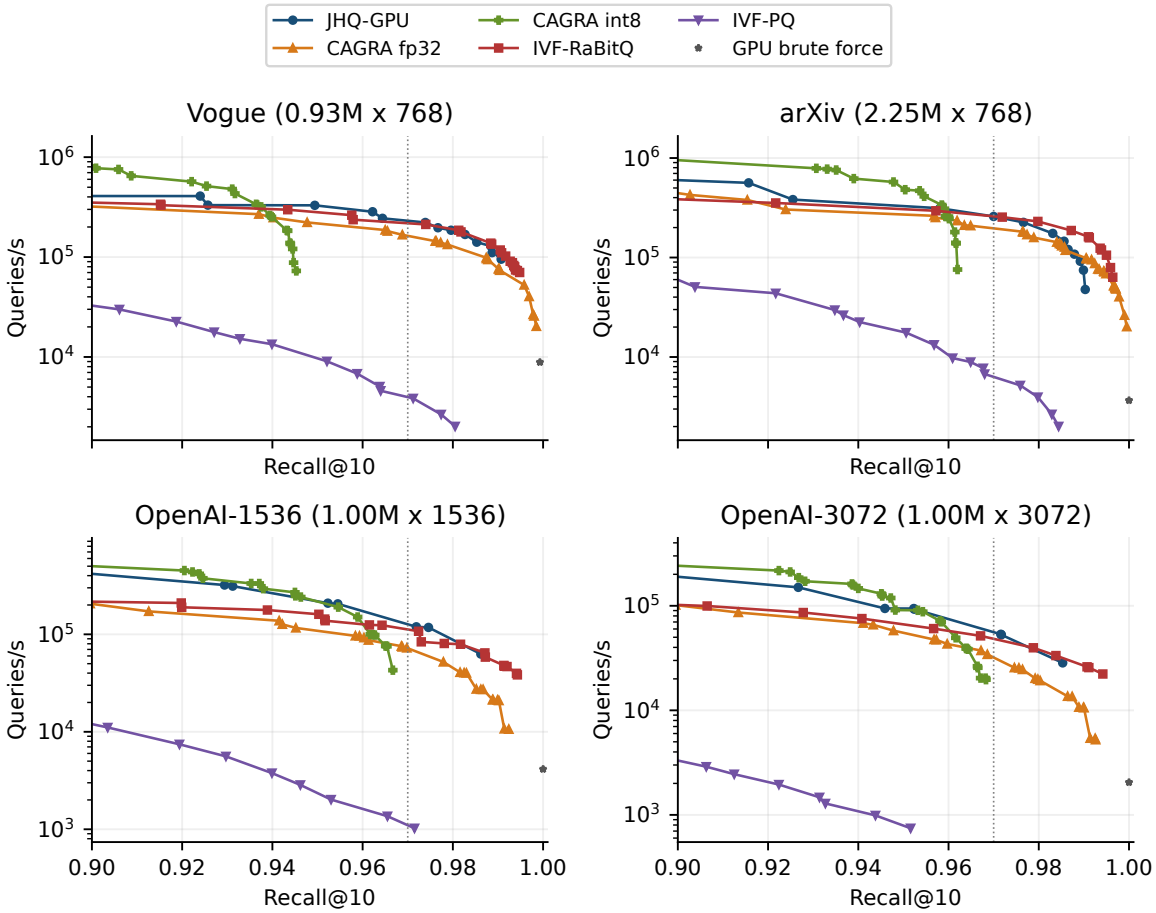


Figure 7.1. Measured host-to-host recall–QPS frontiers on the four E1 datasets. The displayed recall window begins at 0.90, with a vertical line at 0.97; all baseline families, including CAGRA int8, participate in frontier construction. Lines connect measured points; throughput uses a logarithmic axis.

CAGRA int8 is 1.20–1.98× faster at recall 0.90–0.92, but never reaches 0.97 in these grids. RaBitQ regains some 0.98+ targets; CAGRA fp32 wins near 0.99 on arXiv (figure 7.1).

7.2.3 Precision sensitivity on the OpenAI datasets

Supplementary source inspection establishes that the E1 implementation uses TF32 multiplication with FP32 accumulation for OpenAI rotation and centroid GEMMs, despite FP32 input/storage and a misleading source comment (section 4.2). The main E1 frontier remains the historical measurement of that implementation. In campaign D, a separate FP32 variant changes the cuBLAS math-mode switch and replays 37/39 OpenAI configurations at batch 1,000, five rounds each. Execution order alternates between variants across probe settings. Both reuse the same TF32-trained model; the intervention changes add/query arithmetic, including database encoding and routing. It is not a comparison of independently FP32-trained models. Table 7.2 summarises the paired changes.

Table 7.2. FP32 sensitivity. QPS ratio is FP32/TF32 at equal configuration (geometric mean); recall differences have the same direction. The last two columns compare the FP32 frontier at recall 0.97 with archived E1 IVF-RaBitQ and CAGRA fp32 frontiers. Campaign D does not remeasure the baselines.

Dataset	Pairs	QPS ratio	ΔRecall	vs RaBitQ	vs CAGRA
OpenAI-1536	37	0.953	+0.0003–+0.0005	1.086	1.679
OpenAI-3072	39	0.951	+0.0000–+0.0003	1.079	1.629

At equal configuration, FP32 reduces throughput by about 5% and increases recall by 0–0.0005. At recall 0.97, FP32 is about 3% slower than same-session TF32, yet 1.086/1.079× archived RaBitQ and 1.679/1.629× archived CAGRA fp32. New-session v69 measurements give 1.118/1.114 against archived E1 RaBitQ, versus historical 1.130/1.157. D's baseline ratios remain cross-session; only v69/v72 are contemporaneous. Figure 7.2 shows the two JHQ frontiers with their archived comparators. A later T1 campaign remeasures baseline anchors in the same session; section B.4.1 reports those separate, grid-sensitive results.

7.2.4 Interpretation and comparison limits

The OpenAI exports are paired widths of Qdrant DBpedia embeddings [31, 32]. After truncation/renormalisation, all 1,000 query rows and 10,299 sampled base rows have cosine at least 0.9980 (median 1.000000); mean GT overlap is 9.046/10. This supports row alignment without original entity IDs. Doubling width keeps the CAGRA ratio at 1.747 and moves RaBitQ's from 1.130 to 1.157. Within this pair, dimension alone does not explain the gain.

Larger gains accompany smaller 128-probe budget anchors: 4–8 on OpenAI, 64 on Vogue, 100 on arXiv (FULL later selects 200). These are workload associations, not causally isolated or matched-frontier parameters.

The interface, memory and compilation differences in section 3.3 remain relevant to small margins. Section 7.8 examines their empirical sensitivity.

7.2.5 Small batches change the competitive region

T5 compares scheduled JHQ, CAGRA fp32/int8 and IVF-RaBitQ at batches 8/32 on the four E1 datasets. Five synchronous host-to-host rounds use 1,000/992 queries, respectively. Its 1,808 points include JHQ's one-block control; IVF-PQ and brute force are outside this campaign. Figure 7.3 and table 7.3 compare the resulting regions.

At recall 0.97 and batch 8, CAGRA fp32 leads on all four datasets under both estimators: interpolated JHQ ratios are 0.641–0.893. At batch 32, these ratios are 0.622–1.008, with OpenAI-3072 close under interpolation but favourable to JHQ under the measured threshold. At recall 0.95, Vogue's batch-8 lead likewise appears only under interpolation. Grid resolution therefore matters to borderline rankings.

Lower-recall comparisons must retain competitors whose fastest frontier point already exceeds the target. At recall 0.90, batch-8/32 ratios are 1.032–1.479/0.866–1.166; strict thresholds give three/one JHQ wins. Omitting qualifying endpoints would overstate these low-recall leads. Scheduling gains 1.7–3.3× over one block at recall 0.95, without guaranteeing a cross-system win.

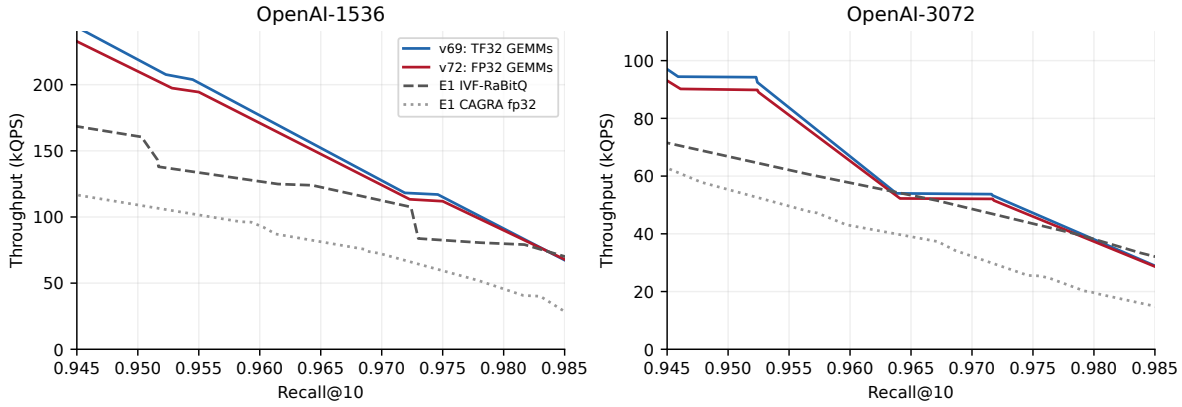


Figure 7.2. Campaign D precision frontiers. Solid curves are same-session JHQ variants; dashed/dotted baselines are historical E1 measurements. All use the chapter's frontier and interpolation rules.

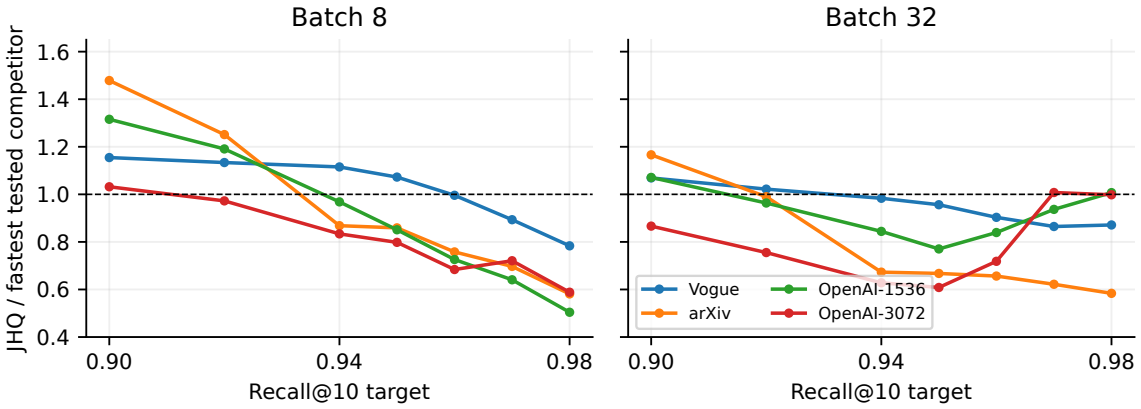


Figure 7.3. T5 JHQ / fastest-competitor QPS under interpolation with qualifying measured endpoints. Above one favours JHQ. Strict measured thresholds are in table 7.3.

Table 7.3. T5 QPS ratios: interpolation with qualifying endpoints / strict measured threshold. The fastest competitor is selected independently for each estimator; higher-recall qualifying endpoints remain eligible without extrapolation.

Dataset	b	$R = 0.90$	$R = 0.95$	$R = 0.97$
Vogue	8	1.155 / 1.091	1.073 / 0.980	0.893 / 0.854
Vogue	32	1.069 / 0.997	0.956 / 0.895	0.865 / 0.839
arXiv	8	1.479 / 1.454	0.859 / 0.851	0.697 / 0.698
arXiv	32	1.166 / 1.148	0.667 / 0.661	0.622 / 0.582
OpenAI-1536	8	1.316 / 1.131	0.851 / 0.825	0.641 / 0.620
OpenAI-1536	32	1.071 / 0.915	0.770 / 0.752	0.937 / 0.914
OpenAI-3072	8	1.032 / 0.953	0.798 / 0.798	0.721 / 0.702
OpenAI-3072	32	0.866 / 0.734	0.608 / 0.771	1.008 / 1.351

Null-call medians are 0.012–0.072 ms and remain included. CAGRA uses Python, JHQ/RaBitQ C++; warm-ups differ (section A.5).

7.3 Cartesian LUT, Packed Loads and Lane Refinement

The RQ2 ablations measure both kernel and application effects.

7.3.1 Exact table factorization and its throughput consequence

The LUT ablation tests the cost model in section 5.2 and FP32 ranking effects.

Figure 7.4 compares Cartesian table grouping on six datasets, probe counts 8/32/128/512, fixed $\alpha = 100$, and $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$. Five rounds use batch 1,000 (Stella 999). Two groups lead in 22/24 cells. Exceptions are near ties: arXiv/128 favours one group by 0.23%; OpenAI-3072/8 favours four by 0.84%. Paired two-versus-one wins are 23/24; all dataset geometric means exceed one (1.021–1.318). Shared-index cells are not independent trials.

The two-group/one-group ratio ranges from 0.998 to 1.914. Four and eight groups each store 16 entries but require more lookups and additions. These arms show that smaller tables need not improve throughput; occupancy and instruction costs were not isolated. The maximum reported recall spread within any four-arm cell is 10^{-4}. At 128 probes and $\alpha = 100$, T4 compares identical candidates on Vogue, arXiv and OpenAI-3072. Full versus $16 + 16$ tables differ in 22.6–26.1% of candidate-score bit patterns, with maximum relative error 4.53×10^{-7}. Across all forms, 20.5–29.4% differ, with maximum relative error 4.64×10^{-7}. Vogue has 2–4/256 final-ID differences, while arXiv/OpenAI have none. Unchanged primary survivor sets can still produce different final IDs because the rounded primary score enters refinement (section A.5).



Figure 7.4. Cartesian table grouping at fixed $\alpha = 100$ (v71). Group counts describe g_{LUT}, not parallel scan groups. Curves are medians of per-dataset QPS ratios to the one-group arm; shading shows their range.

7.3.2 Packed access and cooperative residual scoring

Packing wins 1.012–1.658× across 48 matched pairs (six datasets, four probe counts, $g_{LUT} \in \{1, 2\}$), with recall differences at most 10^{-4}. Both layouts are subspace-major. This tests packing/load/unpack together; one aggregate rate per arm gives less timing evidence than v71's five rounds.

Figure 7.5 and table 7.4 then separate wider loads from lane cooperation in residual refinement. L0 is the original scalar path; L1 changes the load path without distributing a candidate across multiple lanes; L16 assigns 16 lanes to a candidate. The same process rotates arm order across five timing rounds, with three search calls per arm per round. For the twelve ordinary configurations, L16 improves end-to-end throughput by 1.015–1.789×. An additional arXiv configuration at $\alpha = 200$, explicitly a high-budget control, reaches 2.383×; it is not the gain range of the ordinary selected budgets.

The load-only control is bit-identical to L0 in the tested cases, yet already improves arXiv throughput by 1.368×. Lane cooperation adds a larger reduction in refinement time. On arXiv at $\alpha = 100$, the profiled refinement kernel falls from 4,277.5 to 716.8 μs, a 5.97× gain, while total throughput improves 1.789×. OpenAI-3072's refinement kernel improves 8.81×, but total

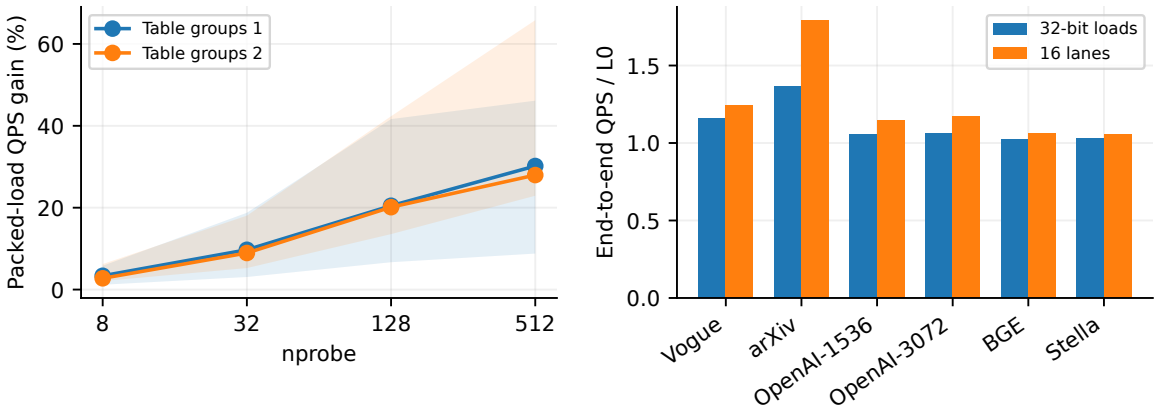


Figure 7.5. Packing and refinement controls. Left: median packed-load QPS gain across six datasets, with shaded minimum–maximum ranges, for each table grouping and probe count. Right: measured load-only and 16-lane end-to-end gains at 128 probes and the ordinary dataset-specific budgets. These panels come from different experiment families.

Table 7.4. Refinement at 128 probes. Ratios are paired end-to-end gains over L0, except the last column, which is the profiled refinement-kernel gain. The kernel and application gains must not be interchanged.

Dataset	α	$L1/L0$ QPS	$L16/L0$ QPS	Kernel gain
arXiv	100	1.368	1.789	5.97
BGE	32	1.026	1.062	1.97
OpenAI-1536	8	1.055	1.145	3.34
OpenAI-3072	4	1.063	1.171	8.81
Stella	16	1.030	1.053	2.66
Vogue	64	1.161	1.240	3.72

throughput improves 1.171×. The remaining routing, screening, selection and transfer costs limit the application gain.

Using refinement's baseline wall-time fraction f and kernel speedup s, Amdahl's estimate is $1/(1-f+f/s)$. For arXiv/OpenAI-3072, $f = 0.529/0.168$ predicts 1.787/1.175×, versus 1.789/1.171× measured. Separate profiling makes these explanatory estimates.

Lane reductions change FP32 association: reported relative distance error reaches about 1.6×10^{-6}, with occasional ranking changes. Sanitizer checks cover small recorded subsets; scoring is not generally bit-identical.

7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation

The representation checks in section 4.6 concern the stored index; this section evaluates query selection and its cost.

7.4.1 Strict CPU reference for computed-score selection

T4 CPU-sorts GPU-produced primary scores under $(D_{\mathrm{P}}, \text{position})$. Its exporter follows scan layout/accumulation and matches returned valid scores bitwise. Source comparison confirms that the diagnostic version changes only export, relative to the table-sweep implementation. The 270 configurations cover Vogue/arXiv/OpenAI-3072, 1/8/128 probes, $\alpha \in \{4, 100, 200\}$, $g_{\mathrm{scan}} \in \{0, 1, 2, 4, 8\}$ and both bound settings: 69,120 query instances (table 7.5).

Table 7.5. T4 CPU-reference checks. Set and order equality use the complete primary key. Valid count checks $\min(c, n_c)$ survivors; final equality sorts retained computed refined scores by $(D_{\mathrm{H}}, \mathrm{ID})$, including their FP32 bits. Counts are query instances, with queries reused across configurations.

Path	Queries	Set / order exact	Tie-only	Valid count	Final exact
$g_{\mathrm{scan}} \geq 1$	55,296	55,296 / 55,296	0	55,296	55,296
$g_{\mathrm{scan}} = 0$	13,824	13,768 / 8,758	56	13,824	13,824

All 55,296 grouped instances match complete-key order. All 13,824 one-block instances satisfy distance order, with 56 cutoff-tie set differences. Underfull valid counts pass in 19,120 cases; all 69,120 final selections match retained-score references. These test lemma 5.1 and propositions 5.1 and 5.2; separate scan/bound and sanitizer checks have their own coverage (section A.5).

7.4.2 Shared bounds preserve the selection target

Proposition 5.2 establishes the selection guarantee. The paired experiment measures the work saved by sharing that bound.

All 48 paired bound comparisons agree on primary/final output; throughput spans 0.994–1.562×. At arXiv $\alpha = 200$, batch 1,000 and eight groups, sharing cuts merge 2,629→471 μs and scan 7,968→5,513 μs. Other cells include slight regressions.

The pilot's first group holds 46–81% of winners. Equal quotas need fallback; unequal quotas leave a dominant group. Shared bounds avoid that fixed-quota choice.

7.4.3 Exact probe selection and the limits of pipeline identity

The radix probe selector keeps prior score arithmetic, including OpenAI TF32 (section 7.2.3). Its 1,792 synthetic cases pass. On all 1,000 arXiv/Vogue/OpenAI-3072 queries, probes and primary positions/distances match the previous selector.

Table 7.6 shows end-to-end improvements of 1.034–1.167×. At batch 8, the profiled probe stage falls from 56.5 to 22.6 μs on arXiv, 52.7 to 14.7 μs on Vogue, and 77.8 to 16.7 μs on OpenAI-3072. The unchanged stages limit full-batch gains. These within-family ratios have different baselines from the preceding ablations.

On Vogue, the probe/primary dump is byte-identical across the two binaries for all 1,000 queries in the one-group, batch-8 test, yet final recall is 0.9642 versus 0.9644. The same baseline binary also changes from 0.9642 in the identity run to 0.9644 in each of its two timing runs. The variation therefore also occurs without the probe change and lies after primary selection in the dump-controlled case. Its root cause remains unresolved. Scheduled and one-block outputs are bit-identical within each checked process; dump coverage is in appendix A.

Finally, the optional v70 FP16 experiment is a separate approximation. Its probe sets match the v69 reference on 936/1,000 arXiv queries, 979/1,000 Vogue queries and 998/1,000 OpenAI-3072 queries, with very high mean set overlap. High overlap and small recall changes do not make the primitive exact. Those results are excluded from the main E1 frontier and from the exact-probe claim.

7.5 Held-out Budget Quality, Throughput and Payback

RQ3 compares the policy of chapter 6 with fixed $\alpha = 100$ and the stronger universal control $\alpha = 64$.

7.5.1 Unlabelled selection and a labelled diagnostic oracle

The six datasets are tested at 128 and 512 probes with the fixed, sampled, FULL, enumeration and oracle controls in section 6.4. The disjoint pools and 64 draws per sample size follow section 3.2.

All rule variants use equation (6.4) with $k = 10$ and $\epsilon = 1$. FULL uses all calibration queries; the diagnostic oracle uses held-out labels (table 6.1).

7.5.2 Full-pool calibration as the quality-first working point

Full-pool calibration uses all 500 calibration queries once per cell. Its selected arms meet the held-out 10^{-3} loss target in 12/12 cells; the largest observed loss is 0.0006. Their rates are 0.908–1.000 of the GT oracle's, and calibration costs 9.5–60.3 ms. The geometric-mean rate ratio to fixed 100 is 1.111×, or 1.193× over the ten cells where fixed 100 also meets the target. Table 7.7 shows the complete comparison. Against fixed 64, full-pool selection is faster in six

Table 7.6. v69/v65 end-to-end throughput ratios for the exact computed-score probe change. Each workload–batch comparison has ten timing rounds per binary, collected in alternating runs. Values above one favour v69.

Dataset	Batch 8	32	256	1,000
arXiv	1.074	1.063	1.035	1.034
Vogue	1.109	1.103	1.052	1.041
OpenAI-3072	1.167	1.117	1.055	1.050

cells, equal in two and slower in four: its main advantage is the observed coverage of all twelve quality targets.

These cells favour FULL for quality coverage and $S = 128$ for lower calibration cost, within the statistical scope of section 6.2.

7.5.3 Lower-cost sampling and its quality risk

Table 7.8 reports the $S = 128$ result. The median selected budget matches the oracle budget in ten of twelve cells; it differs on arXiv at 128 probes and OpenAI-3072 at 512. The selected budgets span 4–200, indicating substantial workload dependence. Median budget and mean recall need not describe one arm. The latter is reconstructed from printed best recall minus mean loss and is approximate at four decimals.

ArXiv has 40/64 and 32/64 loss crossings at 128/512 probes, despite mean loss 0.0009. Fixed 100 loses 0.0012/0.0016: better than default need not meet the target. BGE/512 has 24 crossings and $L_{95} = 0.0014$; Stella reaches 0.0030; OpenAI-3072 has none (table 7.8). Four losses can coexist with reported $L_{95} = 0$ (section A.3).

7.5.4 Estimated throughput and calibration economics

Using the mean selected-arm QPS estimate specified in section A.3, $S = 128$ improves over fixed $\alpha = 100$ in ten cells by 1.028–1.691×. On arXiv the ratios are 0.908 and 0.917: mean recall improves, but throughput falls, giving no throughput payback.

Against fixed 64, sampled-arm QPS wins 9/12 cells: geometric mean 1.054×, range 0.821–1.367× (OpenAI 1.052–1.367×). Both fixed controls fail arXiv quality, so these are not uniformly quality-matched gains.

Calibration takes 3.9–21.3 ms. Under section 6.5's reuse assumptions, equation (6.15) and table 7.9 give maximum positive paybacks of 0.54/2.72 s versus fixed 100/64; FULL gives 1.52/2.25 s. ArXiv controls fail quality. Direct reuse below checks total cost and recall together.

7.5.5 Complete decision traces and finite-pool risk

All 2,304 sampled and twelve FULL curves are monotone and match enumeration. Recorded subset sums reproduce 18,432 grid counts (equation (6.9)). This checks those aggregates, not arbitrary subset/position invariance or the distance-only cases' general nesting (section 6.3).

Maximum sampled loss reaches 0.0050 (arXiv/512) and 0.0046 (BGE/512) at $S = 128$, and 0.0294 at $S = 32$. FULL stays within 0.0006.

ArXiv/128 predicted/observed crossing rates are 0.965/0.984, 0.875/0.875 and 0.620/0.625 for $S = 32, 64, 128$ (figure 7.7). Across 36 comparisons, the largest gap is BGE/512's 0.144

Table 7.7. Full-pool calibration ($S = 500$). Loss is relative to the best measured held-out recall; rates are kQPS from the selected fixed arm. "Cal." is one full-pool calibration call per cell, not a repeated stream timing. Held-out counts are 500, or 499 for Stella.

Dataset	Probe	α_{full}	Loss	Q_{full}	vs 100	vs oracle	Cal. ms
Vogue	128	100	0.0000	202.9	1.000	0.908	10.1
arXiv	128	200	0.0000	151.5	0.728	1.000	10.0
OpenAI-1536	128	8	0.0000	267.7	1.599	1.000	9.5
OpenAI-3072	128	4	0.0000	145.0	1.691	1.000	17.1
BGE	128	64	0.0006	124.9	1.075	0.932	17.1
Stella	128	16	0.0006	110.4	1.153	1.000	20.6
Vogue	512	100	0.0002	100.1	1.000	0.969	20.9
arXiv	512	200	0.0000	69.0	0.831	1.000	25.1
OpenAI-1536	512	8	0.0000	114.9	1.326	1.000	21.8
OpenAI-3072	512	4	0.0002	52.1	1.219	0.977	40.5
BGE	512	64	0.0006	50.8	1.033	1.000	42.3
Stella	512	16	0.0006	36.0	1.041	1.000	60.3

Table 7.8. Held-out quality at $S = 128$. Rule α is the median across 64 draws; oracle α uses held-out labels. $\overline{R}_{\text{rule}}$ is reconstructed mean recall. "Losses" counts draws with recall loss $> 10^{-3}$ relative to the best tested held-out recall; L_{95} is the logged lower-order 95th-percentile loss estimate. R_{100} is fixed-budget recall.

Dataset	Probe	α: rule/oracle	R_{100}	$\overline{R}_{\text{rule}}$	Losses	L_{95}
Vogue	128	64 / 64	0.9680	0.9677	5/64	0.0022
arXiv	128	100 / 200	0.9748	0.9751	40/64	0.0024
OpenAI-1536	128	8 / 8	0.9280	0.9279	4/64	0.0000
OpenAI-3072	128	4 / 4	0.9252	0.9252	0/64	0.0000
BGE	128	32 / 32	0.9198	0.9190	1/64	0.0008
Stella	128	16 / 16	0.9892	0.9884	6/64	0.0030
Vogue	512	64 / 64	0.9916	0.9915	2/64	0.0002
arXiv	512	200 / 200	0.9878	0.9885	32/64	0.0016
OpenAI-1536	512	8 / 8	0.9740	0.9738	4/64	0.0000
OpenAI-3072	512	4 / 8	0.9704	0.9702	0/64	0.0002
BGE	512	64 / 64	0.9670	0.9661	24/64	0.0014
Stella	512	16 / 16	0.9954	0.9945	7/64	0.0030

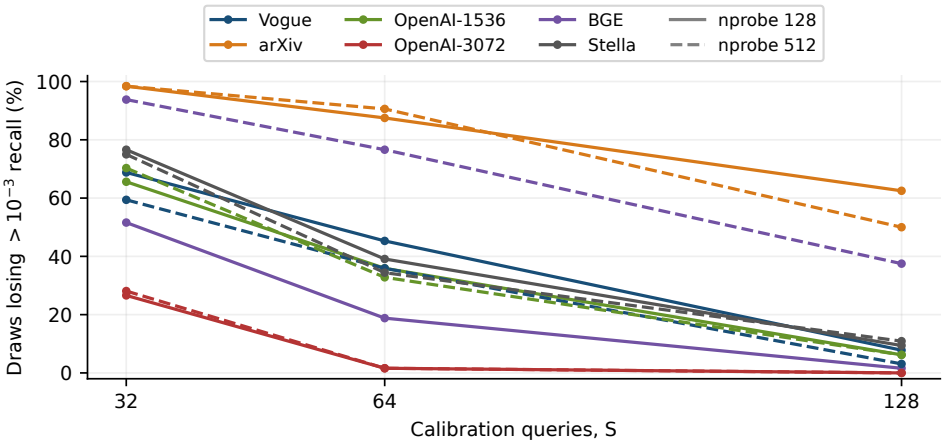


Figure 7.6. Held-out risk across sample sizes. The vertical axis is the percentage of calibration draws losing more than 10^{-3} held-out recall relative to the best tested budget. Increasing S also tightens the allowed slot-disagreement fraction because equation (6.4) keeps threshold one.

(0.519/0.375). Prediction assumes stable counts and uses held-out fixed-arm quality to map choices to GT loss; calibration remains label-free.

Ten alternating repeats support a low-budget timing plateau, without implying generally monotone runtime.

7.5.6 Direct calibration-plus-reuse measurement

Four cells run fixed 64/100, sampled and FULL policies over five rotated rounds of 500 repeated held-out batches. Timing includes gathering, calibration, budget installation and first-batch graph setup; build, warm-up and scan calibration are excluded (table 7.10).

Table 7.9. Calibration economics, $S = 128$. Rates are kQPS and rule rate is mean selected-arm QPS. Ratios compare with fixed 100 and fixed 64. The "Batches" column gives payback relative to fixed 100 using 500 queries, or 499 for Stella; "Never" denotes no throughput payback. The quality column separately marks whether fixed 100 meets the held-out loss target.

Dataset	Probe	Q_{100}	$\overline{Q}_{\text{rule}}$	vs 100	vs 64	Cal. ms	Batches	Fixed 100 quality
Vogue	128	202.9	224.4	1.106	1.004	4.0	17.0	Pass
arXiv	128	207.9	188.8	0.908	0.821	4.2	Never	Fail
OpenAI-1536	128	167.5	268.5	1.603	1.367	3.9	3.5	Pass
OpenAI-3072	128	85.7	145.0	1.691	1.230	5.7	2.4	Pass
BGE	128	116.2	131.2	1.130	1.051	6.5	13.2	Pass
Stella	128	95.7	110.3	1.153	1.087	7.5	10.8	Pass
Vogue	512	100.1	102.9	1.028	0.996	7.5	55.4	Pass
arXiv	512	83.1	76.2	0.917	0.870	9.7	Never	Fail
OpenAI-1536	512	86.6	115.0	1.327	1.268	8.8	6.2	Pass
OpenAI-3072	512	42.8	52.1	1.219	1.052	13.1	6.2	Pass
BGE	512	49.2	51.1	1.039	1.006	15.8	41.2	Pass
Stella	512	34.6	36.0	1.041	1.028	21.3	37.1	Pass

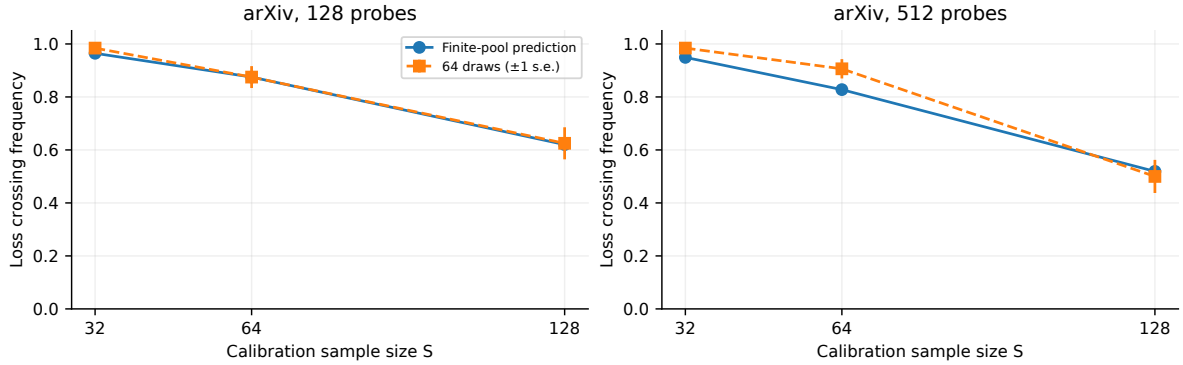


Figure 7.7. Finite-pool prediction and observed arXiv loss crossings. Bars show one estimated standard error over 64 independently drawn subsets conditional on the fixed pool, not uncertainty over future workloads.

Table 7.10. Direct total milliseconds for 500 reuse batches, averaged over five rounds, including budget calibration for sampled and FULL policies. Each batch has 500 queries. Quality is discussed with the timing results.

Dataset/probes	Fixed 100	Fixed 64	Rule $S = 128$	FULL
arXiv/128	1224	1102	1291	1680
OpenAI-1536/128	1491	1264	852	860
OpenAI-3072/128	2757	2160	1769	1783
BGE/512	5185	5018	4921	5062

OpenAI-3072/128 always selects 4: sampling takes 1,769 versus fixed-100's 2,757 ms, both at recall 0.9252. FULL selects the same budget and takes 1,783 ms including its larger calibration cost. OpenAI-1536 FULL gives 860 versus 1,491 ms at recall 0.9280; sampling gives 852 ms at 0.9276.

Direct cumulative savings begin at 4/6 batches (OpenAI-1536) and 4/9 (OpenAI-3072) versus fixed 100/64 (figure 7.8); fractional fits are separate estimates. BGE sampling gives 4,921 versus 5,185 ms but recall 0.9658 versus 0.9670; FULL gives 5,062 ms and 0.9664. ArXiv FULL selects slower 200 for quality; fixed controls fail the target.

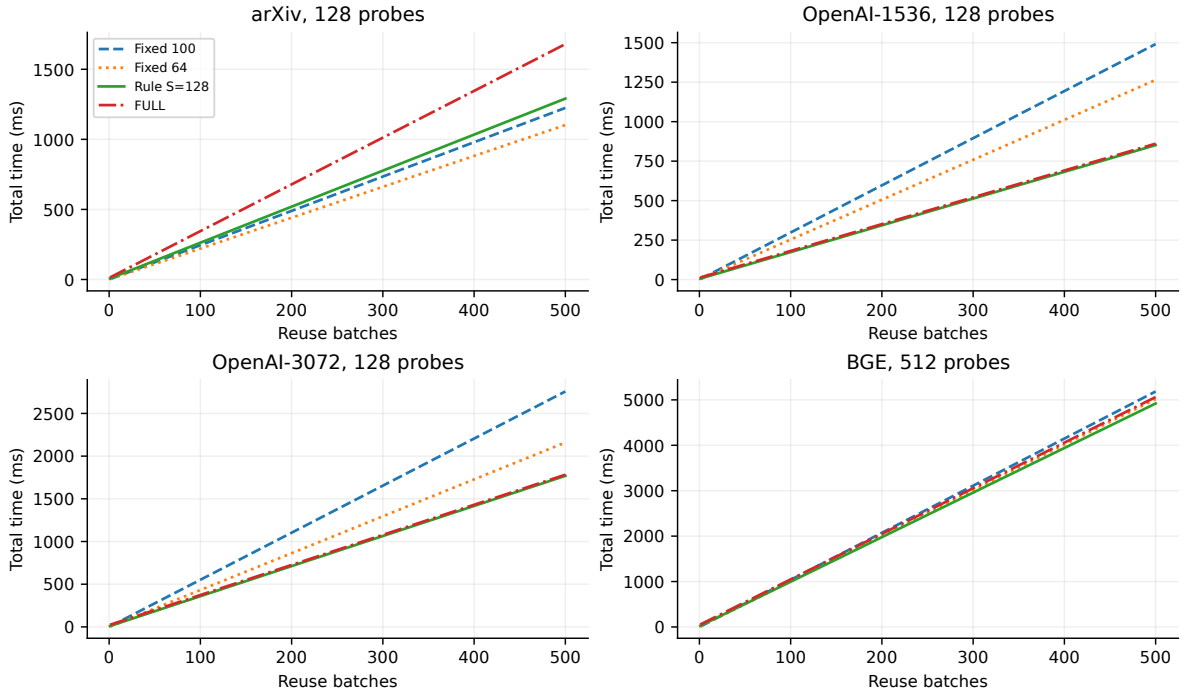


Figure 7.8. Direct mean cumulative time, including budget calibration and first-batch setup. Each curve replays one fixed held-out query pool. Policies can have different recall; timings alone are not a quality-matched ranking. Dataset/probe settings match table 7.10.

7.6 Batch Regimes, G Scheduling and Workload Skew

This section answers the performance-regime part of RQ2: when additional scan groups supply useful parallelism, and when their extra work dominates.

7.6.1 Parallelism versus repeated selection work

At batch 8, eight groups raise throughput 2.06–2.76×: arXiv scan drops 849→216 μs, with 16.7 μs merging. At batch 1,000 and $\alpha = 200$, independent grouping instead raises scan 3,530→7,652 μs plus 2,624 μs merging, reaching only 0.46× one-block QPS. Shared bounds reduce work, without removing this regime change.

7.6.2 A fixed rule evaluated on further configurations

The scheduler follows the fill/skew rule and pseudoquery calibration in section 5.5. Four configurations set the constants before the additional configurations were measured.

The means use thirteen small-batch configurations and eleven full-batch ones (Stella has 999 queries); held-out subsets use nine/seven (table 7.11). Fixed eight groups fall to 0.886× at batch 1,000. Worst scheduling regret is 1.217 on BGE/batch 256.

Vogue's CV 0.338–0.622 forces four groups at batch 1,000. Scheduled/one-block QPS spans 0.895–1.175; $(n_{\text{probe}}, \alpha) = (16, 16), (16, 64), (32, 64)$ lose at 0.955, 0.895, 0.955. CV captures

Table 7.11. Integrated execution gains over v61 L16. Entries are geometric means of per-cell median paired speedups from five rounds. "Held out" excludes the four configurations used to set the rule; it means held-out configurations, not the budget experiment's held-out query split.

Batch	Cells	$g_s = 2$	$g_s = 4$	$g_s = 8$	Scheduled	Held out
8	13	1.318	2.014	2.728	2.727	2.930
32	13	1.323	1.970	2.337	2.356	2.560
256	13	0.997	1.090	1.099	1.057	1.041
1000	11	0.959	0.947	0.886	1.015	1.012

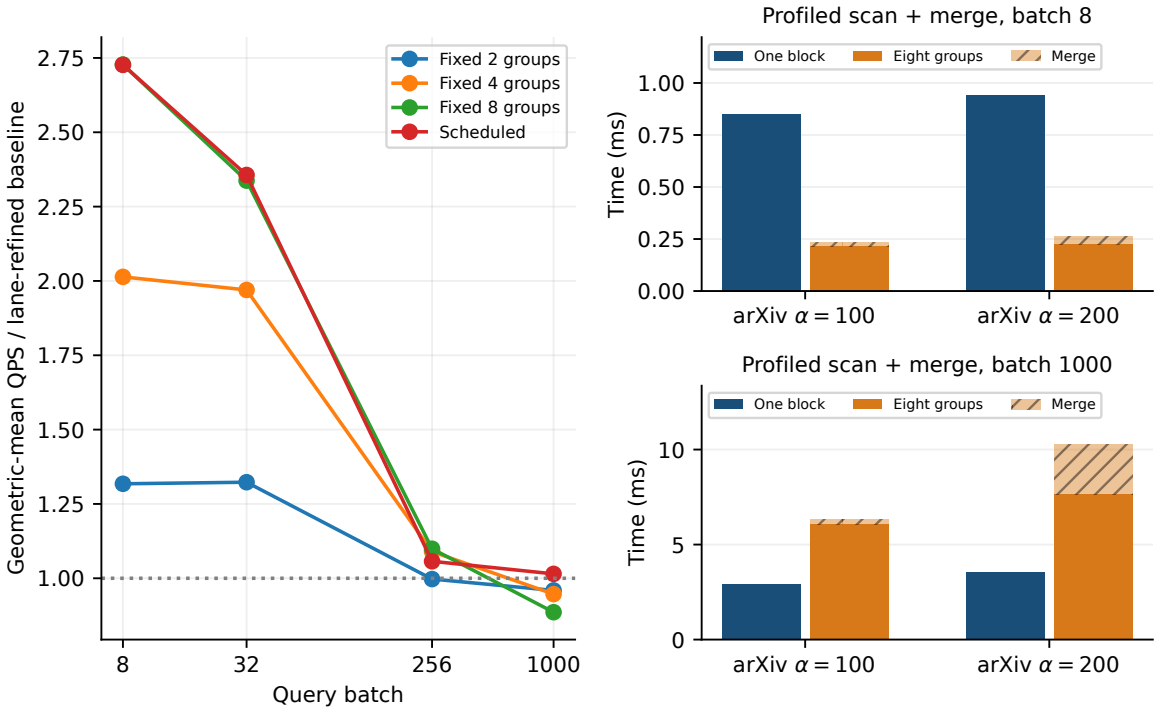


Figure 7.9. Execution regimes. Left: fixed shared-bound group counts and scheduled execution relative to the lane-refined one-block baseline (13 cells at small batches, 11 at batch 1,000). Right: paired scan and merge profiles at batches 8 and 1,000 for arXiv budgets 100 and 200. Hatching denotes merge time. The right panels use the independent-bound scan experiment; they explain a cost, not the scheduler's entire speedup.

dispersion, not absolute work: the same rule enables the 0.97 lead but wastes work at low probes (section 7.2).

Across 250 variant/configuration pairs and 234,320 query comparisons, there are no non-tie primary differences or final-ID-set differences. Tied output order can differ. These are sequential synchronous throughput tests, not open-loop tail-latency tests.

7.7 Cold Construction, Resident Memory and Tested Scale

RQ1 is assessed through cold construction, resident memory and completed large-dataset searches; the E1 memory comparison also informs RQ4.

7.7.1 Cold builds and the lifecycle boundary

Three empty-cache repeats give Vogue training 298–303 ms and total about 0.56 s; its archived 1.33 s run was slow. Stella takes 10.54–10.65 s, near the archived 10.40 s (table 7.12).

Construction and startup boundaries follow sections A.2 and 3.3; figure 7.10 compares these repetitions with baseline configurations.

Stella builds take 74.55/82.45 s for isolated CAGRA int8 (including quantisation) and a three-configuration median 86.59 s for IVF-PQ, versus JHQ's 10.54–10.65 s. Quality and resident memory remain separate comparisons.

Five builds show up to 0.0012 recall range on arXiv. Stella startup is 18.2–18.6 s with warm pages, 129.9 s on first read. At 20 ms resolution, peaks are BGE 26.08 GiB and Stella 22.58 GiB. BGE's 12 GiB assignment buffers precede its 23.05 GiB gather payload; summing them would overstate simultaneous memory (equation (4.13)).

Table 7.12. Cold JHQ construction and separately measured resident memory. Train/add are component medians of three supplementary normal builds; Total is the median paired sum. Memory comes from v71's two-group, 128-probe, $\alpha = 100$ runs and includes context and workspace. "Residual" is the modelled residual-code share of the measured used total, not the fraction read per query.

Dataset	Train s	Add s	Total s	Used GiB	Raw GiB	Residual
Vogue	0.30	0.26	0.56	1.47	2.67	45.4%
arXiv	0.48	0.51	0.99	2.58	6.45	62.5%
OpenAI-1536	0.61	0.38	1.00	2.39	5.72	59.7%
OpenAI-3072	1.16	0.67	1.82	4.14	11.43	69.0%
BGE	3.56	3.65	7.20	11.93	38.50	80.7%
Stella	3.55	6.99	10.55	20.44	67.81	82.9%

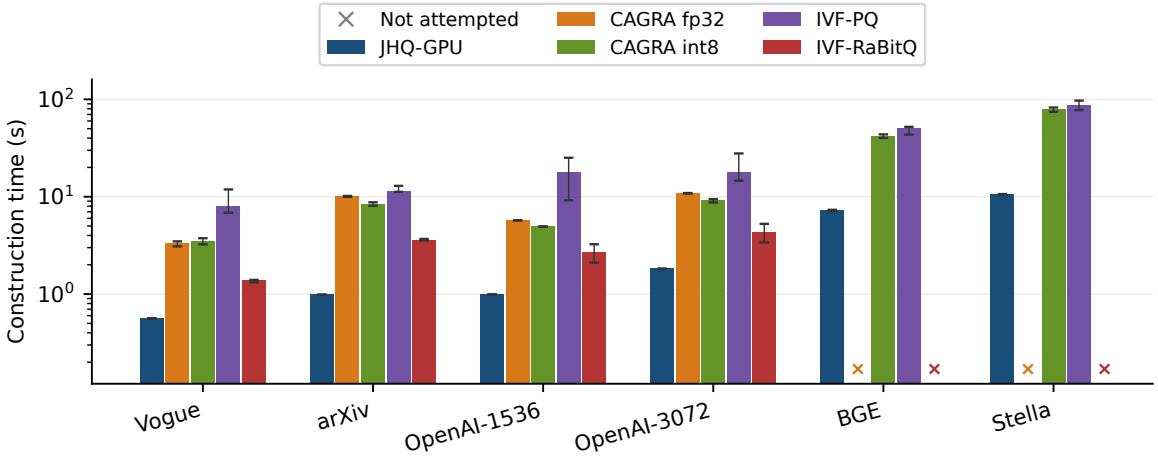


Figure 7.10. Recorded construction costs, with CAGRA int8 included. JHQ is the median and range of three supplementary normal cold builds with warm input pages; other bars are medians across successful measured index configurations, with minimum–maximum whiskers. Int8 includes dataset quantization; RaBitQ excludes the required serialization round trip. Missing large-dataset bars denote unattempted configurations, not measured build failures. The vertical axis is logarithmic.

7.7.2 Resident allocation and where compression helps

Figure 7.11 combines the component model in section 4.4 with measured resident totals. Its unattributed remainder is retained, and the preceding construction measurements give the sampled peaks.

Residuals occupy 80.7% of BGE's 11.93 GiB and 82.9% of Stella's 20.44 GiB totals, including context/workspace. Raw vectors need 38.50/67.81 GiB. Smaller α saves reads, not stored residual bytes (section 4.4).

The E1 measurements provide a separate cross-system view in table 7.13. Subtracting the baseline-measured 507.2 MiB context from JHQ's process total gives approximately 0.29–0.39 of CAGRA fp32's index allocation. JHQ's figure includes workspace, whereas CAGRA's is the build-time index delta. These are parameter-sweep ranges, not memory-matched recall points or a comparison of equal peak budgets. The CAGRA int8 allocations in the same table are of a similar order to JHQ's: compression relative to fp32 is shared by competing designs. The component model in table 4.1 explains this comparison.

7.7.3 Large-dataset feasibility and competitive regions

BGE contains 10,091,524 vectors and Stella 17,776,615, both with 1,024 dimensions and 32,768 IVF lists. Stella's queries are sampled from the corpus; its result therefore characterizes

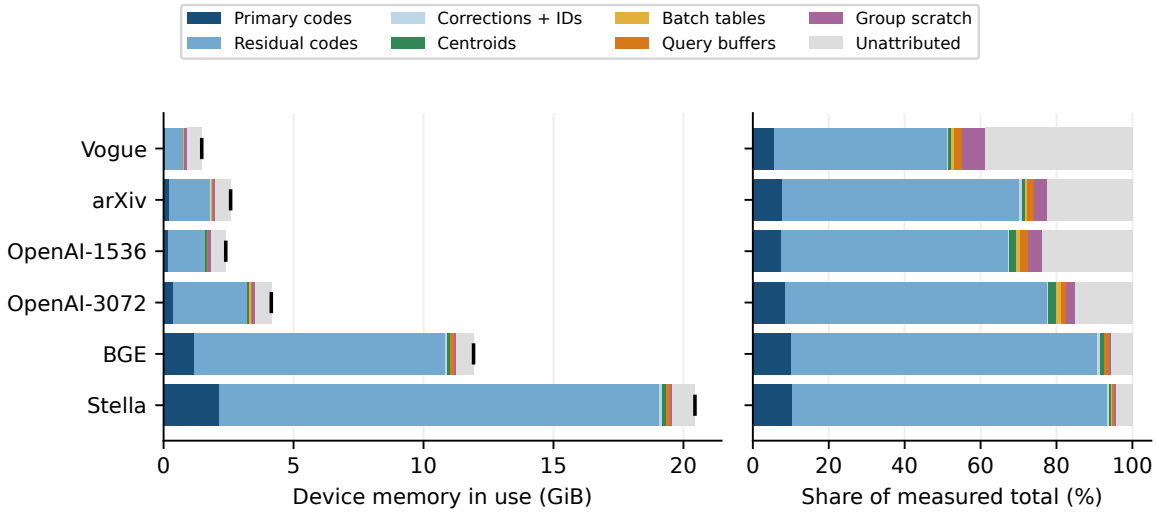


Figure 7.11. Resident-memory decomposition at two-group tables, 128 probes and $\alpha = 100$. Component sizes are modelled, markers are measured used totals, and hatching is the unattributed remainder. The totals include context and workspace. The preceding construction measurements give sampled peaks; these resident markers are not peaks.

Table 7.13. E1 allocation ranges. JHQ is process total minus a 507.2 MiB context estimate, including query workspace; both CAGRA columns are index allocations measured during build. Ranges cover the parameter sweep.

Dataset	JHQ net process GiB	CAGRA fp32 index GiB	CAGRA int8 index GiB
Vogue	0.90–1.01	2.89–2.91	0.89–0.91
arXiv	2.01–2.19	6.84–6.98	2.01–2.15
OpenAI-1536	1.79–2.33	5.95–5.97	1.67–1.68
OpenAI-3072	3.54–3.82	11.67–11.68	3.10–3.11

that benchmark rather than an independently collected external-query distribution. Figure 7.12 includes successful CAGRA int8 and IVF-PQ measurements. Full-precision CAGRA, RaBitQ configurations and FP32 brute force were not attempted in the large-scale comparison when their computed storage requirements exceeded the available device capacity. Those omissions are planned feasibility exclusions, not observed out-of-memory errors or measured recall failures.

CAGRA int8 fits Stella. On BGE it is 1.35–2.96× faster at recall 0.88–0.95; JHQ is 1.086× faster at int8's maximum 0.9579. Maximum BGE/Stella recalls are int8 0.9579/0.9783, JHQ 0.9663/0.9952 and PQ 0.9525/0.9896.

On Stella, JHQ gives 185.4 kQPS at recall 0.978679; int8 gives 66.3 kQPS at 0.978278. That 2.80× measured-point ratio uses different recalls. Int8 has no coverage for much of JHQ's upper region (appendix A).

7.8 Numerical Robustness, Anomalies and Recall Attribution

The robustness checks qualify the RQ2 exactness claims, the RQ3 risk assessment and the small cross-system margins in RQ4.

7.8.1 Quality resolution and repeated execution

One GT hit changes Recall@10 by 10^{-4} over 1,000 queries or 2×10^{-4} over 500 (slightly more over Stella's 499). This resolution is not an uncertainty interval.

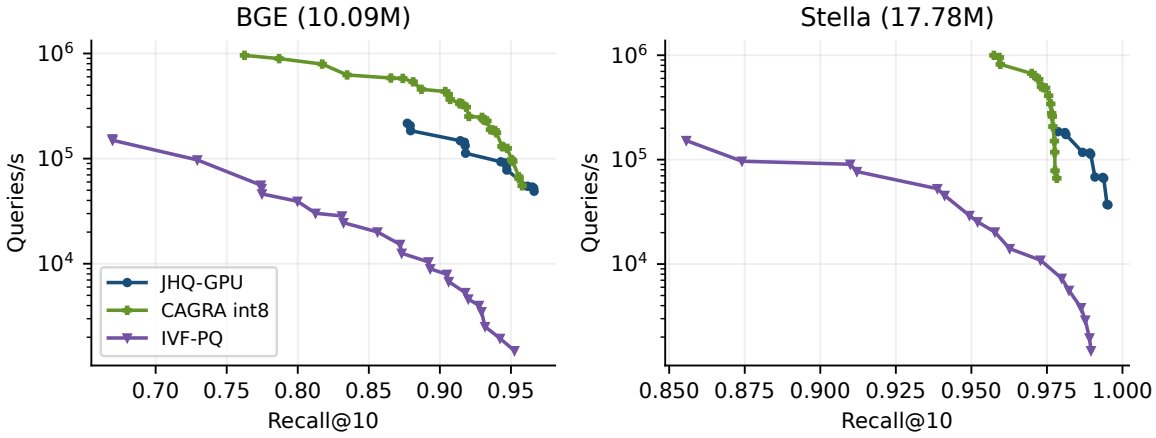


Figure 7.12. Large-dataset host-to-host frontiers, including the successful isolated CAGRA int8 reruns. BGE uses 1,000 queries and Stella 999. An endpoint is the end of the tested grid, not an intrinsic recall limit.

Repeated calls and repeated index builds answer different questions. E1 retains 369 baseline repeat-call identifier flags, including tie-order changes; these points remain in the frontiers. The 401-count convention and cross-protocol checks are separated in section A.2.

Near 0.97, local frontier slopes imply 1.01–3.17% QPS change per 10^{-3} recall before snapping. Five-build recall ranges are Vogue 0.0008, arXiv 0.0012, OpenAI-1536 0.0004 and OpenAI-3072 0.0002. They suggest few-percent local sensitivity, not a bound from retraining whole frontiers/baselines. FP32 sensitivity retains the OpenAI advantage within its cached-model scope (section 7.2.3); arXiv and LUT near ties remain inconclusive.

RaBitQ's median five-round ranges are 3.79%, 3.34%, 2.87% and 5.25% of median QPS on Vogue, arXiv and the two OpenAI datasets. These are descriptive spreads, not ratio confidence intervals. Traces include occasional 30–60 ms calls. As a sensitivity check, recomputing QPS as $10^6/\,\text{median(call latency in ms)}$ for a 1,000-query call reduces the influence of long calls. At recall 0.97, the resulting JHQ/RaBitQ ratios are 1.063, 0.995, 1.118 and 1.100. The three favourable directions and arXiv near parity remain, so the result is not solely a consequence of long calls.

7.8.2 Exactness is attached to a particular stage

The stage contracts in table 5.1 distinguish computed-score selection from raw-space kNN; lane rounding, cutoff ties and process variation qualify different stages.

The archived reconstruction pilot measures the remaining gap to full-probe reconstructed scoring. Across seven workload–budget arms of 128 queries each, scoring all probed candidates in FP64 improves recall over GPU outputs at selected budgets by 0–0.002344. For arXiv at 128 probes, $\alpha = 200$ matches the full-IVF reconstructed answer set on all 128 queries. At 100, the recall is 0.9734375 versus 0.9750 for full-IVF scoring: an additional 0.0015625 budget deficit beside the joint routing/reconstruction deficit of 0.0250 (section 6.2). These are pilot-specific aggregate differences; the seven arms include two budgets on one dataset–probe workload. FP64 reconstructed scoring also differs from the GPU arithmetic, so the pilot does not isolate a pure budget effect in every arm. Separating routing from representation requires additional stage-wise GT coverage.

8 Discussion and Future Work

8.1 Three Cross-Layer Insights

8.1.1 A useful representation exposes a useful unit of reuse

Mailman loses despite more reuse; two-group LUTs usually win (figure 7.4). Reuse must fit table placement and access (sections 5.2 and 5.3), so codebook quality and attainable scorer cost need joint evaluation.

8.1.2 Useful parallelism is bounded by duplicated state and selection

Once query parallelism is sufficient, staging/selection/merge overhead can dominate (figure 7.9). Vogue and BGE expose missing absolute-work and decomposition features (sections 7.2 and 7.6). A richer scheduler must also repay calibration; integrated results do not isolate shared bounds.

8.1.3 Execution speed changes the value of avoiding refinement

Consider a latency model with fixed non-refinement cost $T_0 > 0$, high-/low-budget refinement times $F_h \geq F_\ell > 0$, and a common refinement-time multiplier $0 < \eta \leq 1$. The low-budget speedup and its derivatives are

$$S(\eta, T_0) = \frac{T_0 + \eta F_h}{T_0 + \eta F_\ell}, \tag{8.1}$$
$$\frac{\partial S}{\partial \eta} = \frac{T_0(F_h - F_\ell)}{(T_0 + \eta F_\ell)^2} \geq 0, \qquad \frac{\partial S}{\partial T_0} = \frac{\eta(F_\ell - F_h)}{(T_0 + \eta F_\ell)^2} \leq 0.$$

For $F_h > F_\ell$, faster refinement reduces the value of cutting its budget; lower fixed cost increases it. The derivatives hold with non-refinement work fixed. Budget-dependent scan maintenance and resource transitions require the controlled comparisons of sections 7.5 and 7.5.6. Quality and reuse remain constraints (table 7.7 and figure 7.6).

8.2 Two Deep Negative-Result Case Studies

8.2.1 Mailman-style shared sums: arithmetic reuse meets scatter cost

Mailman reuses finite-alphabet matrix–vector sums [20]. We test atomic buckets, private histograms and widths 4/6/8 against packed two-table scoring on frozen Vogue/arXiv/OpenAI-3072 streams, 128 probes, batches 8/1,000. Codes and mathematical scores stay fixed.

The best alternatives reach only 0.187–0.581 of baseline throughput (figure 8.1). Profiles spend most cycles in bucket accumulation: list-local bias repeats destinations; private histograms trade atomics for larger state and shared updates. Narrower groups help some variants, but none wins. This stopped these implementations, not all finite-alphabet reuse.

The pilot emits scores, not neighbours. FP32/FP64 cutoff differences therefore do not establish pipeline equivalence. With scorer ratio s and replaceable fraction f, $1/[(1-f)+f/s]$ is only a replacement model: integration stopped before end-to-end testing. A lossless-dictionary study also stopped at structural statistics. Neither supplies a measured system-frontier point.

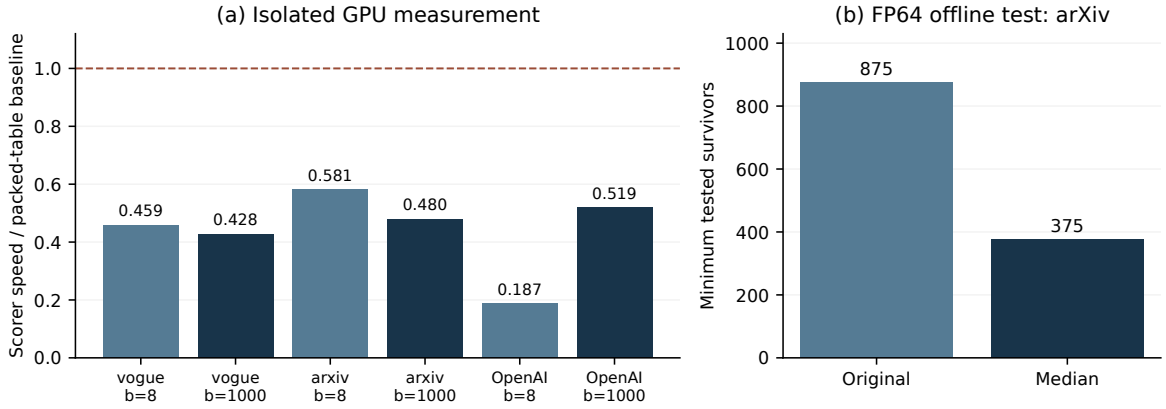


Figure 8.1. Two distinct evidence boundaries. Left: best measured Mailman scorer / packed-table scorer speed on an RTX 5090, with Vogue, arXiv and OpenAI-3072 at two batches and 128 probes; all ratios are below one. Right: minimum survivor counts meeting the original JHQ recall target on arXiv's 400-query FP64 offline test split. The candidate reduction is not a GPU speedup.

8.2.2 Rank-oriented coding: a local gain is not a general replacement

The ranking study replays exported candidates in FP64 with 400/200/400 calibration/validation/test queries. It tests weighted scores, median/list thresholds, learned losses and linear-MMSE decoding. Better reconstruction or inner-product MSE can still worsen nearest-candidate order; changing primary levels also changes residual reference points.

On arXiv test queries, median thresholds cut the minimum tested qualifying survivor count from 875 to 375; the other two workloads fail the same strict criterion. Rotation preserves corpus-mean norm, so Gaussian intuition does not remove bias. Centring may help, but must also preserve the refined-score ceiling. The improvement is workload-specific.

A separate GPU follow-up imports median thresholds/levels and retrains residuals. At arXiv's original $\alpha = 100$ target, gains are about 1.40/1.21 at batches 8/1,000. Vogue stops at recall 0.9630, below target 0.9644; OpenAI-3072 is tied. These three-round results use separate states/queries and are excluded from E1.

8.3 Threats to Internal and External Validity

Table 8.1 consolidates threats to the main interpretations. Chapter 3 gives controls; section 7.8 checks repeats, rebuilding and precision.

Table 8.1. Threats that can reverse or narrow a conclusion.

Claim at risk	Alternative or reversal	Control and remaining test
Execution gain	Changed scores, ties or index state explain an apparent speedup	Strict T4 CPU sorting and table-form diagnostics; finite coverage, with candidate dumps no longer retained
Budget quality	Finite-pool sampling or drift hides damaging queries	Disjoint GT evaluation, full traces, subset-sum checks and FULL enumeration; future-query and arbitrary-subset invariance remain untested
Competitive margin	Grid density, call tails or API overhead changes the ordering	Both recall estimators, retained slow calls and int8 baseline; TF32/FP32 sensitivity; T5 exposes small-batch reversals; cross-device comparisons remain unmeasured
Capacity/build claim	Transient allocations exceed a resident sample	Phase bytes, repeated empty-cache builds and 20 ms device peaks; sub-interval allocations and full retrained frontiers remain unmeasured

Coverage is one GPU, squared-L2 top-10 and six prepared datasets (section 7.2.5). Query removal does not exclude near duplicates; relevance labels are absent. Updates, concurrency, drift, service latency, billion-scale and multi-device capacity remain untested.

8.4 Testable Research Directions

8.4.1 Risk-controlled agreement with a reuse-aware objective

A future policy could minimise calibration-plus-reuse time with valid query-level reference-agreement bounds. It needs matched-regime counts, a fixed reference, explicit sampling/order assumptions and drift-triggered renewal. An absolute GT target additionally needs labels or justified reference-to-GT assumptions.

Compare fixed sampling, reusable rank curves and cost-aware selection on held-out pools, measuring total time and tail agreement. An older four-cell sequential controller lost because extra launches outweighed sample savings. Success must therefore save total time against a quality-matched control while satisfying the stated risk bound.

8.4.2 Capacity-aware residual placement

A residual cache could keep primary codes on-device and fetch missed residual blocks from host memory under a fixed device cap. Hold codes, routing, survivors and arithmetic fixed; vary cap, batch and locality. Measure transfer bytes, hits, overlap, tail latency and throughput. It must extend usable capacity at acceptable cost. This placement study is unimplemented; lowering α does not provide it.

8.5 Conditional Deployment Recommendation

For recall 0.97, E1 supports batch-1,000 throughput; T5 favours CAGRA fp32 on every batch-8 dataset. At batch 32, only OpenAI-3072's near tie changes with the estimator (section 7.2.5). Start with FULL, then validate deployment quality, reuse cost and memory (section 8.3).

9 Conclusion

9.1 Answers to the Four Research Questions

RQ1: Construction and capacity. GPU training, batched encoding, IVF placement and packed arrays produce a searchable resident hierarchy. The largest tested database contains 17.78 million vectors at 20.44 GiB resident device memory; the corresponding supplementary train-plus-add range is 10.54–10.65 s (section 7.7). Its sampled construction peak is 22.58 GiB, while BGE reaches 26.08 GiB at add entry.

RQ2: Correct and efficient execution. The inherited Cartesian code admits exact real-arithmetic table factorisation, and complete-key retention/merge and shared-bound arguments explain the grouped path's selection conditions. Two-group tables lead in 22/24 measured cells. Integrated scheduling achieves 2.727× geometric-mean throughput at batch 8 but only 1.015× at batch 1,000 against the lane-refinement reference (sections 7.3 and 7.6). Thus useful parallelism depends on batch and work distribution. T4's strict CPU reference matches all 55,296 grouped instances in complete-key order; 56 one-block set differences are exclusively cutoff ties (section 7.4.1).

RQ3: Refinement budget and quality. FULL calibration meets the tested relative recall-loss target in 12/12 cells, with maximum observed loss 0.0006. At $S = 128$ reaches 40/64 crossings in one cell (section 7.5). All FULL choices match enumeration, but ten distance-only cells lack a general nesting guarantee. Reference agreement is not population GT recall. Four direct reuse tests show workload-dependent quality/time gains.

RQ4: Competitive operating regions. At recall 0.97 and batch 1,000, JHQ leads on three datasets and is near RaBitQ on arXiv; Vogue needs grouping (section 7.2). OpenAI E1 uses TF32. Cached-model FP32 checks retain advantages over archived D baselines and contemporaneous T1 anchors, without FP32 retraining (section B.4.1). Int8 wins lower targets; RaBitQ wins some higher ones. CAGRA fp32 wins all batch-8 datasets; batch-32 near ties depend on the estimator (section 7.2.5).

9.2 Closing Principle and Outlook

Representation, execution and effort must share a quality target. Next steps are reusable risk-controlled calibration and capacity-aware residual placement (section 8.4).

Bibliography

[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.*, 9(4):288–299, 2015. doi: 10.14778/2856318.2856324.

[2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.*, 43 (5):1666–1677, 2021. doi: 10.1109/TPAMI.2019.2952606.

[3] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control, 2022. URL `https://arxiv.org/abs/2110.01052v5`. Version 5; Section 2.3.1.

[4] Anthropic. Claude: Implementation, experiment and review assistance record. Project AI-use record, 2026. Implementation and experiment records; original prompts and interaction metadata unavailable.

[5] Tony Assi. Vogue933k embeddings. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/tonyassi/vogue933k-embeddings`. Upstream attribution recorded in the project data preparation source.

[6] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. DARTH: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data*, 3 (4):242:1–242:26, 2025. doi: 10.1145/3749160. URL `https://arxiv.org/abs/2505.19001`.

[7] Artem Chirkin, Akira Naruse, Tamás Fehér, Yong Wang, and Corey Nolet. Accelerating vector search: NVIDIA cuVS IVF-PQ part 2, performance tuning, 2024. URL `https://developer.nvidia.com/blog/accelerating-vector-search-nvidia-cuvs-ivf-pq-performance-tuning-part-2/`. 18 July; accessed 28 October 2026.

[8] FAISS contributors. FAISS clustering parameters, version 1.9.0. `https://github.com/facebookresearch/faiss/blob/v1.9.0/faiss/Clustering.h`, 2024. Default minimum-points warning threshold; accessed 28 October 2026.

[9] FAISS contributors. *Index I/O, Cloning and Hyper-parameter Tuning*, n.d.. URL `https://github.com/facebookresearch/faiss/wiki/Index-IO,-cloning-and-hyper-parameter-tuning`. Auto-tuning the runtime parameters; accessed 28 October 2026.

[10] FAISS contributors. *IndexRefine: Candidate Expansion and Refinement*, n.d.. URL `https://faiss.ai/cpp_api/file/IndexRefine_8h.html`. C++ API documentation; accessed 28 October 2026.

[11] Jianyang Gao and Cheng Long. RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2405.12497`. SIGMOD 2024 paper.

[12] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2409.09913`.

[13] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization for approximate nearest neighbor search. In *IEEE CVPR*, pages 2946–2953, 2013. doi: 10.1109/CVPR.2013.379. URL `https://openaccess.thecvf.com/content_cvpr_2013/html/Ge_Optimized_Product_Quantization_2013_CVPR_paper.html`.

[14] Oded Green, Robert McColl, and David A. Bader. GPU Merge Path: A GPU merging algorithm. In *Proceedings of the 26th ACM International Conference on Supercomputing*, pages 331–340, 2012. doi: 10.1145/2304576.2304621. URL `https://davidbader.net/publication/2012-gm-ba/2012-gm-ba.pdf`.

[15] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3887–3896, 2020. URL `https://proceedings.mlr.press/v119/guo20h.html`.

[16] Jiabao Han, Mengxuan Zhang, and Goce Trajcevski. JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.*, 19(7):1530–1543, 2026. doi: 10.14778/3801059.3801067.

[17] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[18] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, 2017. URL `https://arxiv.org/abs/1702.08734`.

[19] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. Improving approximate nearest neighbor search through learned adaptive early termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020. doi: 10.1145/3318464.3380600. URL `https://www.pdl.cmu.edu/PDL-FTP/BigLearning/mod0246-liA.pdf`.

[20] Edo Liberty and Steven W. Zucker. The Mailman algorithm: A note on matrix–vector multiplication. *Information Processing Letters*, 109(3):179–182, 2009. doi: 10.1016/j.ipl.2008.09.028. URL `https://www.cs.yale.edu/publications/techreports/tr1402.pdf`.

[21] Rui Luo and Mengxuan Zhang. JHQ-GPU: Representation-driven GPU execution and workload-adaptive hierarchical refinement. Project manuscript source, 2026. Project manuscript.

[22] National Institute of Standards and Technology. *Dataplot: Exact Binomial Confidence Limits*, n.d.. URL `https://itl.nist.gov/div898/software/dataplot/refman2/auxillar/exacbino.htm`. One-sided exact binomial limits; accessed 28 October 2026.

[23] National Institute of Standards and Technology. *Dataplot: Hypergeometric Probability Density Function*, n.d.. URL `https://www.itl.nist.gov/div898/software/dataplot/refman2/ch8/hyppdf.pdf`. Sampling without replacement; accessed 28 October 2026.

[24] NVIDIA. *NVIDIA Blackwell Tuning Guide*, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/blackwell-tuning-guide/index.html`. Section 1.4.1.1; accessed 28 October 2026.

[25] NVIDIA. *CUDA C++ Programming Guide, Release 13.0.1*. NVIDIA, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/cuda-c-programming-guide/index.html`. Sections 10.14.1.4 and 18.5.3.3; accessed 27 October 2026.

[26] NVIDIA. *PTX ISA, Version 9.0*, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/parallel-thread-execution/index.html`. Sections 8.1, 8.4.2, 8.7 and 8.10.3; accessed 28 October 2026.

[27] NVIDIA. *CUDA Programming Guide: The CUDA Platform*, 2026. URL `https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/cuda-platform.html`. PTX compatibility and just-in-time compilation; accessed 28 October 2026.

[28] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *IEEE ICDE*, 2024. doi: 10.1109/ICDE60146.2024.00323.

[29] OpenAI. ChatGPT: Project review assistance record. Project AI-use record, 2026. Retained review excerpts; original prompts and interaction metadata unavailable.

[30] OpenAI. Codex: Selected thesis-preparation interactions. Project AI-use record, 2026. Selected prompts and uses; compiled 29 October 2026. Partial record; historical interaction metadata unavailable.

[31] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 1536 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-1536-1M`. Accessed 28 October 2026.

[32] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 3072 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-3072-1M`. Accessed 28 October 2026.

[33] Jifan Shi, Jianyang Gao, James Xia, Tamás Béla Fehér, and Cheng Long. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.*, 19(11):3454–3467, 2026. doi: 10.14778/3836663.3836701.

[34] The Alexandria Index. arXiv abstracts dataset. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/macrocosm/arxiv_abstracts`. Accessed 28 October 2026; exported embedding preparation is project-specific.

[35] The Information Engineering Lab. Stella TREC24 BioGen embedding. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/ielabgroup/stella_trec24_biogen_embedding`. Corpus split; accessed 28 October 2026.

[36] Upstash. Wikipedia 2024-06 BGE-M3. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/Upstash/wikipedia-2024-06-bge-m3`. Italian configuration; accessed 28 October 2026.

[37] Karthik Venkatasubba, Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. BANG: Billion-scale approximate nearest neighbour search using a single GPU. *IEEE Transactions on Big Data*, 11(6):3142–3157, 2025. doi: 10.1109/TBDATA.2025.3581085. URL `https://ssomesh.github.io/docs/papers/tbd25.pdf`.

[38] Jingrong Zhang, Akira Naruse, Xipeng Li, and Yong Wang. Parallel top-k algorithms on GPU: A comprehensive study and new methods. In *SC '23*, 2023. doi: 10.1145/3581784.3607062. URL `https://sc23.supercomputing.org/proceedings/tech_paper/tech_paper_pages/pap294.html`.

[39] Weijie Zhao, Shulong Tan, and Ping Li. SONG: Approximate nearest neighbor search on GPU. In *IEEE ICDE*, pages 1033–1044, 2020. doi: 10.1109/ICDE48307.2020.00094. URL `https://www.cs.rit.edu/~wjz/papers/conference/2020-icde-song.pdf`.

A Reproducibility and Experiment Manifest

This appendix specifies the experimental environment and measurement protocol. Available source and measurement materials are listed in section B.5.

A.1 Source, Environment and Trained States

Table A.1 lists run-header/report versions. Environment and JIT settings follow Chapter 3; historical executable and model fingerprints remain incomplete.

Data/GT follow table 3.2. Sweeps reuse trained caches; cold builds do not. Isolated large-baseline reruns replace contaminated builds. Corrected Stella search uses 999 queries; skipped batch-1,000 runs supply no points.

A.2 Measurement Contracts

E1 takes the median of five round QPS values, normally 24 calls each. Warm-up classification reduces calls at 274 points; its first-call conversion/validation interval differs from steady-state timing. Host staging, int8 quantisation and transfers are timed; API/output-memory differences remain. Device-only times are auxiliary.

The 369 repeat-ID flags include tie-order changes; 401 also counts cross-protocol disagreement. Flags alone do not exclude points, and JHQ lacks the same flag. Cold timers exclude constructor/host QR/warm-up. Build sampling includes context at 20 ms; synchronised one-buffer diagnostics are separate.

A.3 Budget Harness Contract

The campaign overrides the fallback grid starting at 100 with one starting at 200. Pools/draws follow Chapter 3. Initial warm-up visits all queries but supplies neither GT nor outputs to agreement; scan calibration uses separate database pseudoqueries.

Fixed arms use one warm-up and three timed calls. Calibration times searches plus $O(Sk^2)$ host comparison, bypassing graph capture; gathering and scan calibration are excluded. p95 uses index $\lfloor 0.95(64-1) \rfloor$. The helper returns an index, so callers must apply `set_alpha(A[index])`. Original selected-arm rates reuse fixed-arm timings.

Original enumeration covers 2,304 draws; supplementary checks add twelve FULL decisions. All choices and 18,432 subset-sum grid counts match. General ordering/invariance conditions remain those of section 6.3 and equation (6.9); GT-risk mapping uses held-out labels.

A.4 Supplementary Evidence

A1 separates first-read, warm-input and synchronised builds. C1 checks four layouts and sampled recoding; FP64 norms use stored rotations, not GPU rotation outputs. A3 checks all OpenAI queries and 10,299 base rows without original entity IDs.

Table A.1. Recorded component provenance. Report and script identities are distinguished from raw run headers.

Component	Recorded source evidence
E1 final pipeline	Environment logs include `a31c22d`; extensions retain separate metadata
Lane refinement (v61)	Raw header `b37efa4`
Grouped scan (v62)	Report `862543a`; run script `e0d0518`
Shared bound (v63)	Source and build flags archived; no inferred run head
Batch scheduling (v65)	Raw header `d425ab8`
Radix probe (v69)	Comparative runs and source archived; binary SHA unrecorded
Budget / table sweep	Raw header `d5e1526`
Large int8 rerun	Stella degree-64 metadata `ad433a4`
Historical budget	Raw header `bf31b41`, v57 family

A2 reassembles extracted sm_80 PTX with CUDA 13.0.88 `ptxas` for sm_120. Aligned eight-byte key accesses support the PTX contract; toolkit SASS is not captured driver-JIT code.

D varies add/query GEMMs on TF32-trained caches against archived baselines. T1 supplies separate same-session anchors (section B.4.1); neither retrains FP32 models.

B reproduces `srand(12345)` decisions. Direct reuse instead uses a separate seed, five rotated-policy rounds and 500 repeated batches. Its timer includes gathering, calibration, budget installation and first-graph setup, but excludes build, scan calibration and warm-up. Schedule startup is reported separately.

A.5 Strict Selection, Small Batches and Artifact Access

T4 v73 differs from v71 by two export blocks. GPU-produced scores feed CPU sorting; bitwise equality covers returned valid primary entries, and padding checks cover valid counts. Table-form checks fix 128 probes, $\alpha = 100$, 256 queries, shared bounds, $g_{\text{scan}} \in \{0, 2\}$ and $g_{\text{LUT}} \in \{1, 2, 4, 8\}$. Candidate dumps were deleted; summaries retain finite counts. Empty/adversarial lists and the unexplained cross-process Vogue discrepancy are separate questions.

T5 uses fixed small synchronous batches and five host-to-host rounds, including null-call costs and int8 query quantisation. JHQ/CAGRA reuse E1 grids; RaBitQ has a separate sweep. Both threshold estimators retain qualifying endpoints. APIs/warm-ups differ; service concurrency is untested.

GPU reproduction requires original vectors, matching trained states and builds, and the specified environment. Section B.5 identifies which materials are available.

B Supplementary Results and Correctness Checks

Parameter coverage and secondary checks follow below; protocol is in Appendix A.

B.1 Parameter Coverage

The E1 comparison contains 1,590 valid measured configurations, with coverage in table B.1. The sets below describe observed parameter values; their Cartesian products are not assumed to have been measured.

Table B.1. Exact E1 configuration counts, including coverage extensions. Counts are parameter points, not independent index builds.

Family	Vogue	arXiv	OpenAI-1536	OpenAI-3072	Total
JHQ-GPU v69	27	29	37	39	132
CAGRA fp32	64	64	64	64	256
CAGRA int8	64	64	64	64	256
IVF-RaBitQ	132	132	264	198	726
IVF-PQ	54	54	54	54	216
GPU brute force	1	1	1	1	4
Total	342	344	484	420	1590

JHQ-GPU. All E1 points use $B = B_r = 8$, $M = d/8$, $k = 10$ and batch 1,000. Vogue uses 4,096 IVF lists; the other three use 8,192. Vogue observes probes 16/32/64/96/128/192/256/384/512 and $\alpha \in \{16, 32, 64, 128\}$ over 27 tuples. ArXiv adds probe 1,024 and uses $\alpha \in \{25, 50, 100, 200\}$ over 29 tuples. Both OpenAI grids use probes 32/64/128/256/512/1,024; their observed multipliers are respectively $\{4, 8, 16, 32, 64, 128, 256, 512\}$ and $\{2, 4, 8, 16, 32, 64, 128, 256\}$, over 37 and 39 tuples. The reported scheduled path uses four groups on Vogue and one block elsewhere. OpenAI uses TF32 rotation/centroid GEMMs in historical v69; the v72 FP32 sensitivity is a separate family (section 7.2.3).

CAGRA fp32 and int8. Each representation has 64 points per dataset: graph degrees 32/64, intermediate degree 128, `ivf_pq` construction, `itopk_size` in 32/64/96/128/192/256/384/512 and search width 1/2/4/8. Int8 records quantile 0.99 for dataset scaling and includes query quantisation in the host search boundary.

IVF-PQ. Eight-bit subcodes use 192/384/768-byte codes at dimension 768, and 384/768/1,536-byte codes on OpenAI. Each length has fp16/fp32 lookup tables and probes 16/32/64/96/128/192/256/384/512: 54 points per dataset. The recorded refinement ratio is one and list counts match JHQ. Code lengths and memory budgets differ.

IVF-RaBitQ and brute force. RaBitQ sweeps 4/8/9 bits and probes 16/32/64/96/128/192/256/384/512/768/1,024. Modes are LUT16/LUT32/QUANT4/QUANT8 except OpenAI-3072 omits LUT16. OpenAI adds 4,096-list indexes to default 8,192-list coverage; exact combinations remain in the registry. Each dataset has one FP32 squared-L2 brute-force point. Missing coverage is not failure.

Auxiliary grids. LUT sweeps six datasets, four probe counts (8/32/128/512) and $g_{\text{LUT}} \in \{1, 2, 4, 8\}$ at $\alpha = 100$: 24 cells, 96 arms. Budget evaluation uses probes 128/512 and $A = (200, 100, 64, 32, 16, 8, 4, 2)$ on the same six datasets, with 64 draws at each $S \in \{32, 64, 128\}$ and $\epsilon = 1$. FULL separately uses all 500 calibration queries. Held-out counts are 500, or 499 for Stella. Boundaries are specified in sections A.3 and A.4.

B.2 Correctness and Numerical Evidence

Table B.2 separates numerical, algorithmic and memory evidence. Chapter 5's ordering arguments concern computed scores under their stated contracts; raw-vector recall is evaluated separately.

Table B.2. Validation coverage and the object actually checked.

Boundary	Completed evidence and scope
Representation	Four all-row offset/ID audits; sampled raw-input FP64 recoding and stored-matrix geometry; TF32 deviations identified
Cartesian LUTs	Real-arithmetic factorisation; 96 timed arms; T4 per-candidate FP32 error and ranking diagnostics across four table forms
Global selection	T4: 270 configurations, 69,120 query instances; 55,296 grouped instances match strict CPU complete-key sorting in order
Shared bound	48 paired checks; extracted scalar 64-bit PTX and toolkit SASS support whole-key access; actual driver SASS not captured
Scheduling	250 variant/configuration and 234,320 query comparisons, retaining distinctions between ID sets and tie order
Probe selector	1,792 synthetic cases with zero failures; real-query ID/offset checks, relative to computed routing scores
Budget search	2,304 monotone sampled curves and 12 FULL enumeration matches; 18,432 sampled grid counts reproduce full-pool subset sums

The smaller representation-audit databases have identical primary and residual bytes across one-/two-pass builds when joined by ID. Corrections differ by at most 8.94×10^{-8} and 1/1/40 rows change list assignment; Stella validates its natural two-pass path (section 4.6.2). T4 supplies the strict computed-score selection reference (section A.5). The controlled same-model Vogue cross-process diagnosis remains unperformed: archived processes with matching probe/primary dumps still report final recall 0.9642/0.9644. Within-run selection checks do not localise that separate anomaly.

B.3 Secondary Results and Evidence Status

Adverse regimes. Worst scheduling regret is 1.217 at batch 256; Vogue E1 ratios span 0.895–1.175 (section 7.6). T5 CAGRA fp32 wins every batch-8 dataset at recall 0.97; batch-32 ties depend on the estimator (section 7.2.5).

B.4 Construction Capacities and Precision Anchors

Table B.3 separates staging from assignment scratch. Defaults use three buffers, 65,536 add rows and 128 MiB per input/rotation array; these are capacities, not current batch sizes or process peaks.

Table B.3. Default construction-buffer capacities. The last column is $4n_b n_{\text{list}} a_{\text{assign}}/2^{30}$; substituting a_{in} would give an incorrect general model.

d	n_{list}	a_{in}	a_{assign}	Dots GiB
768	4,096	43,690	8,192	0.375
1,024	32,768	32,768	32,768	12.000
1,536	8,192	21,845	32,768	3.000
3,072	8,192	10,922	32,768	3.000

B.4.1 Contemporaneous precision anchors

T1 remeasures JHQ and baselines near recall 0.97 in one session with E1 pools. Twelve configuration pairs per OpenAI dataset bracket baselines with two five-round JHQ repetitions; QPS combines their medians geometrically. FP32/TF32 means are 0.956/0.964. Both reuse TF32-trained caches and change add/query arithmetic.

Table B.4. T1 at recall 0.97 and batch 1,000. Baseline columns report interpolation / strict measured threshold; the FP32/TF32 column is interpolated. All methods here are contemporaneous; these values are separate from D's cross-session baseline ratios.

Dataset	FP32/TF32 at 0.97	FP32 / RaBitQ	FP32 / CAGRA
OpenAI-1536	0.980	1.126 / 1.053	1.752 / 1.633
OpenAI-3072	0.973	1.328 / 1.316	1.825 / 2.019

Both estimators retain numerical FP32 advantages. OpenAI-3072's RaBitQ bracket (0.9669–0.9796 recall) exposes grid sensitivity. These anchors neither replace E1 nor retrain FP32 models.

B.5 Artifact Access

The project artifact archive provides source/measurement snapshots and selected T4/T5, T1 and A2 inputs. Complete vectors, model states and historical executables are absent; GPU reproduction needs those plus the matching environment.